

SOSA JUAN PABLO

TUS PRIMEROS PASOS EN LA PROGRAMACIÓN,
TU NUEVO MUNDO.

PYTHON DESDE CERO,
FUNDAMENTOS
CLAROS



@copyleft2024 - juanpablososa@gmail.com

Python desde cero, fundamentos claros.

Autor: Juan Pablo Sosa, juanpablososa@gmail.com

Última edición Marzo 2025 v25.03.08

Este libro es gratuito y está disponible para su descarga en

https://drive.google.com/drive/folders/19taxx_ebNiZoUHLvEYvTx0-CzYL-DfbF?usp=sharing

Además, está licenciado bajo copyleft <https://www.gnu.org/licenses/copyleft.es.html>, lo que significa que tienes la libertad de compartir, copiar y redistribuir el libro en cualquier medio o formato, así como de adaptarlo y modificarlo según tus necesidades. También puedes editar el contenido libremente y crear nuevas obras basadas en este libro.

Te animamos a aprovechar esta libertad para adaptar el libro a tus necesidades específicas, compartirlo con otros y contribuir al conocimiento abierto y colaborativo.

Es un libro vivo, es decir que tiene actualizaciones constantes, y voy detallando su última versión y copiando a su versión en PDF solamente la última disponible.

¡Disfruta del libro y del proceso de aprendizaje!

UNIDAD 1

INTRODUCCION A PYTHON Y A LA PROGRAMACION

En este módulo aprenderemos sobre:

Los fundamentos de la programación, es decir, como funcionan las computadoras, como se ejecutan los programas y, como se definen y construyen los lenguajes de programación.

- La diferencia entre un lenguaje de programación compilado e interpretado.
- Tipos de lenguaje de programación. ¿Qué son los algoritmos?
- Qué es Python y qué herramientas utilizar para trabajar.
- ¿Qué es y Cómo funciona un programa?

Los programas hacen que una computadora sea útil; sin programas, una computadora, incluso la más potente, no es más que un pisapapeles, o cualquier otra cosa parecida.

Definición de programa:

Un programa informático o programa de computadora es una secuencia de instrucciones u órdenes basadas en un lenguaje de programación que una computadora interpreta para resolver un problema o una función específica.

Pero, ¿ entonces qué es un lenguaje de programación ?

Un lenguaje de programación es un conjunto de reglas gramaticales, que instruyen a que un ordenador o dispositivo se comporte de una cierta manera. Cada lenguaje de programación tiene un vocabulario, un conjunto único de palabras clave que sigue a una sintaxis especial para formar y organizar instrucciones del ordenador.

Y acá vamos a ver una diferencia entre lo que son lenguajes formales y lenguajes naturales.

Un **lenguaje formal** tiene símbolos, letras y números. Y tienen reglas específicas para unir todos esos símbolos. Los símbolos son el alfabeto y las reglas, la sintaxis o la gramática.

Los **lenguajes naturales** están llenos de ambigüedades, puede haber palabras con diferentes significados. Al contrario en los formales los símbolos están bien definidos y las reglas nos permiten crear enunciados con significados concretos.

Los lenguajes de programación se utilizan para crear algoritmos los cuales pueden controlar el funcionamiento de una máquina.

Es la forma en que podemos decirle a las máquinas como queremos que se comporten.

Aquí tienes una versión mejorada del texto, con mayor claridad y fluidez. También te incluyo ideas para dibujos que podrías agregar al libro para enriquecer la experiencia visual del lector.

Definición de Algoritmo

Un algoritmo, en términos generales y sin entrar todavía en programación, puede entenderse como un conjunto de instrucciones ordenadas que permiten alcanzar un objetivo o resolver un problema. En esencia, son los pasos necesarios para llevar a cabo una tarea.

Por ejemplo, pensemos en la preparación de una torta. Para lograrlo, debemos seguir ciertas etapas: identificar los ingredientes necesarios, comprarlos, medirlos con precisión y seguir una serie de pasos para mezclarlos, cocinarlos y obtener el resultado deseado.

En el contexto de la programación, un algoritmo es el primer paso que desarrollamos al crear un programa. Un programa, a su vez, puede describirse como un conjunto de algoritmos organizados de forma lógica y sistemática para ser interpretados y ejecutados por una computadora.

Características y Estructura de los Algoritmos

Independientemente de su clasificación o tipo, todos los algoritmos comparten una estructura común formada por tres partes esenciales:

1. **Entrada:** Es el conjunto de datos iniciales que el algoritmo necesita para funcionar.
2. **Proceso:** Son las operaciones o pasos que el algoritmo aplica a los datos de entrada para transformarlos.
3. **Salida:** Es el resultado generado por el algoritmo tras completar el proceso.

Además, existen ciertas características fundamentales que todo algoritmo debe cumplir:

- **Exactitud:** Cada paso debe estar claramente definido, con un orden preciso que no deje lugar a ambigüedades.
- **Definición:** Si el algoritmo se ejecuta varias veces con los mismos datos de entrada, debe producir siempre los mismos resultados.
- **Complejidad:** El algoritmo debe considerar todas las posibles situaciones o escenarios asociados al problema.

- **Finitud:** Todo algoritmo debe tener un número limitado de pasos; es decir, debe terminar en algún momento.
- **Claridad:** Las instrucciones que lo componen deben ser simples, comprensibles y ejecutables por quien lo implemente.



(Qué es un algoritmo)

Diccionario del Programador. Kiko Palomares explica de manera sencilla términos básicos para comprenderlos bien cuando comenzamos en el mundo de la programación.

Links:

https://www.youtube.com/watch?v=P4f5OEPz2LY&list=PLnunbwZjHqMMo9fDBBC9-eLJefsJyM7jr&index=4&t=28s&ab_channel=KikoPalomares



Y del canal de @AbsoluteSite

<https://www.youtube.com/watch?v=sQLn2asTefo&t=25s&end=2m15s>

(25s - 2m15s)

Tipos de lenguajes de programación:

Los lenguajes de programación se dividen en dos categorías principales: **lenguajes de bajo nivel** y **lenguajes de alto nivel**. Cada uno tiene sus propias características, usos y ventajas.

Lenguajes de Bajo Nivel

Los lenguajes de bajo nivel están estrechamente ligados al hardware de la computadora. Esto significa que sus instrucciones ejercen un control directo sobre los componentes físicos del sistema. Aunque pueden ser más difíciles de aprender y usar, ofrecen una velocidad y eficiencia superior debido a su proximidad al hardware.

Características:

- **Dependencia de la máquina:** Están diseñados para funcionar en una arquitectura de hardware específica.

- **Alta eficiencia:** Ofrecen un rendimiento rápido y óptimo al interactuar directamente con el hardware.
- **Complejidad:** Su uso requiere un conocimiento profundo de la estructura interna de la computadora.

Un ejemplo típico de un lenguaje de bajo nivel es el **lenguaje ensamblador**. Este lenguaje se utiliza principalmente para programar funciones críticas de los sistemas operativos, controladores de dispositivos y aplicaciones en tiempo real, donde el rendimiento es crucial.

Dejamos un ejemplo de lo que es por ejemplo el lenguaje ensamblador, como escribimos en bajo nivel.

004012A7	90	NOP	
004012A8	53	PUSH EBX	
004012A9	56	PUSH ESI	
004012AA	57	PUSH EDI	
004012AB	8BF2	MOV ESI,EDX	
004012AD	8BD8	MOV EBX,EAX	
004012AF	85F6	TEST ESI,ESI	
004012B1	8BFB	MOV EDI,EBX	
004012B3	74 35	JE SHORT RTRACE.004012EA	
004012B5	6A 04	PUSH 4	
004012B7	68 00100000	PUSH 1000	
004012BC	68 00100000	PUSH 1000	
004012C1	53	PUSH EBX	
004012C2	E8 15870000	CALL <JMP.&KERNEL32.VirtualAlloc>	Protect = PAGE_READWRITE AllocationType = MEM_COMMIT Size = 1000 (4096.) Address VirtualAlloc
004012C7	85C0	TEST EAX,EAX	
004012C9	75 0F	JNZ SHORT RTRACE.004012DA	
004012CB	8BD3	MOV EDX,EBX	
004012CD	8BC7	MOV EAX,EDI	
004012CF	2BD7	SUB EDX,EDI	
004012D1	E8 1E000000	CALL RTRACE.004012F4	
004012D6	33C0	XOR EAX,EAX	
004012D8	EB 15	JMP SHORT RTRACE.004012EF	
004012DA	81C3 00100000	ADD EBX,1000	
004012E0	81EE 00100000	SUB ESI,1000	
004012E6	85F6	TEST ESI,ESI	
004012E8	75 CB	JNZ SHORT RTRACE.004012B5	
004012EA	B8 01000000	MOV EAX,1	
004012EF	5F	POP EDI	
004012F0	5E	POP ESI	
004012F1	5B	POP EBX	
004012F2	C3	RETN	
004012F3	90	NOP	

Lenguajes de Alto Nivel

Los lenguajes de alto nivel son más abstractos y están diseñados para ser independientes del hardware. Esto significa que el mismo código puede ejecutarse en diferentes tipos de computadoras sin necesidad de modificaciones significativas. Estos lenguajes son más fáciles de aprender y usar, ya que sus instrucciones se asemejan más al lenguaje humano.

Características:

- **Independencia de la máquina:** Pueden ejecutarse en múltiples plataformas sin cambios significativos.
- **Facilidad de uso:** Ofrecen una sintaxis más amigable y cercana al lenguaje humano, lo que facilita el aprendizaje y la escritura de código.
- **Flexibilidad:** Son adecuados para una amplia gama de aplicaciones, desde desarrollo web hasta inteligencia artificial.

Los lenguajes de alto nivel, como Python, permiten a los programadores enfocarse más en resolver problemas lógicos que en los detalles técnicos del hardware. Estos lenguajes suelen requerir la traducción a lenguaje de máquina para ser ejecutados, lo que se realiza mediante dos procesos principales:

- **Compilación:** El código fuente se traduce completamente antes de su ejecución, generando un archivo binario (por ejemplo, un archivo `.exe` en Windows). Este archivo puede distribuirse y ejecutarse sin necesidad del código fuente original. El **compilador** es el programa que realiza esta traducción, permitiendo que el código sea ejecutado directamente por la máquina.
- **Interpretación:** El código fuente se traduce línea por línea durante su ejecución. Este método permite una mayor flexibilidad, ya que los cambios en el código pueden probarse de inmediato. El **intérprete** es el programa que ejecuta el código directamente, traduciéndolo a medida que se necesita.

Ambos métodos tienen sus pros y contras. La compilación suele ser más rápida en la ejecución, mientras que la interpretación permite una mayor flexibilidad durante el desarrollo.

Comparación entre Compilación e Interpretación en Lenguajes de Programación

Método	Pros	Contras
Compilación	- La ejecución del código traducido es generalmente más rápida, ya que se ejecuta directamente en lenguaje máquina. - El usuario final no necesita el compilador, solo el archivo ejecutable.	- La compilación puede ser un proceso lento, especialmente para proyectos grandes, impidiendo la ejecución inmediata tras cada modificación. - Es necesario disponer de compiladores específicos para cada plataforma de hardware en la que se desee ejecutar el programa.
Interpretación	- Permite ejecutar el código inmediatamente después de escribirlo, sin una fase de traducción adicional. - El código fuente es independiente del hardware, lo que facilita su ejecución en diferentes sistemas.	- El desempeño puede verse afectado, ya que el intérprete consume recursos al traducir el código en tiempo de ejecución. - El usuario final necesita tener el intérprete para poder ejecutar el programa, lo que puede complicar la distribución.



(Lenguajes compilados VS interpretados)

Diccionario del Programador. Kiko Palomares explica de manera sencilla términos básicos para comprenderlos bien cuando comenzamos en el mundo de la programación.

Links:

<https://www.youtube.com/watch?v=OJ-vZKPlqk8>

¿Qué es Python?

Python es un lenguaje de programación de alto nivel, interpretado, orientado a objetos, de código abierto, y de uso generalizado con semántica dinámica. Se utiliza ampliamente para la programación de propósito general, abarcando desde desarrollo web hasta análisis de datos, inteligencia artificial, automatización en servidores y más.

Explicamos cada detalle mencionado:

- **Alto Nivel:** Python es un lenguaje de alto nivel, lo que significa que está más cerca del lenguaje humano y más alejado del lenguaje máquina. Esto hace que sea más fácil de leer, escribir y mantener, en comparación con lenguajes de bajo nivel como el ensamblador. Los lenguajes de alto nivel permiten a los programadores centrarse en la lógica del problema en lugar de en los detalles del hardware.
- **Interpretado:** Python es un lenguaje interpretado, lo que significa que el código fuente es ejecutado línea por línea por un intérprete. Esto permite una mayor flexibilidad y facilidad de depuración, ya que no es necesario compilar el código antes de ejecutarlo. Los cambios se pueden probar de inmediato, lo que acelera el proceso de desarrollo.
- **Orientado a Objetos:** Python admite la programación orientada a objetos (POO), un paradigma que organiza el software en "objetos", que son instancias de "clases". Este enfoque permite una estructuración más modular y reutilizable del código, facilitando la creación y gestión de aplicaciones complejas.
- **Código Abierto:** Python es de código abierto, lo que significa que su código fuente está disponible para que cualquiera lo use, modifique y distribuya. Esta característica ha fomentado una gran comunidad de desarrolladores que contribuyen a su crecimiento y evolución constante.
- **Uso Generalizado:** Python es un lenguaje de propósito general, lo que implica que no está limitado a un tipo específico de desarrollo. Puede ser utilizado en múltiples áreas como desarrollo web, ciencia de datos, automatización de tareas, desarrollo de software, inteligencia artificial, entre otros.
- **Semántica Dinámica:** Python utiliza semántica dinámica, lo que significa que la vinculación de tipos ocurre en tiempo de ejecución en lugar de en tiempo de compilación. Esto permite una mayor flexibilidad en la programación, aunque puede introducir ciertos desafíos en términos de rendimiento y detección de errores en comparación con lenguajes de tipado

estático.

Python fue creado por Guido van Rossum a finales de los años 80 y ha evolucionado gracias a la contribución de una vasta comunidad de programadores. Aunque el nombre pueda recordar a la serpiente pitón, el lenguaje en realidad toma su nombre de "Monty Python's Flying Circus", una popular serie de comedia británica.



(Conocemos un poco de la vida de Guido Van Rossum)

<https://www.youtube.com/watch?v=D29POINahlg> (1m30s)

¿Qué es código abierto?

El **software de código abierto** es un tipo de programa informático desarrollado y mantenido de manera colaborativa. Su principal característica es que su **código fuente está disponible para el público**, lo que permite que cualquiera pueda **usarlo, estudiarlo, modificarlo y redistribuir** según sus necesidades. Este modelo fomenta la transparencia y la innovación, ya que una comunidad global de desarrolladores puede contribuir a mejorar el software continuamente.

En contraste, el **software propietario** (o de **código cerrado**), como *Microsoft Word* o *Adobe Illustrator*, es creado y distribuido por una empresa o titular de derechos de autor. En estos casos, el código fuente está protegido y no se puede modificar, mejorar ni redistribuir sin el permiso explícito del propietario.

El código abierto no solo es una filosofía de desarrollo, sino que también es un motor clave en el avance tecnológico. Herramientas ampliamente usadas como **Linux**, **Python**, y **Git** son ejemplos de proyectos de código abierto que han revolucionado la industria. También tenemos muchos lenguajes de programación open source, como ser los más conocidos: PHP, Javascript, Ruby, Rust, Go, etc



¿Qué es Open Source o código abierto?

https://www.youtube.com/watch?v=DcticRZyUSg&ab_channel=Arsys

¿Qué necesitamos para comenzar a trabajar con Python?

Para desarrollar programas en Python, necesitamos algunas herramientas básicas que nos ayudarán a escribir, probar y depurar nuestro código de manera eficiente. A continuación, te explicamos lo esencial para empezar:

1. Editor de Texto o IDE (Entorno de Desarrollo Integrado)

Un editor de texto o un IDE (Integrated Development and Learning Environment) es el lugar donde escribirás tu código en Python. Los IDEs ofrecen herramientas integradas que facilitan el desarrollo de software, como resaltado de sintaxis, autocompletado de código, depuradores, control de versiones, y más. Algunos de los editores e IDEs más populares para Python son:

- **Visual Studio Code:** Un editor de código gratuito y muy extendido, con extensiones para casi cualquier lenguaje, incluyendo Python.
- **PyCharm:** Un IDE específico para Python que ofrece potentes herramientas de desarrollo.
- **Sublime Text:** Conocido por su velocidad y flexibilidad.
- **Atom:** Un editor de código personalizable y fácil de usar.
- **Vim:** Un editor de texto avanzado para usuarios experimentados.
- **IDLE:** El entorno de desarrollo oficial de Python, simple y adecuado para principiantes.

2. Consola o Terminal

La consola es donde ejecutas tus programas de Python. Aquí puedes correr tus scripts, interactuar con el intérprete de Python en tiempo real y detener la ejecución de programas si algo sale mal. La consola es una herramienta esencial para probar rápidamente fragmentos de código y ver resultados inmediatos.

3. Depurador (Debugger)

Un depurador es una herramienta que te permite ejecutar tu código paso a paso, inspeccionando el estado de las variables en cada punto de la ejecución. Esto es invaluable para identificar errores lógicos y comprender cómo se está ejecutando tu código. Muchos IDEs, como PyCharm y Visual Studio Code, vienen con depuradores integrados.

Versiones de Python: Python 2 vs. Python 3

Existen dos versiones principales de Python: Python 2 y Python 3. Es importante conocer la diferencia entre ellas:

- **Python 2:** Esta es una versión más antigua que dejó de ser mantenida oficialmente en 2020. A pesar de esto, todavía se utiliza en algunos entornos heredados. Sin embargo, no se recomienda comenzar nuevos proyectos en esta versión.
- **Python 3:** Es la versión actual de Python y la que se utiliza en la mayoría de los proyectos modernos. Python 3 incluye muchas mejoras y nuevas características que no están presentes en Python 2, y es la versión que usaremos a lo largo de esta materia.

En todas las prácticas y ejemplos de este libro utilizaremos Python 3, garantizando que trabajes con las versiones más recientes del lenguaje y aproveches sus últimas características y mejoras.

Descarga e instalación de Python3

Es probable que los usuarios de Linux y macOS ya tengan Python instalado, ya que este lenguaje es ampliamente utilizado en muchos componentes de ambos sistemas operativos. Sin embargo, es importante verificar qué versión está disponible antes de continuar.

Para comprobar si Python está instalado en tu sistema, abre una terminal o consola y escribe uno de los siguientes comandos:

```
python --version
```

o, en el caso de Linux y macOS, donde a menudo Python 3 está instalado como ``python3``, puedes usar:

```
python3 --version
```

Si Python está instalado, verás algo similar a esto:

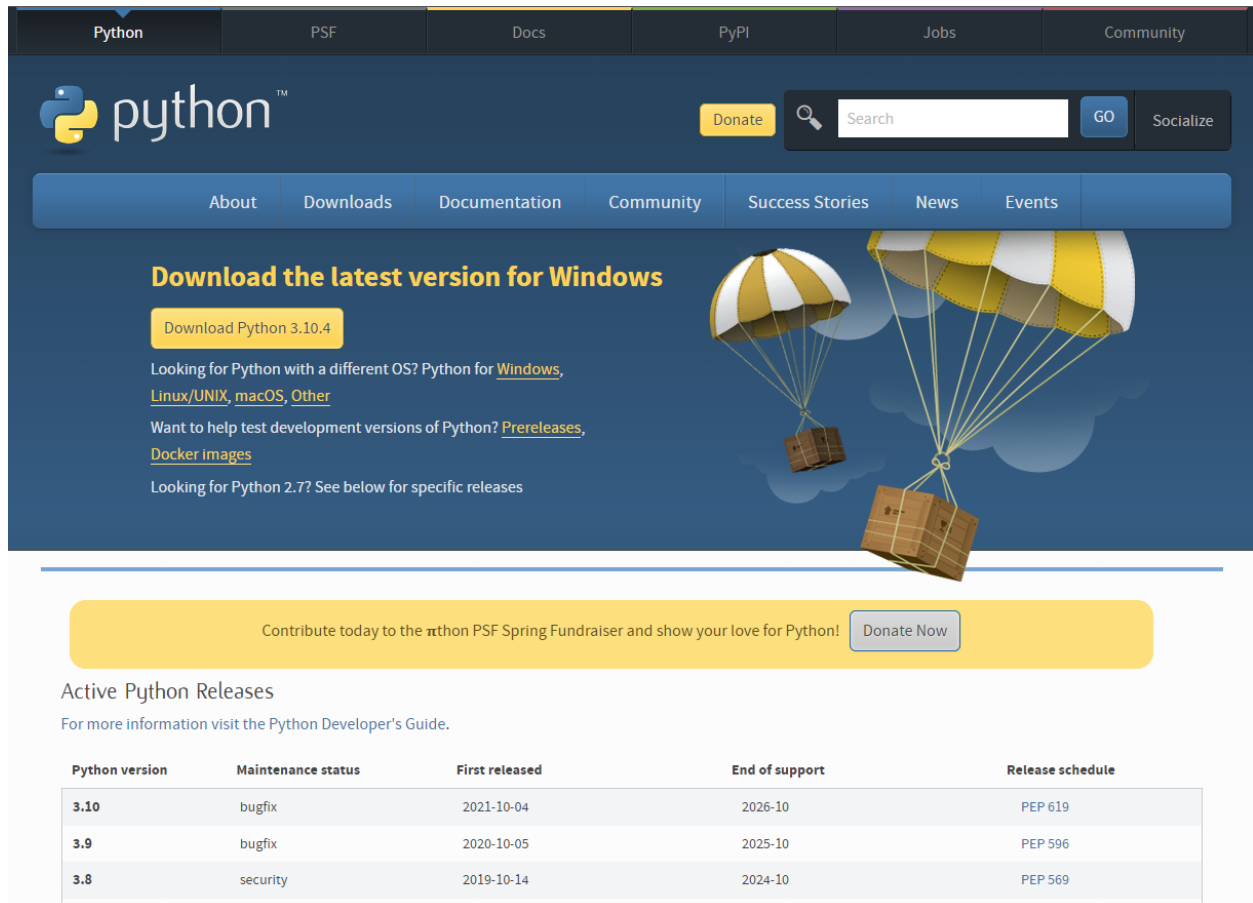
```
Python 3.12.3
```

Esto indica que Python 3.12.3 está instalado. Si el comando no produce ninguna salida o ves un mensaje de error como "Command not found" o "python: no se encontró la orden", significa que Python no está instalado o que necesitas configurar correctamente tu entorno para encontrarlo.

Aclaración: Algunos sistemas pueden incluir versiones antiguas de Python (como Python 2), que no son compatibles con este libro. Asegúrate de estar utilizando Python 3 para todas las prácticas y ejemplos.

Los usuarios de Windows, MacOS y de Linux que aun no lo tengan instalado, deberán ingresar a <https://www.python.org/downloads/>

Usaremos las configuraciones predeterminadas que el instalador sugiere por ahora, con una excepción: selecciona la casilla de verificación denominada Agregar Python 3.x a PATH



The screenshot shows the Python.org website. The top navigation bar includes links for Python, PSF, Docs, PyPI, Jobs, and Community. Below this is a search bar and a 'Donate' button. The main content area features a large banner with the text 'Download the latest version for Windows' and a button to 'Download Python 3.10.4'. To the right of the banner is an illustration of two parachutes carrying boxes. Below the banner, there is a yellow bar with a fundraising message and a 'Donate Now' button. Further down, the 'Active Python Releases' section is displayed, including a link to the Python Developer's Guide and a table of active releases.

Python version	Maintenance status	First released	End of support	Release schedule
3.10	bugfix	2021-10-04	2026-10	PEP 619
3.9	bugfix	2020-10-05	2025-10	PEP 596
3.8	security	2019-10-14	2024-10	PEP 569

Una vez instalado, verificaremos que el entorno de Python esté completo y funcional. Este procedimiento será muy simple y lo haremos con un pequeño ejercicio.

Instalación de Python en Windows / Linux / MacOS



Para windows les voy a dejar el siguiente video, la instalación es muy simple!

https://www.youtube.com/watch?v=UiQGhWZ7UHU&ab_channel=SnekCato

Nuestro primer programa en Python

El primer paso es crear un nuevo archivo fuente y llenarlo con el código.

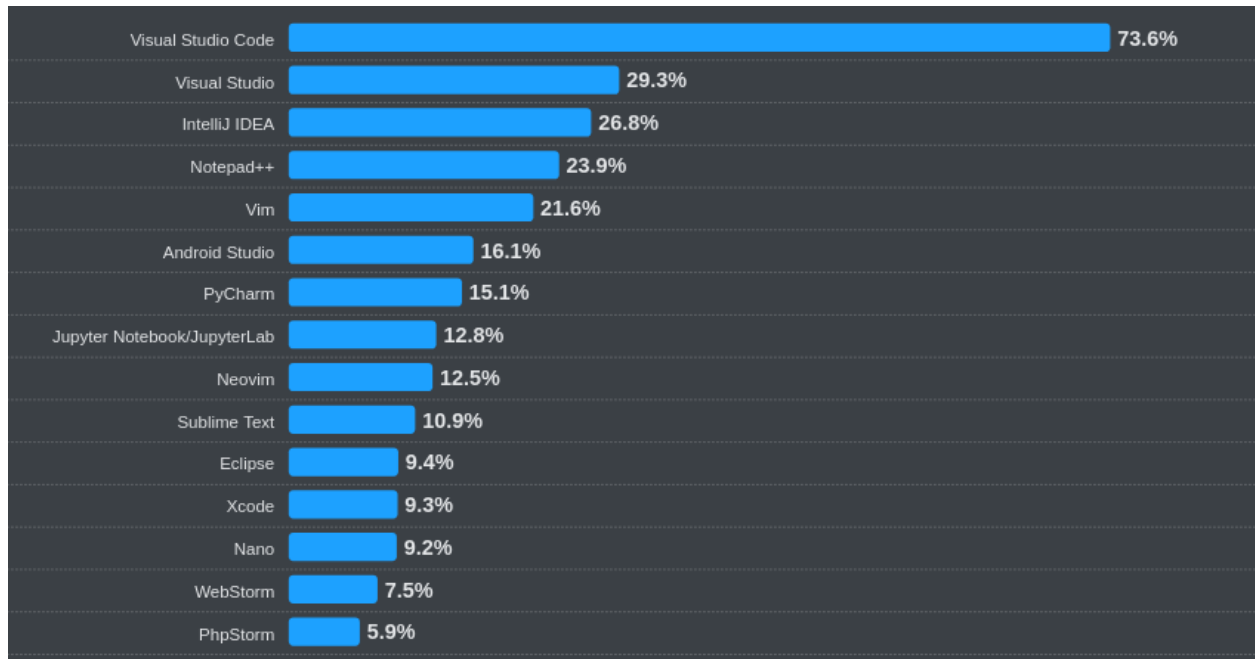
Para editar el archivo podemos utilizar el editor Visual Studio Code (VS Code).

Es un editor de código fuente desarrollado por Microsoft.

Es software libre y multiplataforma, está disponible para Windows, GNU/Linux y macOS. VS Code tiene una buena integración con Git, cuenta con soporte para depuración de código, y dispone de un sin número de extensiones, que básicamente te da la posibilidad de escribir y ejecutar código en cualquier lenguaje de programación.

Tendencias en el Uso de Editores de Código: Comparativa 2024

En este gráfico, basado en la encuesta de Stack Overflow 2023, se presenta una comparativa de los editores de código más utilizados por desarrolladores a nivel global. Visual Studio Code lidera como la opción preferida tanto entre principiantes como profesionales, reflejando su versatilidad y amplio soporte en el ecosistema de desarrollo. Esta visualización nos permite entender las preferencias y tendencias actuales en herramientas de desarrollo, esenciales para cualquier programador en formación.



<https://survey.stackoverflow.co/2024/technology/#1-integrated-development-environment>



Descargar e instalar el Visual Studio Code

<https://code.visualstudio.com/>



Instalar Visual Studio en los diferentes SO

<https://www.youtube.com/watch?v=tA8fXGbtavg>

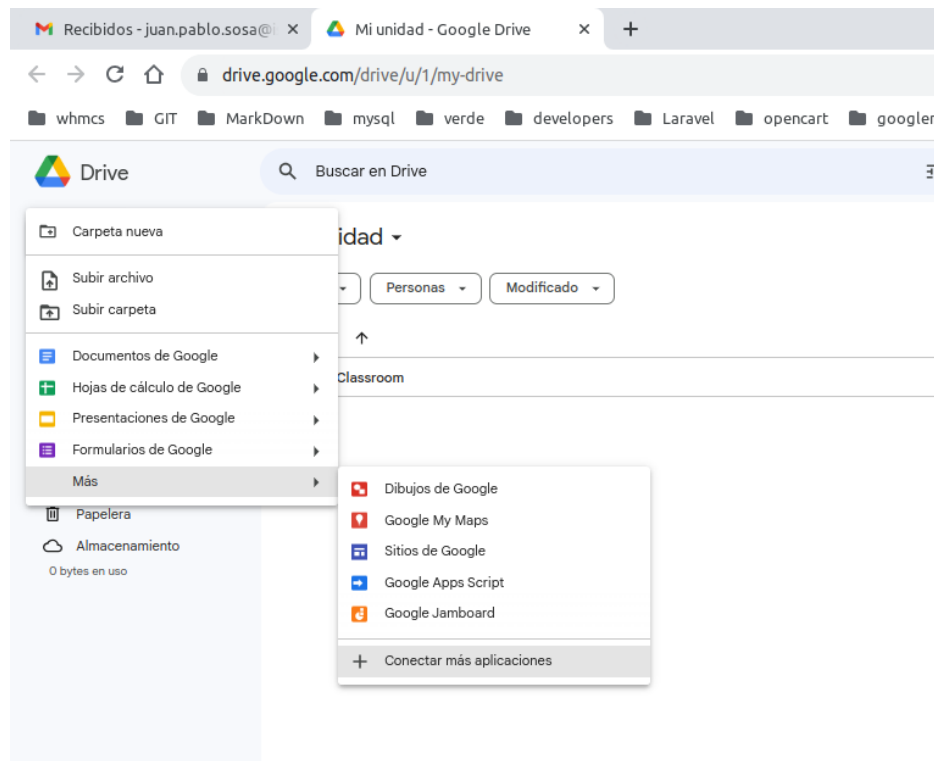
Otras opciones para ejecutar python

En mis clases, siempre recomiendo que los estudiantes utilicen un **editor de código a elección** y trabajen **localmente** para ejecutar sus programas en Python. Esta es la mejor manera de familiarizarse con el entorno de desarrollo y el flujo de trabajo de programación. Sin embargo, para aquellos que aún no han podido instalar Python en sus equipos, hay opciones alternativas, como **Colaboratory** de Google.

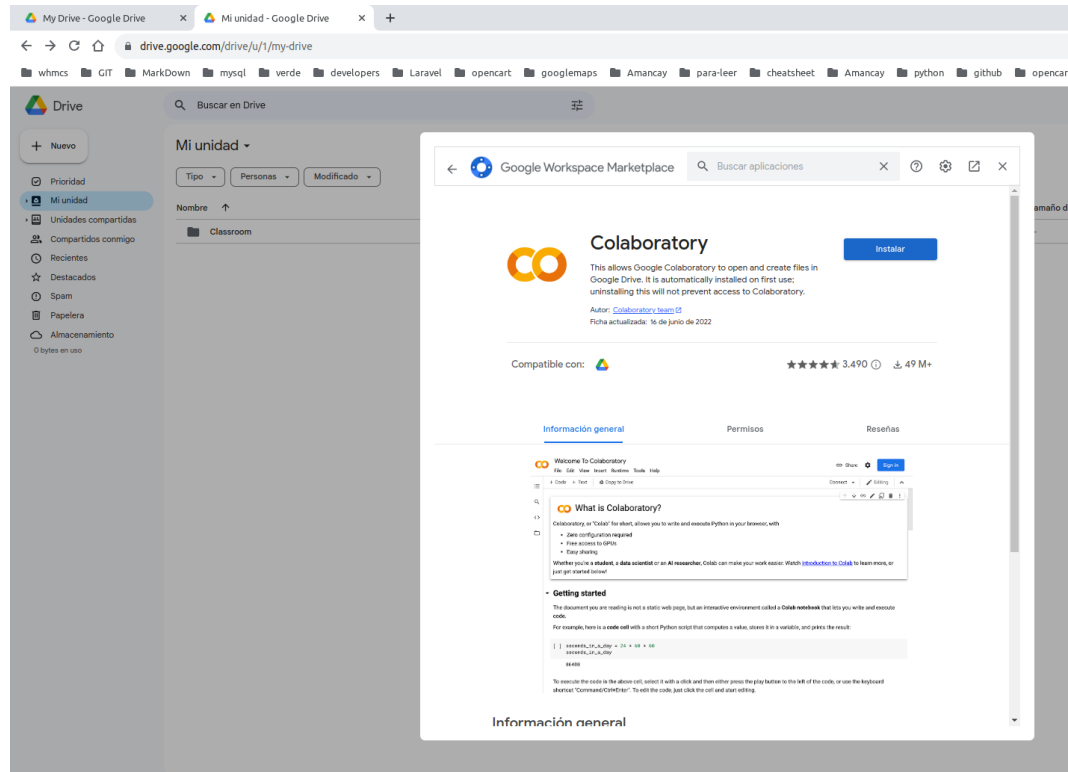
Google Colaboratory (o **Colab**) es una herramienta gratuita proporcionada por Google Research. Permite escribir y ejecutar código Python directamente en el navegador, sin necesidad de instalar nada. Es ideal para tareas de aprendizaje automático, análisis de datos y también para fines educativos.

Para comenzar, sigue estos pasos:

1. Abre tu **Google Drive**.
2. Haz clic en “**Nuevo**” y luego en “**Más aplicaciones**”.
3. Busca “**Colaboratory**” e instálalo.



Una vez que Colaboratory esté instalado, puedes empezar a crear tus programas en Python. Puedes optar por escribirlos directamente en tu editor de código preferido, o bien utilizar esta extensión de Google Drive para ejecutarlos en línea.



Ahora, pongámonos manos a la obra y escribamos el famoso **Hola Mundo**.

Recuerda que todos los archivos de Python deben tener la extensión `.py`.

Creando tu primer programa

El primer paso es definir qué queremos que haga nuestro programa. En este caso, vamos a hacer algo simple: mostrar el mensaje “¡Hola, Mundo!” en pantalla. Es una excelente forma de empezar, ya que te permite ver de inmediato cómo Python funciona.

Algoritmo:

1. Iniciar el programa.
2. Escribir el texto en pantalla.
3. Finalizar el programa.

Este conjunto de pasos ordenados y estructurados que describe lo que debe hacer el programa se llama **algoritmo**. Ahora, vamos a traducirlo a código real de Python.

Escribe el siguiente código en tu editor de código (local o en Colab):

```
print("¡Hola, Mundo!")
```

Y ejecútalo en la terminal o en el editor.

Entendiendo el código

Ahora, vamos a analizar lo que acabamos de escribir:

1. **`print`**: Es el nombre de una **función**. No siempre que veamos la palabra ``print`` se refiere a una función, sino que depende del contexto. En este caso, ``print`` es una función incorporada en Python.
2. **Paréntesis**: Los paréntesis son necesarios para indicar los argumentos que la función ``print`` necesita. En este caso, es el texto que queremos mostrar en la consola.
3. **Comillas**: El texto que queremos imprimir debe ir entre comillas, ya sea dobles (") o simples (').

¿Qué es una función?

Una **función** es un bloque de código que realiza una tarea específica y puede:

- **Causar un efecto**: Como mostrar un mensaje en la pantalla, crear un archivo o reproducir un sonido.
- **Devolver un valor**: Como calcular la raíz cuadrada de un número o la longitud de un texto.

En el caso de ``print``, realiza ambas cosas: causa el efecto de imprimir en pantalla y no devuelve un valor que podamos usar más adelante.

¿De dónde provienen las funciones?

Las funciones pueden originarse de distintas fuentes:

1. **Funciones integradas de Python:** `print` es un ejemplo de una función que viene directamente con Python. La versión actual de **Python 3** incluye 71 funciones incorporadas, que puedes consultar en su documentación oficial: <https://docs.python.org/3/library/functions.html>

.

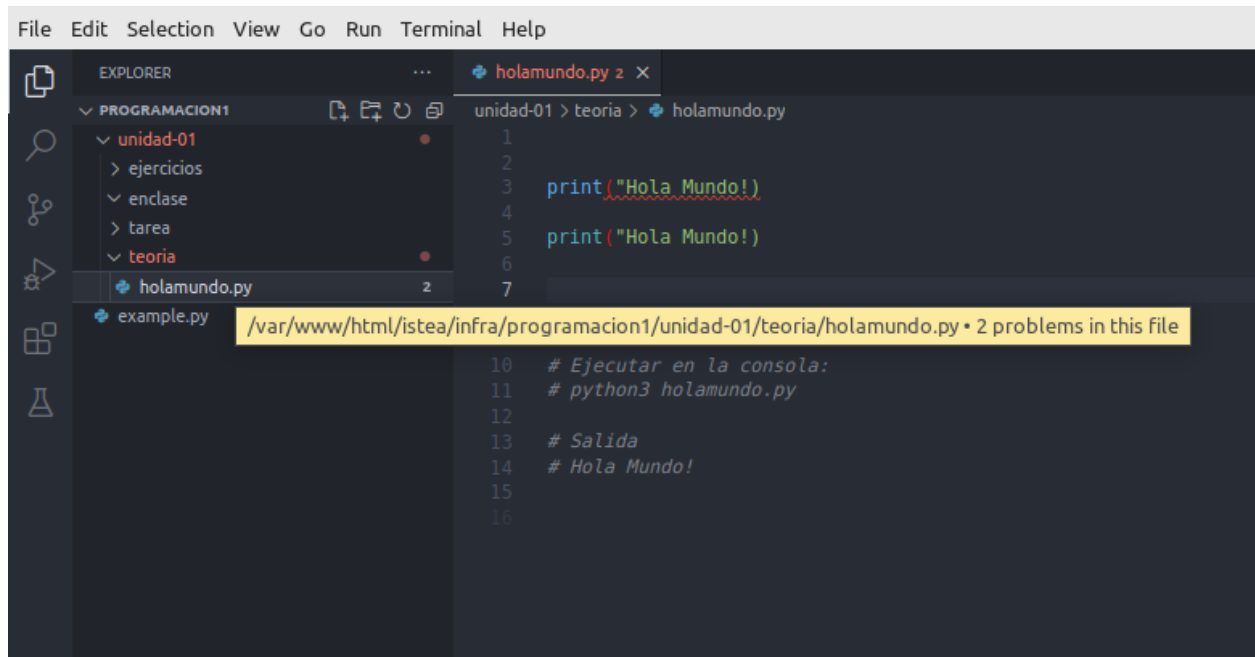
2. **Módulos adicionales:** Algunas funciones provienen de **módulos** que puedes instalar y usar en tu código. Algunos módulos vienen preinstalados con Python, mientras que otros deben instalarse por separado.

3. **Funciones definidas por el usuario:** Puedes crear tus propias funciones para hacer tu código más modular y claro. El nombre de una función debe ser **significativo** y describir claramente su propósito. Por ejemplo, el nombre `print` es adecuado porque describe exactamente lo que hace la función: imprime en la consola.

Depuración de nuestro primer programa

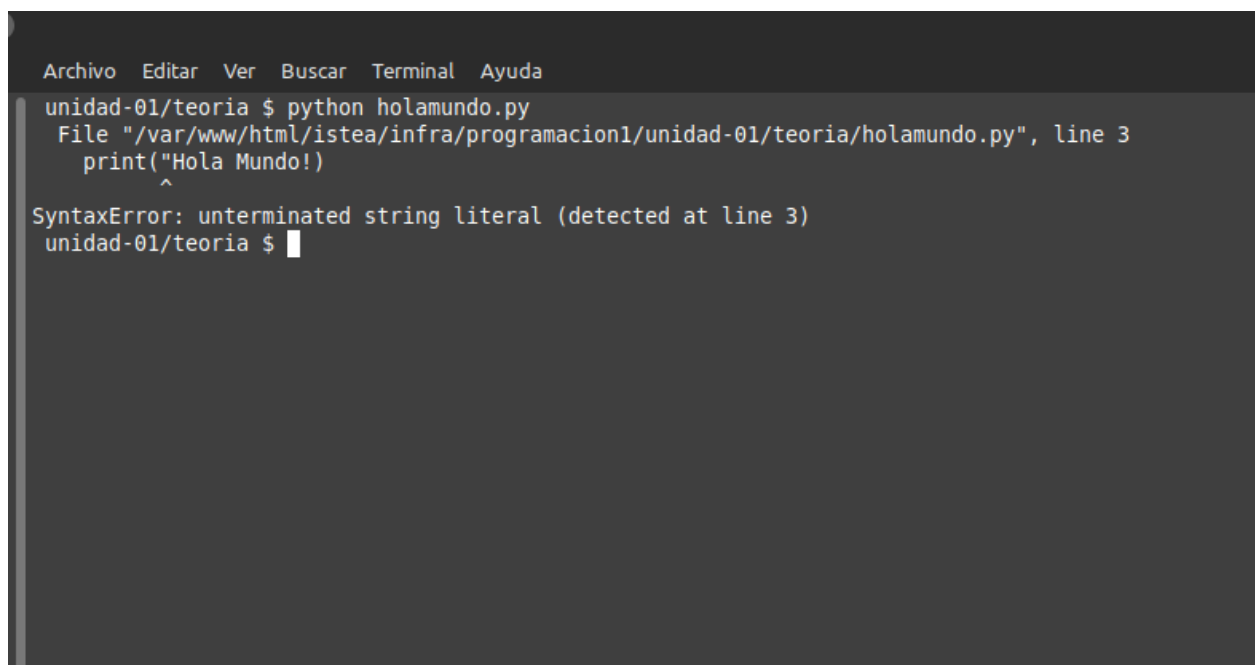
El Visual Studio y otros editores nos ayudan a depurar el código, avisándonos de antemano de los errores. También hay muchas extensiones que se pueden instalar para esto. Por ejemplo, podemos tener instalada la extensión Pylance, la cual nos va a avisar cuando detecte algún error, aún antes de ejecutar el programa.

Vamos a probar ahora por ejemplo no cerrar el último paréntesis, o bien olvidándonos alguna doble comilla. ¿Qué es lo que sucede?



Vamos a notar que el mismo editor nos está avisando de los errores.

Y qué pasa si lo ejecutamos igual ?



Por consola nos va a marcar lo siguiente:

¿Qué sucede cuando Python nos muestra un error?

Cuando Python encuentra un error en nuestro código, generalmente nos proporciona información útil para poder identificar y corregirlo. A continuación, desglosamos los elementos principales que veremos en un mensaje de error:

1. **Ubicación exacta del error:** Python nos indica la línea específica donde se ha producido el error, lo cual nos ayuda a localizarlo rápidamente.
2. **El trace (o traza):** Este es el recorrido que hace el código a través de diferentes partes del programa hasta llegar al error. En un programa simple, la traza puede no ser muy relevante, y puedes ignorarla por ahora, ya que generalmente será vacía o no tan útil para entender el problema.
3. **Detalles de la ubicación:** El mensaje de error incluirá el nombre del archivo donde se ha producido el error, el número de línea correspondiente y, en algunos casos, el nombre del módulo. **Nota importante:** El número de línea que Python muestra no siempre coincide exactamente con la ubicación del error real, ya que Python señala la primera línea donde detecta el efecto del error, no necesariamente el lugar exacto donde ocurrió.
4. **La línea de código con el error:** Python mostrará la línea de código que ha causado el problema, e intentará marcarla con un símbolo `^` debajo del lugar donde detecta el error. Sin embargo, ten en cuenta que esta marca no siempre es precisa, especialmente en casos de errores complejos.
5. **Descripción del error:** Python también nos dará el nombre del error y una breve explicación del problema. En algunos casos, también puede ofrecer sugerencias o soluciones que pueden ayudar a resolver el inconveniente.

UNIDAD 2

TIPOS DATOS | OPERADORES | VARIABLES | COMENTARIOS

En esta unidad aprenderemos sobre:

- ¿Qué son los comentarios?
- La función print() y en qué elementos se divide. Función type()
- Tipos de datos, operadores y expresiones en Python
- Qué son las variables y cuáles son las reglas que las gobiernan
- Cómo realizar operaciones básicas de entrada y salida

Comentarios en Python

A veces necesitamos incluir notas o explicaciones en nuestro código. Estas notas no están dirigidas a Python, sino a otras personas (¡o a nosotros mismos en el futuro!) para entender mejor lo que hace el programa. Estas notas se llaman comentarios y son completamente ignoradas por Python al ejecutar el código.

Un comentario:

- Puede explicar cómo funciona el código.
- Documenta quién creó el programa y cuándo.
- Permite "desactivar" partes de código temporalmente para realizar pruebas.

En Python, un comentario es un texto que comienza con el símbolo # (numeral o almohadilla) y se extiende hasta el final de la línea.

En cambio si queremos agregar un comentario de varias líneas debemos comenzar con `"""`, y luego terminar con `"""`, ejemplo:

```
"""
Este es un comentario largo
puede abarcar varias líneas
"""
```

La función print()

La función print() es una de las más utilizadas en Python. Se usa para mostrar información en la consola, como mensajes o resultados de cálculos.

La sintaxis básica es:

```
print(argumento1, argumento2, ...)
```

Cómo funciona print()

Cuando ejecutamos print(), Python realiza varios pasos:

- Comprueba que el nombre print es válido.
- Verifica los argumentos que le pasaste. Si algo no coincide, muestra un error.
- Ejecuta la función, transforma los argumentos en texto comprensible y los muestra en pantalla.



```
>>> # Vamos a probar la función print
>>> print("Hola, Python")
```

Resultado en consola:

Hola, Python

Carácter especial: \n

La barra invertida (\) se usa para incluir caracteres especiales. Por ejemplo, \n indica un salto de línea:

Ejemplo:



```
>>> # Ejemplo con \n
>>> print("Hola mundo!\n Hola Python\n")
>>>
>>> ## Salida consola
>>> # Hola mundo!
>>> # Hola Python
```

Existen otros caracteres especiales, marcamos los más importantes:

Aquí tienes una tabla con los caracteres especiales más comunes utilizados en la función `print()` de Python y su propósito:

Carácter especial	Nombre	Descripción
\n	Nueva línea	Inserta un salto de línea, moviendo el cursor al inicio de la siguiente línea.
\t	Tabulación Horizontal	Añade una tabulación horizontal en el texto, similar a presionar la tecla TAB.
\\	Barra invertida	Inserta una barra invertida (\) en el texto.
\'	Comilla simple	Inserta una comilla simple dentro de un string delimitado por comillas simples.
\"	Comilla doble	Inserta una comilla doble dentro de un string delimitado por comillas dobles.

Múltiples argumentos

Puedes pasar varios argumentos separados por comas:



```
>>> print("Hola", "Python")  
>>> # Salida consola  
>>> # Hola Python
```

Como podemos visualizar en el ejemplo lo que hace Python es agregarle un espacio a los strings, cuando pasamos varios argumentos separados por coma.

Otros argumentos de la función print()

La forma en que pasamos los argumentos a la función print() de manera más común en Python se denomina **manera posicional**. Este término hace referencia a que el significado de cada argumento está determinado por su posición en la llamada a la función. Por ejemplo, si proporcionamos tres argumentos, el segundo se imprimirá inmediatamente después del primero, y el tercero después del segundo, respetando siempre este orden.

Sin embargo, Python también nos permite utilizar otro mecanismo para transmitir información a las funciones: los **argumentos de palabra clave**. Este mecanismo es particularmente útil cuando queremos modificar el comportamiento predeterminado de una función sin depender únicamente del orden de los argumentos.

No entraremos en demasiados detalles por ahora, ya que profundizaremos en este tema cuando abordemos funciones más adelante en el libro. Por ahora, aprenderemos a utilizar algunos de los argumentos de palabra clave más útiles de la función print().

Un argumento de palabra clave está compuesto por tres elementos:

Una palabra clave que identifica el argumento.

Un signo igual (=).

Un valor asignado a ese argumento.

Es importante recordar que cualquier argumento de palabra clave debe colocarse después del último argumento posicional en la llamada a la función.

El argumento end

Por defecto, la función `print()` finaliza cada línea con un salto de línea (`\n`). Esto significa que cualquier texto que imprimas después aparecerá en una nueva línea. Si deseas cambiar este comportamiento, puedes usar el argumento de palabra clave `end`.

Ejemplo:



```
>>> # Argumento end
>>> print("Hola", end="-")
>>> print("Mundo")
>>>
>>> # Salida consola
>>> # Hola-Mundo
```

El argumento `sep`

Otro argumento útil de la función `print()` es `sep` (abreviatura de separator). Este argumento especifica qué carácter (o cadena de caracteres) se debe utilizar para separar múltiples argumentos posicionales en la salida. Por defecto, el separador es un espacio ().

Ejemplo:



```
>>> # Argumento sep
>>> print("Python", "es", "genial", sep="-")
>>>
>>> # Salida consola
>>> # Python-es-genial
```

En este caso, el separador entre cada argumento es un guion (`-`). Si no se especifica el argumento `sep`, la función `print()` insertará espacios automáticamente entre los argumentos.

Puedes combinar los argumentos `end` y `sep` para personalizar completamente el formato de la salida:



```
>>> # Combinando sep y end
>>> print("Aprender", "Python", sep="→", end="!!!\n")
>>>
```

```
>>> # Salida consola
>>> # Aprender→Python!!!
```

En resumen, los argumentos end y sep son herramientas sencillas pero poderosas que te permiten personalizar el comportamiento de la función print() para que se adapte a tus necesidades específicas.

Función type()

Y antes de comenzar con el siguiente tema, vamos a ver una nueva función. type()

Lo que nos va a devolver esta función es el tipo de dato que le vamos a pasar, es decir nos va a decir que dato estamos trabajando. La vemos aplicada directamente con el próximo tema. Por ahora solo hagamos:

```
type(10)
```

¿Nos devuelve algo? ¿Por qué no devuelve nada?

Exacto, debemos imprimir en pantalla lo que le estamos solicitando, entonces:

```
print( type(10) )
```

Tipos de datos

¿Qué son los tipos de datos?

En cualquier lenguaje de programación, incluidos los de alto nivel como Python, manejamos **tipos de datos**. Pero, ¿qué significa esto? Los **tipos de datos** definen un conjunto de valores con características y propiedades específicas.

Imagina por un momento que estás de vuelta en el colegio, en clase de matemáticas. Seguro recuerdas cuando te enseñaron sobre los distintos conjuntos de números:

- Los **naturales**: 1, 2, 3, 4, ...
- Los **enteros**: ..., -2, -1, 0, 1, 2, ...
- Los **reales**: ..., -1.1, -0.3, 2.1, ...

Cada uno de estos conjuntos tiene reglas específicas, ¿verdad? Por ejemplo, no puedes sumar un número negativo y obtener un número natural, porque los naturales no incluyen negativos.

En programación (y especialmente en Python), sucede algo similar: los valores que utilizamos pertenecen a diferentes **tipos de datos**. Cada tipo tiene sus propias reglas, sus valores permitidos y las operaciones que puedes realizar con ellos.

En Python, cualquier valor que asignas a una variable tiene asociado un **tipo de dato**. Por ejemplo:



```
>>> edad = 25          # El número 25 es del tipo entero
>>> precio = 19.99     # El número 19.99 es del tipo punto flotante
>>> nombre = "Ana"     # El texto "Ana" es del tipo cadena de caracteres
>>> es_estudiante = True # El valor True es del tipo booleano
```

En resumen, un **tipo de dato** determina:

1. Qué valores puede tomar una variable.
2. Qué operaciones puedes realizar con ella.

¿Qué tipos de datos trae consigo Python?

Python incluye varios tipos de datos básicos que se utilizan con frecuencia en cualquier programa. Entre ellos:

1. Booleanos

- Representan valores de verdad: **True** (verdadero) y **False** (falso).

Ejemplo:

```
es_mayor = True  
es_menor = False
```

○

2. Numéricos

- **Enteros (`int`)**: Números sin parte decimal, como `5`, `-3` o `42`.
- **Punto flotante (`float`)**: Números con parte decimal, como `3.14` o `-0.5`.

Ejemplo:

```
numero_entero = 10  
numero_decimal = 3.1416
```

○

3. Cadenas de caracteres (`str`)

- Representan texto, como palabras, frases o incluso números tratados como texto.
- Se definen usando comillas simples (`'`) o dobles (`"`).



```
# Valores tipo string (str)  
saludo = "Hola, mundo"  
letra = 'A'
```

Estos son los **tipos de datos básicos** que Python trae "de fábrica". Más adelante en el libro aprenderemos sobre otros tipos más avanzados que Python también define y que son muy útiles en ciertas situaciones.

[Cuadro sintético de los tipos de datos principales de Python](#)

TIPO DATO	DESCRIPCION	EJEMPLO
Booleano (bool)	Representa valores de verdad: True (verdadero) y False (falso).	es_mayor = True
Entero (int)	Números enteros, positivos o negativos, sin parte decimal.	numero_entero = 42
Punto flotante (float)	Números con parte decimal, positivos o negativos.	numero_decimal = 3.14
Cadena de caracteres (str)	Texto o secuencia de caracteres, definidos entre comillas simples (') o dobles (").	nombre = "Ana"

Por ahora, lo importante es entender que cada tipo de dato tiene características propias que determinan cómo puedes trabajar con él. Con esta base, estarás listo para explorar los datos y operaciones en Python. ¡Sigamos aprendiendo!

Operadores Aritméticos

Este tema es fundamental, ya que los cálculos matemáticos son una parte esencial de muchos programas. Vamos a desglosar cada operador para que entiendas cómo funcionan, incluso si nunca has programado antes.

¿Qué son los operadores aritméticos?

Los operadores aritméticos son símbolos que usamos para realizar operaciones matemáticas como suma, resta, multiplicación, división y mucho más. Estos son los operadores más comunes que verás en Python:

OPERADOR	NOMBRE	EJEMPLO	RESULTADO
+	Suma	5 + 3	8
-	Resta	7 - 4	3
*	Multiplicación	2 * 4	8
/	División	5 / 2	2.5
//	División entera	5 / 2	2
%	Resto o módulo	5 % 2	1
**	Exponenciación	2 ** 3	8

Vamos a analizar cada uno con más detalle y ejemplos claros.

Suma (+)

La suma es una de las operaciones más sencillas. Puedes sumar dos números y obtener su resultado.



```
>>> resultado = 10 + 5
>>> print(resultado)
>>>
>>> # Salida en Consola:
>>> # 15
```

Resta (`-`)

La resta te permite calcular la diferencia entre dos números.

Ejemplo:



```
>>> resultado = 10 - 5
>>> print(resultado)
>>> # Salida en Consola:
>>> # 5
```

Multiplicación (`\*`)

La multiplicación se usa para calcular el producto de dos números.

Ejemplo:



```
>>> resultado = 10 * 5
>>> print(resultado)
>>> # Salida en Consola:
>>> # 50
```

División (`/`)

La división siempre devuelve un número decimal (o "flotante" en Python).

Ejemplo:



```
>>> resultado = 10 / 4
>>> print(resultado)
>>> # Salida en Consola:
>>> # 2.5
```

División Entera (`//`)

Si necesitas sólo la parte entera de la división, puedes usar `//`.

Ejemplo:



```
>>> resultado = 10 // 4
>>> print(resultado)
```



```
>>> # Salida en Consola:  
>>> # 2
```

Resto o Módulo (`%`)

El operador `%` devuelve el resto de una división.

Ejemplo:



```
>>> resultado = 10 % 4  
>>> print(resultado)  
>>> # Salida en Consola:  
>>> # 2
```

Esto es útil, por ejemplo, para saber si un número es par o impar. Siempre hacemos el resto sobre 2. Si nos da 0 es par. Si no es impar.

Exponenciación (`\*\*`)

Si necesitas calcular potencias, puedes usar `\*\*`.

Ejemplo:



```
>>> resultado = 2 ** 3  
>>> print(resultado)  
>>> # Salida en Consola:  
>>> # 8
```

Prioridades de los Operadores

Cuando escribimos una expresión que incluye varios operadores, Python sigue un orden para evaluarla. Este orden se llama "prioridad de los operadores". Por ejemplo:



```
>>> resultado = 2 + 3 * 4  
>>> print(resultado)  
>>> # Salida en Consola:  
>>> # 14
```

Python primero realiza la multiplicación (`3 \* 4 = 12`) y luego suma `2 + 12 = 14`.

La prioridad de los operadores en Python es:

1. Paréntesis `()`
2. Exponenciación `\*\*`
3. Multiplicación `\*`, División `/`, División Entera `//`, Módulo `%`
4. Suma `+` y Resta `-`

Si quieres cambiar el orden de evaluación, puedes usar paréntesis:



```
>>> resultado = (2 + 3) * 4
>>> print(resultado)
>>> # Salida en Consola:
>>> # 20
```

Asociatividad o "Binding"

La asociatividad determina el orden en el que Python evalúa operadores con la misma prioridad. La mayoría de los operadores tienen asociatividad de izquierda a derecha ("left-to-right binding"). Por ejemplo:



```
>>> resultado = 10 - 5 - 2
>>> print(resultado)
>>> # Salida en Consola:
>>> # 3
```

Python calcula de izquierda a derecha: `(10 - 5) - 2 = 5 - 2 = 3`.

Por otro lado, la exponenciación tiene asociatividad de derecha a izquierda ("right-to-left binding"):



```
>>> resultado = 2 ** 3 ** 2
>>> print(resultado)
>>> # Salida en Consola:
>>> # 512
```

Python evalúa primero `3 \*\* 2 = 9` y luego `2 \*\* 9 = 512`.

Resumen

En este capítulo aprendiste:

- Los operadores aritméticos en Python y cómo usarlos.
- La prioridad de los operadores y cómo afecta el orden de las operaciones.
- La asociatividad o "binding" de los operadores.

Con esta base, estarás listo para realizar cálculos y escribir expresiones matemáticas en Python sin problemas. ¡Manos a la obra!

Variables

Hasta ahora, hemos aprendido a realizar operaciones aritméticas. Pero seguramente te habrás preguntado: ¿cómo podemos almacenar los resultados de estas operaciones para utilizarlos más tarde? Aquí es donde entran en juego las **variables**.

Las variables son como contenedores que permiten guardar datos y recuperarlos cuando los necesitemos. El nombre "variable" se debe a que su contenido puede cambiar o "variar" a lo largo de la ejecución del programa.

Componentes de una variable

En Python, una variable tiene dos elementos fundamentales:

1. **Un nombre:** el identificador que decides darle.
2. **Un valor:** el dato almacenado en esa variable.

Reglas para nombrar variables

Al nombrar variables en Python, es importante seguir estas reglas:

- Los nombres pueden incluir letras (mayúsculas y minúsculas), dígitos y el carácter de guión bajo `\_`.
- Deben **comenzar con una letra** o con un guión bajo `\_`, pero **no con un número**.
- Python distingue entre mayúsculas y minúsculas, por lo que `variable`, `Variable` y `VARIABLE` son tres nombres diferentes.
- No puedes usar palabras reservadas de Python (como `if`, `else`, `True`, etc.) como nombres de variables.

Ejemplo de nombres válidos:

- ``miVariable``
- ``edad``
- ``_contador``
- ``variable_super_larga``

Ejemplo de nombres no válidos:

- ``123variable`` (no puede comenzar con un número)
- ``mi variable`` (no se permiten espacios)
- ``True`` (es una palabra reservada)

Lista de palabras reservadas en Python

Python tiene un conjunto de palabras reservadas que no puedes usar como nombres de variables. En la mayoría de las versiones son las siguientes:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',  
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',  
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda',  
'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while',  
'with', 'yield']
```

Para obtener la lista completa en tu entorno, puedes usar este comando:



```
>>> import keyword  
>>> print(keyword.kwlist)
```

¿Qué podemos guardar en una variable?

Cualquier tipo de dato:

- **Números enteros (int):** ``edad = 25``
- **Números decimales (float):** ``altura = 1.75``
- **Cadenas de texto (string):** ``nombre = "Ana"``
- **Valores booleanos (bool):** ``es_estudiante = True``

En Python, no es necesario declarar una variable antes de usarla. Basta con asignarle un valor, y la variable se creará automáticamente.



```
>>> x = 10 # Crea la variable x y le asigna el valor 10  
>>> print(x) # Muestra el valor de x
```

Reasignación de valores

Puedes cambiar el valor de una variable tantas veces como necesites. Por ejemplo:



```
>>> x = 5
>>> x = x + 2 # Ahora x vale 7
```

Operadores de asignación abreviados

Python nos permite simplificar operaciones comunes mediante operadores de asignación abreviados. En lugar de escribir:



```
>>> x = 10
>>> x = x + 2
>>> # en Python se puede simplificar la operación anterior de la siguiente
manera:
>>> x += 2 # Equivalente a x = x + 2
>>> print(x)
>>> # Salida en Consola:
>>> # 14
```

Otros operadores abreviados son:

Operador	Ejemplo	Equivalente a
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	<code>x -= 2</code>	<code>x = x - 2</code>
<code>*=</code>	<code>x *= 4</code>	<code>x = x * 4</code>

```
/=      x /= 2    x = x / 2
//=     x //= 3   x = x // 3
%=      x %= 5    x = x % 5
**=     x **= 2   x = x ** 2
```

Buenas prácticas al nombrar variables

1. Usa nombres descriptivos que expliquen su propósito:

 `x = 25`

 `edad = 25`

2. Evita nombres excesivamente largos o complejos.

3. Si necesitas separar palabras en un nombre, usa el guion bajo `\_`:

 `precio\_total`, `nombre\_cliente`.

input()

Hasta ahora, hemos usado la función `print()` para enviar datos a la pantalla. Pero ¿qué pasa si queremos que el usuario nos proporcione datos? Ahí es donde entra en juego una nueva función que parece ser el "reflejo" de `print()`.

¿Por qué "reflejo"? Porque mientras `print()` muestra información al usuario, la función `input()` hace lo contrario: ****recoge información del usuario****.

La función `input()` permite que un programa sea interactivo, es decir, que el usuario pueda comunicarse con él introduciendo datos que luego el programa puede usar para hacer cálculos, tomar decisiones, o cualquier otra cosa que necesitemos.

¿Cómo funciona `input()`?

Veamos paso a paso cómo usar esta función:

1. El modo básico

La forma más sencilla de usar `input()` es llamándola sin argumentos. Cuando el programa llegue a esta función, la consola se pondrá "en espera" para que el usuario escriba algo. Verás un cursor parpadeando que indica que el programa está listo para recibir datos.

Una vez que el usuario escriba algo y presione `Enter`, ese texto será enviado al programa.

2. Guardar el resultado

Es importante asignar el resultado de `input()` a una variable, porque, de lo contrario, los datos que el usuario introduzca se perderán.



```
>>> nombre = input()
>>> print("Hola, " + nombre)
```

En este ejemplo, el programa espera que el usuario escriba algo. Supongamos que el usuario escribe "Ana". El programa mostrará:

Hola, Ana

Mostrar un mensaje al usuario

La función `input()` también puede mostrar un mensaje antes de que el usuario escriba algo. Para lograr esto, simplemente le pasamos un texto como argumento. Por ejemplo:

```
>>> nombre = input("¿Cómo te llamas? ")
>>> print("Hola, " + nombre)
```

Aquí, el programa mostrará el texto:

¿Cómo te llamas?

y esperará que el usuario responda.

4. El resultado siempre es un string

Es importante saber que lo que `input()` devuelve siempre será un `**string**`, aunque el usuario escriba números. Por ejemplo:



```
>>> edad = input("¿Cuántos años tienes? ")
>>> print("Tu edad es: " + edad)
```

Si el usuario introduce ``25``, el programa no lo considera un número, sino una cadena de texto: `"25"`.

En la siguiente unidad vamos a ver como poder convertir el tipo de dato para poder tratarlos. Por que por ejemplo Python no puede sumar dos strings. Deben ser los dos datos numéricos.

Resumen

- La función `input()` permite que el usuario introduzca datos.
- Puedes mostrar un mensaje al usuario pasándolo como argumento a la función.
- Los datos devueltos por `input()` siempre son de tipo string, aunque parezcan números.

UNIDAD 3

STRINGS | COMPARADORES | CONDICIONALES

En esta unidad aprenderemos sobre:

- Operadores de strings
- Datos de tipo Booleano
- Conversiones con ``int()``, ``float()`` y ``str()``
- Cómo tomar decisiones en Python
- Condiciones y ejecución condicional

Operadores de Strings

En el módulo anterior aprendimos sobre operadores aritméticos y operadores abreviados. Ahora vamos a analizar cómo algunos de estos operadores (``+`` y ``*``) también tienen usos específicos en el contexto de los **strings**.

Un **string** es simplemente una secuencia de caracteres, como palabras, frases o incluso números tratados como texto. Los strings están rodeados por comillas, ya sean simples (``texto``) o dobles (``"texto"``).

Concatenación

El operador `+` puede usarse para **concatenar** strings, es decir, para unir dos o más strings en uno solo.



```
>>> mensaje_1 = "Hola"
>>> mensaje_2 = "Mundo"
>>> resultado = mensaje_1 + " " + mensaje_2
>>> print(resultado)
>>> # Salida en Consola:
>>> # Hola Mundo
```

Tener en cuenta:

- El operador `+` no puede mezclar tipos de datos distintos. Por ejemplo, no puedes concatenar un string con un número directamente (`"Hola" + 5` generará un error). Si necesitas hacerlo, puedes convertir el número en un string usando `str()`.
- La concatenación **no es conmutativa**. Esto significa que `"ab" + "ba"` no es lo mismo que `"ba" + "ab"`. El orden importa.

Replicación

El operador `\*` también puede usarse con strings, pero en este caso para **replicarlos**. Esto significa que puedes repetir un string un número determinado de veces.

```
>>> mensaje = "Python "
>>> resultado = mensaje * 3
>>> print(resultado)
>>> # Salida en Consola:
>>> # Python Python Python
```

Tener en cuenta:

- La replicación solo funciona con un string y un número. Si intentas usar dos strings (`"Hola" \* "Mundo"`), obtendrás un error.
- El número debe ser un entero. Si usas un número decimal (`float`), Python no sabrá qué hacer.

Conversión de datos: `int()`, `float()`, `str()`

Cuando trabajamos con datos en Python, muchas veces necesitamos convertir un tipo de dato a otro. Por ejemplo, convertir un string que representa un número a un entero para hacer cálculos, o convertir un número en un string para mostrarlo como texto en la pantalla.

Python nos proporciona funciones sencillas para realizar estas conversiones:

Conversión a Enteros: `int()`

La función `int()` convierte un string (o incluso un número decimal) a un número entero.



```
>>> numero_como_texto = "10"
>>> numero = int(numero_como_texto)
>>> print(numero)
>>> # Salida en Consola:
>>> # 10
```

⚠ Nota importante: Si el string no representa un número válido, obtendrás un error. Por ejemplo, intentar convertir "Hola" a entero con `int("Hola")` no funcionará. Más adelante veremos cómo manejar estos errores de manera segura.

Conversión a Flotantes: `float()`

La función `float()` convierte un string o un número entero en un número con decimales.



```
>>> numero_como_texto = "3.14"
>>> numero = float(numero_como_texto)
>>> print(numero)
>>> # Salida en Consola:
>>> # 3.14
```

Conversión a Strings: `str()`

La función `str()` convierte cualquier dato en un string.



```
>>> numero = 42
```

```
>>> texto = str(numero)
>>> print("El número es: " + texto)
>>> # Salida en Consola:
>>> # El número es: 42
```

Esto es especialmente útil cuando necesitas combinar texto con números en un mensaje.

Ejemplo práctico: Calculadora de hipotenusa

Veamos un ejemplo donde combinamos strings y conversiones de datos para resolver un problema matemático. Supongamos que queremos calcular la hipotenusa de un triángulo rectángulo usando la fórmula de Pitágoras:

```
37
38
39 # Calculo de una hipotenusa
40
41 # Pedimos las longitudes de los catetos
42 cateto_a = float(input("Ingresa la longitud del primer cateto: "))
43 cateto_b = float(input("Ingresa la longitud del segundo cateto: "))
44
45 # Calculamos la hipotenusa
46 hipotenusa = (cateto_a**2 + cateto_b**2) ** 0.5
47
48 # Mostramos el resultado como un string
49 print("La hipotenusa es: " + str(hipotenusa))
50
51 # Salida esperada:
52 # Si el usuario ingresa 3 y 4, el programa mostrará:
53 # La hipotenusa es: 5.0
54
55
```

Este ejemplo combina varias habilidades: tomar datos del usuario, realizar cálculos y mostrar resultados en un formato amigable.

Operadores de comparación

La comparación es una parte fundamental de la programación porque nos permite tomar decisiones basadas en si algo es igual, mayor, menor, o diferente a otra cosa.

Operador de igualdad (==)

Para responder a la pregunta: **¿Son dos valores iguales?**, usamos el operador `==` (igual igual).

Es muy importante diferenciarlo del operador `=` (igual), ya que cumplen funciones completamente diferentes:

`=` es un **operador de asignación**. Se usa para asignar un valor a una variable. Por ejemplo:

```
a = b # Asigna el valor de b a la variable a
```

`==` es un **operador de comparación**. Se usa para preguntar si dos valores son iguales. Por ejemplo:

```
a == b # ¿Son a y b iguales?
```

El operador `==` compara los valores de dos elementos y devuelve uno de estos dos resultados:

`True` si los valores son iguales.

`False` si los valores son diferentes.

```
>>> 1 == 2 # False, porque 1 y 2 no son iguales.
```

```
>>> 2 == 2 # True, porque 2 y 2 son iguales.
```

```
>>> "abc" == "abc" # True, porque ambas cadenas son idénticas.
```

```
>>> "ABC" == "abc" # False, porque las letras mayúsculas y minúsculas son diferentes.
```

Operador de desigualdad (!=)

El operador `!=` (no es igual a) sirve para verificar si dos valores **no** son iguales.

A diferencia de `==`, este operador devuelve:

- **True** si los valores son diferentes.
- **False** si los valores son iguales.

Repasemos los mismos ejemplos, pero ahora usando **!=**:

```
>>> 1 != 2 # True, porque 1 y 2 son diferentes.
>>> 2 != 2 # False, porque 2 y 2 son iguales.
>>> "abc" != "abc" # False, porque las cadenas son iguales.
>>> "ABC" != "abc" # True, porque las cadenas son diferentes (por las mayúsculas y minúsculas).
```

Mayor que (>)

Para saber si un valor es **mayor que** otro, usamos el operador **>** (mayor que).

Por ejemplo:

```
manzanas > peras
```

Esto significa: "¿Hay más manzanas que peras?".



```
>>> 5 > 3 # True, porque 5 es mayor que 3.
>>> 2 > 7 # False, porque 2 no es mayor que 7.
```

Menor que (<)

De manera similar, el operador **<** (menor que) responde la pregunta opuesta:

¿Es un valor menor que otro?



```
>>> 5 < 3 # False, porque 5 no es menor que 3.
>>> 2 < 7 # True, porque 2 es menor que 7.
```

Mayor o igual que (>=)

El operador **>=** se usa cuando queremos saber si un valor es **mayor o igual** que otro.

Es como una combinación de **>** y **=**.



```
>>> temperatura >= 0 # True si la temperatura es mayor o igual a 0 grados.
```

Menor o igual que (<=)

Por último, el operador `<=` nos permite preguntar si un valor es **menor o igual** que otro. Funciona de manera similar a `>=`, pero para el caso opuesto.



```
>>> dinero <= 100 # True si el dinero es menor o igual a 100.
```

Tabla de prioridad de operadores

Cuando usamos varios operadores en una misma expresión, Python evalúa de acuerdo a una jerarquía o **prioridad de operadores**.

Aquí tienes una tabla que muestra la prioridad de los operadores que hemos aprendido hasta ahora (de mayor a menor prioridad):

Operador	Significado
<code>> < >= <=</code>	Comparaciones de mayor, menor, etc.
<code>== !=</code>	Igualdad y desigualdad.

Los operadores de comparación tienen menor prioridad que otros operadores, como los aritméticos (`+`, `-`, `*`, `/`).

Por ejemplo:



```
2 + 3 > 4 # True, porque primero se realiza la suma (2 + 3 = 5) y luego la comparación (5 > 4).
```

Condiciones y ejecución condicional

En la vida diaria, tomamos decisiones basándonos en condiciones: "Si está lloviendo, llevo paraguas; si no, no lo llevo". En programación, ocurre lo mismo: necesitamos tomar decisiones basándonos en ciertas condiciones.

En Python, esto se logra utilizando la sentencia `if`, que nos permite hacer preguntas dentro de nuestro código y actuar según las respuestas, ya sea `True` (verdadero) o `False` (falso).

La sentencia `if`

Cuando queremos comprobar si se cumple una condición, usamos `if`. Su sintaxis es muy simple:

```
if condición:  
    consecuencia
```

La condición es una pregunta que Python evalúa como `True` o `False`. Si es `True`, se ejecuta la consecuencia (el código dentro del bloque). Si es `False`, Python simplemente lo ignora.

ejemplo:



```
>>> alumno_nombre = "Juan"  
>>> alumno_edad = 18  
>>>  
>>> if alumno_edad ≥ 18:  
>>>     print("El alumno", alumno_nombre, "es mayor de edad")
```

En este ejemplo:

- Definimos las variables `alumno_nombre` y `alumno_edad`.
- Con `if`, verificamos si `alumno_edad` es mayor o igual a 18.
- Si la condición es `True`, Python imprime: "El alumno Juan es mayor de edad".

 **Nota importante, cuidado con la sangría:** En Python, la sangría es fundamental. Todo el código que pertenece al bloque del `if` debe estar indentado (usualmente con cuatro espacios). Si agregamos más líneas dentro del bloque `if`, todas deben tener la misma sangría.

Agregando una alternativa con `else`

A veces, queremos que el programa haga algo diferente si la condición no se cumple. Para eso usamos `else`.

if condición:

 consecuencia_si_true

else:

 consecuencia_si_false

Veamos el ejemplo anterior, pero esta vez considerando también a los menores de edad:



```
>>> alumno_nombre = "Juan"
>>> alumno_edad = 17
>>> if alumno_edad ≥ 18:
>>>     print("El alumno", alumno_nombre, "es mayor de edad")
>>> else:
>>>     print("El alumno", alumno_nombre, "es menor de edad")
```

En este caso:

- Si `alumno_edad` es mayor o igual a 18, se imprime que el alumno es mayor de edad.
- Si no, se imprime que es menor de edad.

Condiciones múltiples con **elif**

¿Qué pasa si queremos comprobar varias condiciones diferentes? Por ejemplo, queremos clasificar a los alumnos según su edad:

- Mayor de 40 años
- Mayor de 30 años
- Mayor de 20 años
- Menor de 20 años

Podríamos usar **if** y **else** anidados, pero eso puede ser difícil de leer. En su lugar, usamos **elif**, una forma más compacta de decir "else if".

```
if condición1:
    consecuencia1
elif condición2:
    consecuencia2
elif condición3:
    consecuencia3
else:
    consecuencia4
```

Veamos el ejemplo:

```
alumno_nombre = "Juan"
alumno_edad = 18

if alumno_edad > 40:
    print("El alumno", alumno_nombre, "es mayor de 40 años")
elif alumno_edad > 30:
    print("El alumno", alumno_nombre, "es mayor de 30 años")
elif alumno_edad > 20:
    print("El alumno", alumno_nombre, "es mayor de 20 años")
else:
    print("El alumno", alumno_nombre, "es menor de 20 años")
```

¿Qué hemos aprendido?

1. **if**: Evalúa una condición y ejecuta un bloque de código si es verdadera.
2. **else**: Especifica qué hacer si la condición es falsa.
3. **elif**: Permite verificar varias condiciones en cascada.

Buenas prácticas con **if, **elif** y **else**:**

- Cada **else** corresponde al **if** del mismo nivel de sangría.
- **else** siempre es la última rama de una cascada y es opcional.
- Si una rama **if** o **elif** es verdadera, las siguientes ramas no se evalúan.

UNIDAD 4

ITERACIONES | OPERADORES RELACIONALES

En este módulo aprenderemos sobre:

- Repetición de la ejecución de código usando los bucles (**while**).
- Operadores relacionales.

Bucles (`while`)

Un **bucle** o **loop** es una estructura que nos permite ejecutar un bloque de código varias veces. En Python, un bucle `while` se escribe de la siguiente manera:

```
while expresion_condicional:  
    Bloque de instrucciones  
    Más instrucciones...
```

¿Notas alguna similitud con la instrucción `if`? Es completamente normal. De hecho, la única diferencia en la sintaxis es la palabra clave: usamos `while` en lugar de `if`.

La verdadera diferencia está en su **comportamiento**:

- Cuando usamos `if`, las instrucciones dentro del bloque se ejecutan ****una sola vez**** si la condición es verdadera.
- En cambio, con `while`, las instrucciones se ejecutan ****repetidamente**** mientras la condición sea verdadera.

En otras palabras, una vez que el programa termina de ejecutar las instrucciones dentro del bloque, regresa al inicio del `while` para verificar nuevamente la condición. Si la condición sigue siendo verdadera, el programa repite las instrucciones. Este ciclo continúa hasta que la condición sea falsa, momento en el cual el programa sale del bucle y continúa con la línea de código siguiente.

Reglas importantes sobre `while`

1. Si deseas ejecutar más de una instrucción dentro del bucle, todas deben tener ****sangría**** (espaciado) al mismo nivel.
2. El conjunto de instrucciones dentro del `while` se conoce como el ****cuerpo del bucle****.
3. Si la condición es falsa desde el principio, el cuerpo del bucle no se ejecutará ni una sola vez.
4. Es importante que las instrucciones dentro del cuerpo del bucle ****modifiquen la condición**** en algún momento; de lo contrario, el bucle podría ejecutarse indefinidamente (esto se llama un "bucle infinito").

Bucle infinito

Un ****bucle infinito**** ocurre cuando la condición del `while` nunca se vuelve falsa, lo que provoca que el programa quede atrapado en una repetición interminable. Por ejemplo:



```
>>> while True:
>>>     print("Esto es un bucle infinito")
```

Para salir de un bucle infinito mientras el programa se está ejecutando, puedes presionar `Ctrl + C` (o `Ctrl + Break` en algunas computadoras). Esto generará una excepción llamada `KeyboardInterrupt`, que detendrá el programa.

Veamos un ejemplo práctico:



```
>>> pares = 0
>>> impares = 0

>>> while True:
>>>     numero = int(input("Ingresa un número (0 para salir): "))
>>>     if numero == 0:
>>>         break # Salimos del bucle
>>>     if numero % 2 == 0:
>>>         pares += 1
>>>     else:
>>>         impares += 1
>>>
>>> print("Números pares:", pares)
>>> print("Números impares:", impares)
```

Explicación del programa:

- Este programa permite al usuario ingresar una cantidad ilimitada de números.
- Cuando el usuario ingresa `0`, el bucle se detiene.
- Durante cada iteración, el programa verifica si el número ingresado es par o impar, y aumenta el contador correspondiente.
- Al final, muestra el total de números pares e impares ingresados.

Usando un contador para limitar las iteraciones

En lugar de permitir que el usuario decida cuándo finalizar, podemos usar una **variable contador** para limitar el número de repeticiones. Por ejemplo:



```
>>> contador = 5
>>>
>>> while contador > 0:
>>>     print("Estoy dentro del bucle ... Quedan", contador, "iteraciones")
>>>     contador -= 1 # Restamos 1 al contador
>>>
>>> print("Fin del programa")
```

Explicación:

1. El bucle comienza con un contador inicial de 5.
2. En cada iteración, el valor de `contador` disminuye en 1.
3. Cuando `contador` llega a 0, la condición del `while` se vuelve falsa y el programa sale del bucle.

Otra forma de escribir la misma condición:

En Python, escribir `while contador:` es equivalente a escribir `while contador != 0:`. Esto se debe a que cuando una variable entera tiene el valor `0`, se interpreta como `False`.

El bucle `while` y la rama `else`

Una característica interesante de los bucles en Python es que podemos combinar `while` con una rama `else`. La rama `else` se ejecutará **una sola vez**, al final del bucle, **siempre y cuando el bucle no se interrumpa con un `break`**.



```
>>> n = 3
>>>
>>> while n > 0:
>>>     print("Valor de n:", n)
>>>     n -= 1
>>> else:
>>>     print("El bucle terminó correctamente")
```

Explicación:

- Mientras `n > 0`, el bucle se ejecuta.
- Cuando `n` llega a 0, la condición es falsa y se ejecuta el bloque `else`.

Agrandamos un poco la parte lógica

Hasta ahora, hemos trabajado con condiciones bastante simples. Sin embargo, en la vida cotidiana, las decisiones suelen depender de múltiples factores. Consideremos este ejemplo:

"Si tengo tiempo libre y hay buen clima, saldré a caminar."

Aquí usamos un **y**, lo que significa que saldremos a caminar solo si ambas condiciones se cumplen al mismo tiempo. En el lenguaje de la lógica, esta conexión entre condiciones se llama **conjunción**.

Veamos otro ejemplo:

"Si tengo efectivo o tarjeta de crédito, iré al supermercado."

En este caso, usamos un **o**, lo que significa que ir al supermercado depende de que al menos una de las dos condiciones se cumpla. En lógica, esta composición se llama **disyunción**.

Python, como cualquier lenguaje de programación, incluye operadores para expresar este tipo de relaciones lógicas. Sin estos operadores, sería imposible construir condiciones complejas. A estos los llamamos ****operadores lógicos****.

Los operadores lógicos en Python

En Python, los operadores lógicos principales son:

- **AND** (conjunción): Combina dos condiciones y evalúa como `True` solo si ambas son verdaderas.
- **OR** (disyunción): Evalúa como `True` si al menos una de las condiciones es verdadera.
- **NOT** (negación): Invierte el valor lógico de una condición.

AND (conjunción lógica)

El operador ****AND**** permite verificar si se cumplen dos condiciones simultáneamente. Su tabla de verdad es:

Condición A	Condición B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False



```
>>> # Verificar si ambas condiciones son verdaderas
>>> edad = 25
>>> salario = 30000
>>> if edad > 18 and salario > 25000:
>>>     print("Cumple con los requisitos.")
```

En este caso, el mensaje se imprimirá solo si ****edad > 18**** y ****salario > 25000**** son ciertas al mismo tiempo.

OR (disyunción lógica)

El operador ****OR**** verifica si al menos una de las condiciones es verdadera. Su tabla de verdad es:

Condición A	Condición B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False



```
>>> # Verificar si al menos una condición es verdadera
>>> efectivo = 0
>>> tarjeta = True
>>> if efectivo > 0 or tarjeta:
>>>     print("Puedes hacer las compras.")
```

NOT (negación lógica)

El operador **NOT** invierte el valor lógico de una condición. Si algo es `True`, se convierte en `False`, y viceversa.

Condición A	NOT A
True	False
False	True



```
>>> # Invertir una condición
>>> llueve = False
>>> if not llueve:
>>>     print("Podemos salir a caminar.")
```

En este caso, el mensaje se imprimirá solo si `llueve` es falso.

La prioridad de los operadores lógicos

Los operadores lógicos tienen una prioridad específica en Python:

1. **NOT** (más alta)
2. **AND**
3. **OR** (más baja)

Esto significa que una expresión como esta:

a or b and not c

Se evalúa como:

a or (b and (not c))

Si quieres cambiar el orden de evaluación, puedes usar paréntesis:

(a or b) and not c

Ejemplo práctico

Unamos todo lo aprendido con un ejemplo:

```
```python
Verificar múltiples condiciones
edad = 20
salario = 18000
clima = "soleado"

if edad > 18 and salario > 15000 and (clima == "soleado" or clima == "nublado"):
 print("Puedes participar en la actividad al aire libre.")
else:
 print("No cumples con los requisitos.")
...
```
```

Aquí verificamos que la persona cumpla con tres condiciones: tener más de 18 años, ganar más de \$15,000, y que el clima sea favorable.

Recursos adicionales

Antes de continuar, los invito a revisar esta presentación y video, que resumen todo lo aprendido sobre operadores lógicos:

- [Video sobre operadores lógicos](https://iteduccion.com.ar/recursos/python/python3-operadores-logicos.mp4)

Veamos los siguientes ejemplos:

```
1
2 ### and ###
3
4 numero = 15
5
6 if numero > 10 and numero < 20:
7     print("El número está entre 10 y 20.")
8 else:
9     print("El número está fuera del rango de 10 a 20.")
10
11 ### or ###
12
13 numero = 120
14
15 if numero < 0 or numero > 100:
16     print("El número es negativo o mayor que 100.")
17 else:
18     print("El número está entre 0 y 100.")
19
20 ### not ###
21
22 numero = 5
23
24 if not numero == 0:
25     print("El número no es igual a cero.")
26 else:
27     print("El número es igual a cero.")
~
~
```

El operador **and** se utiliza para combinar dos condiciones, y la expresión resultante es True sólo si ambas condiciones son True. Si al menos una de las condiciones es False, la expresión completa se evalúa como False.

El operador **or** se utiliza para combinar dos condiciones, y la expresión resultante es True si al menos una de las condiciones es True. Solo se evalúa como False si ambas condiciones son False.

El operador **not** se utiliza para invertir el valor de una condición. Si una condición es True, not la convierte en False, y si una condición es False, not la convierte en True.

Resumen de como quedaría nuestra tabla de prioridades operacionales:

| OPERADOR | ARIDAD | PRIORIDAD |
|-------------|---------|-----------|
| +, -, NOT | Unario | 1 |
| ** | Binario | 2 |
| *, /, //, % | Binario | 3 |
| +, - | Binario | 4 |
| < <= > >= | Binario | 5 |
| == != | Binario | 6 |
| AND | Binario | 7 |
| OR | Binario | 8 |

Otra forma iterar, Strings, Módulos y Librerías

En este módulo aprenderemos sobre:

- Otras formas de iterar, for()
- Manipulación de strings
- Módulos, Librería Math, Random y DateTime

Otra forma de iterar: el bucle `for`

Introducción al bucle `for`

Hasta ahora, hemos utilizado el bucle `while` para repetir acciones. Sin embargo, en Python existe otra forma de iterar que es más sencilla para ciertos casos: el bucle `for`. Este bucle es especialmente útil cuando sabemos de antemano cuántas veces queremos que se repita el código. Es la herramienta perfecta para repeticiones controladas.

Vamos a verlo con más imaginación para comprender bien lo que es `for`).

Imagina que tienes que entregar volantes en las casas de una calle. Con `while`, estarías constantemente preguntando "¿aún quedan casas por visitar?". Con `for`, simplemente dirías "voy a visitar cada casa de esta calle, una por una". El `for` se encarga automáticamente de llevar la cuenta por ti.

Imaginemos que queremos imprimir los números del 0 al 99. Con `while`, escribiríamos algo así:



```
>>> # Ejemplo con while
```

```
>>> i = 0
>>> while i < 100:
>>>     print(i)
>>>     i += 1
```

El bucle funciona, pero podemos simplificarlo usando `for` y la función `range()`:



```
>>> # Ejemplo con for
>>> for i in range(100):
>>>     print(i)
```

¡Qué simple, verdad? Ahora, exploremos cómo funciona este bucle.

¿Cómo funciona el bucle `for`?



```
>>> for i in range(5):
>>>     print("Hola, alumnos!, línea ", i)
```

Esto imprimirá "Hola, alumnos!" cinco veces. Ahora desglosamos los elementos clave:

1. **La palabra reservada `for`**: Inicia el bucle.
2. **Variable de control (`i`)**: Es la que lleva el "conteo" de las repeticiones. En este caso, toma los valores generados por `range()`. Y los va a ir imprimiendo dentro del bucle.
3. **La palabra reservada `in`**: Conecta el bucle con un conjunto de valores que se recorrerán.
4. **La función `range()`**: Genera los valores para la variable de control.
En este ejemplo, produce los números `0, 1, 2, 3, 4`.

La función `range()`

La función `range()` nos permite controlar el inicio, el final y el incremento de los valores generados.



```
>>> # Uso de range() con un solo argumento
>>> for i in range(5):
>>>     print(i)
>>> # Salida en consola:
>>> # `0, 1, 2, 3, 4`
```



```
>>> ##### Dos argumentos: inicio y fin
>>> for i in range(3, 8):
>>>     print(i)
>>> # Salida en Consola:
>>> # `3, 4, 5, 6, 7`
```

El primer número es el inicio y el segundo indica hasta dónde llegar (sin incluirlo).



```
>>> ##### Tres argumentos: inicio, fin y secuencia
>>> for i in range(2, 17, 3):
>>>     print("El valor de i es:", i)
>>> # Salida en Consola:
>>> # El valor de i es: 2
>>> # El valor de i es: 5
>>> # El valor de i es: 8
>>> # El valor de i es: 11
>>> # El valor de i es: 14
```

Instrucciones especiales: ``break`` y ``continue``

A veces, necesitamos un control más fino sobre la ejecución de los bucles. Python nos ofrece dos herramientas poderosas para ello:

``break``

Detiene el bucle inmediatamente y pasa a la siguiente línea de código fuera del bucle.



```
>>>
>>> for i in range(10):
>>>     if i == 5:
>>>         break
>>>     print(i)
>>>
>>> # Salida en consola:
>>> # 0
>>> # 1
>>> # 2
>>> # 3
>>> # 4
```

2. `**`continue`**`

Salta el resto del código en la iteración actual y pasa directamente al siguiente giro.

```
>>> for i in range(10):
>>>     if i % 2 == 0: # Si es par, saltamos a la próxima iteración
>>>         continue
>>>     print(i)
```

```
>>>  
>>> # Salida: `1, 3, 5, 7, 9`
```

Ambas instrucciones se consideran "azúcar sintáctica" (syntactic sugar), ya que no son indispensables pero hacen nuestro código más limpio y fácil de escribir.

Else dentro del for

qué hace el else en un for?

El bloque **else** en un **for** (o incluso en un **while**) se ejecuta solo si el bucle completa todas sus iteraciones sin que se encuentre un **break**.



```
>>> numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9]  
>>> objetivo = 20  
>>> for num in numeros:  
>>>     if num == objetivo:  
>>>         print(f"¡Encontré el número {objetivo}!")  
>>>         break
```

```
>>> else:
>>>     print(f"No encontré el número {objetivo}")
```

En este caso, al no estar el número 20, el bucle `for` va a recorrer todos los elementos. Y luego va a imprimir “No encontré el número 20”. Entonces el `else` se ejecuta solamente si terminó normalmente el `for` de recorrer todos los elementos. **En otro caso no se ejecuta.**

Otro ejemplo, más simple del `else` sí se ejecuta:



```
>>> for i in range(5):
>>>     print(i)
>>> else:
>>>     print("El bucle terminó normalmente.")
>>>
>>> # Salida:
>>> # 0
>>> # 1
>>> # 2
>>> # 3
>>> # 4
>>> # El bucle terminó normalmente.
```

Manipulando STRINGS

Al igual que otros tipos de datos, las strings tienen su propio conjunto de operaciones permitidas, aunque son bastante limitadas en comparación con los números.

En general, las cadenas pueden ser:

- Concatenadas (unidas). (con +)
- Replicadas. (con *)

La capacidad de usar el mismo operador en tipos de datos completamente diferentes (como números o cadenas) se llama *overloading* - sobrecarga (debido a que el operador está sobrecargado con diferentes tareas).

Veamos un ejemplo de ambas operaciones:



```
>>> subcadena_uno = "mi"
>>> subcadena_dos = "la"
>>> subcadena_tres = "ne"
>>> subcadena_cuatro = "sas"
>>>
>>>
>>> # Sumo las subcadenas
>>> cadena = subcadena_uno + subcadena_dos + subcadena_tres +
subcadena_cuatro
>>> print(cadena)
>>>
>>> # Multiplico la subcadena_uno por 4
>>> print(subcadena_uno * 4)
>>>
>>>
>>> # salida terminal
>>> # $ python operaciones_strings.py
>>> # milanesas
>>> # mimimimi
```

len()

La operación len() (de la palabra length, inglés de la palabra longitud) nos indicará mediante un número entero la cantidad total de caracteres del string indicado.



```
>>> comidas = "milanesas"
>>>
>>> print("La longitud de " + comidas + " es: " + str(len(comidas)))
>>>
>>> # salida terminal
>>> # $ python operaciones_strings.py
>>> #
>>> # La longitud de la palabra milanesas es: 9
```

Notas:

- Cualquier cadena puede estar vacía. Si es el caso, su longitud es 0.
- La diagonal invertida (\) empleada como un carácter de escape, no está incluida en la longitud total de un string.

index[]

Ya dijimos antes que las cadenas de Python son secuencias de caracteres. Es hora de mostrarte lo que significa realmente. Por ejemplo, si deseas acceder a cualquiera de los caracteres de una cadena, puedes hacerlo usando indexación.



```
>>> provincia = "Mendoza"
>>> print(provincia[2])
>>>
```

```
>>> # salida terminal
>>> # $ python operaciones_strings.py
>>> #
>>> # n
```

Puedo recorrer todos los caracteres de un string utilizando un for y mostrando el índice.
Ejemplo



```
>>> ciudad = "Mar del Plata"
>>> len_total = len(ciudad)
>>>
>>> for indice in range(len_total):
>>>     print(ciudad[indice])
```

Acá lo que estamos haciendo es hacer un recorrido de la longitud total de la palabra, entonces la variable “índice” va a ir tomando los número 0,1,2, etc hasta que termine con la longitud, y entonces va mostrando exactamente el índice de ese string, cun hace el “print”

string[x:x:x]

¿Qué pasa cuando queremos extraer un “fragmento” de uno de nuestros strings? Para esto podemos utilizar la siguiente expresión para obtener este resultado:

string[start:end:step]

Su sintaxis está compuesta de:

- El nombre de la variable que contiene nuestro string.
- **START**, indicará la posición inicial desde donde comenzará a indexar nuestro string. En caso de que no sea especificado, se asume que su valor es 0 (recordemos que Python es un

lenguaje con numeración base 0, esto significa que la primera posición en nuestra cadena es el 0). Ingresar un valor negativo nos indicará la posición dentro del string, pero comenzando desde el final (pensémoslo como leer de atrás hacia adelante).

- **END**, indicará la posición final desde donde dejará de indexar nuestro substring. En caso de que este valor no sea especificado o sea mayor al valor del largo de nuestro string, se asume que será igual al valor de `len(string)`. Ingresar un valor negativo tiene el mismo comportamiento que **START**.
- **STEP**, como lo señala su nombre, indica el tamaño de los “saltos” que realizará Python al momento de indexar nuestro substring. Si no es especificado, su valor por defecto es 1.

Nota: estos 3 valores son llamados ÍNDICES y son representados por valores enteros (aunque no todos realizan algo al ingresar valores negativos).

Veamos un par de ejemplos para ver cómo funciona:



```
>>> cadena = "Python es un lenguaje multiproposito"
>>> # mostrar los primeros 6 caracteres
>>> print(cadena[0:6])
>>>
>>> # mostrar la primer posicion
>>> print(cadena[0])

>>> # mostrar la palabra "es un"
>>> print(cadena[7:12])
>>>
>>> # mostrar el último carácter
>>> print(cadena[-1])
>>>
>>> #mostrar la palabra multipropósito
>>> print(cadena[-14:])
>>>
>>> # mostrar la cadena sin los 2 primeros y 2 últimos caracteres
>>> print(cadena[2:-2])
>>>
>>> # Mostrar los caracteres en posición impar
>>> print(cadena[::2])
```

In / not in

El operador IN no debería sorprenderte cuando se aplica a strings, simplemente comprueba si el argumento izquierdo (un string) se puede encontrar en cualquier lugar dentro del argumento derecho (otro string).

El resultado es simplemente True(Verdadero) o False(Falso).

Como seguramente podemos deducir, el operador NOT IN realiza lo opuesto y comprueba si el primer argumento no se puede encontrar en el segundo argumento. Su resultado también es True(Verdadero) o False(Falso).



```
>>> abecedario = "abcdefghijklmnñopqrstuvwxyz"
>>>
>>> if "a" in abecedario:
>>>     print("La letra a está en el abecedario")
>>>
>>> if "33" not in abecedario:
>>>     print("El numero 33 no está en el abecedario")
>>>
>>> if "z" in abecedario:
>>>     print("La letra z está en el abecedario")
```

Dependiendo de la condición va a responder si está o no dentro del abecedario.

index()

El método `index()` (es un método, no una función) busca la secuencia desde el principio, para encontrar el primer elemento del valor especificado en su argumento.

Nota: el elemento buscado debe aparecer en la secuencia, su ausencia causará una excepción `ValueError`.

El método devuelve el índice de la primera aparición del argumento (lo que significa que el resultado más bajo posible es 0).



```
>>> cadena = "Python es un lenguaje multiproposito"
>>>
>>> indice = cadena.index("l")
>>>
>>> print(indice)
>>>
>>> # encuentra la primera ocurrencia de la letra l
>>> # Salida en consola:
>>> # 13
```

Nota:

El método `find()` funciona de la misma manera que el método `index()`, con unas diferencias:

- Es más seguro, no genera un error para un argumento que contiene una subcadena inexistente (devuelve -1 en dicho caso).
- Funciona solo con strings.

count()

El método **`count()`** cuenta todas las apariciones del elemento dentro de la secuencia. La ausencia de tal elemento no causa ningún problema.



```
>>> cadena = "Python es un lenguaje de código abierto"
>>>
```

```
>>> letra_e_cantidad = cadena.count("e")
>>> print("Cantidad de veces que aparece: " + str(letra_e_cantidad))
>>>
>>> # Salida a consola
>>> # Cantidad de veces que aparece la letra e: 5
```

isdigit()

El método **isdigit()** nos va a devolver True si el string es un número o False si no lo es. Esto es importante por ejemplo para saber si lo que ingresó el usuario a través de la terminal es un número.

Documentación:

<https://docs.python.org/3/library/stdtypes.html#str.isdigit>

upper() / lower()

Los métodos **upper()** y **lower()** permiten darle formato a nuestro string, en este caso en particular, convierte todos los caracteres del string en mayúsculas, en el caso de UPPER, y en minúsculas con LOWER



```
>>> cadena = "Python es un lenguaje de código abierto"
>>>
>>> cadena_mayuscula = cadena.upper()
>>> print(cadena_mayuscula)
>>> cadena_minuscula = cadena.lower()
>>> print(cadena_minuscula)
>>>
>>> # Salida a consola:
>>> #
>>> # PYTHON ES UN LENGUAJE DE CÓDIGO ABIERTO
```

```
>>> # python es un lenguaje de código abierto
```

lstrip() / rstrip()

El método sin parámetros `lstrip()` devuelve una cadena recién creada, formada a partir de la original que elimina todos los espacios en blanco iniciales.

El método con un parámetro `lstrip()` hace lo mismo que su versión sin parámetros, pero elimina todos los caracteres incluidos en el argumento, no solo espacios en blanco:

El método sin parámetros `rstrip()` hace exactamente lo mismo, pero desde el lado opuesto del string



```
>>> cadena = "Python es un lenguaje de alto nivel"
>>>
>>> cadena_centrado_izquierda = cadena.ljust(50, '-')
>>> print(cadena_centrado_izquierda)
>>> cadena_centrado_derecha = cadena.rjust(50, '-')
>>> print(cadena_centrado_derecha)
>>>
>>> # Salida a consola:
>>> #
>>> # Python es un lenguaje de alto nivel—————
>>> # —————Python es un lenguaje de alto nivel
```

replace()

El método `replace()` con dos parámetros devuelve una copia de la cadena original en la que todas las apariciones del primer argumento han sido reemplazadas por el segundo argumento. Si el segundo argumento es una cadena vacía, reemplazar significa realmente eliminar el primer argumento de la cadena.

Se puede usar un tercer argumento (un número) para limitar el número de reemplazos.



```
>>> cadena = "Python es un lenguaje orientado a objetos"
>>> print(cadena)
>>> cadena_reemplazada_php = cadena.replace("Python", "PHP")
>>> print(cadena_reemplazada_php)
>>> cadena_reemplazada_javascript = cadena.replace("Python", "Javascript")
>>> print(cadena_reemplazada_javascript)
>>>
>>> # Salida en Consola:
>>> #
>>> # Python es un lenguaje orientado a objetos
>>> # PHP es un lenguaje orientado a objetos
>>> # Javascript es un lenguaje orientado a objetos
```

Las cadenas de Python tienen un número significativo de métodos destinados exclusivamente al procesamiento de caracteres. No esperes que trabajen con otras colecciones. La lista completa se presenta aquí: <https://docs.python.org/3.4/library/stdtypes.html#string-methods>

¿Qué es un módulo?

El código de computadora tiene tendencia a crecer y a ser actualizado, siempre.

El código creciente es, de hecho, un problema creciente. Un código más grande siempre significa un mantenimiento más difícil. La búsqueda de errores siempre es más fácil cuando el código es más pequeño.

Si se desea que dicho proyecto de software se complete con éxito, se deben tener los medios que permitan:

- Dividir todas las tareas entre los desarrolladores.
- Unir todas las partes creadas en un todo funcional.

Cada una de estas partes se puede dividir en otras más pequeñas, y así sucesivamente.

Acá vienen los módulos. Un módulo en Python es un fichero .py que alberga un conjunto de funciones, variables, objetos, entidades y que puede ser usado por otros módulos. Nos permiten reutilizar código y organizarlo mejor. Por ejemplo, podemos definir un módulo `mimodulo.py` con dos funciones `suma()` y `resta()`.

Si se desea utilizar cualquier módulo se necesita saber su nombre. Se entrega una cantidad (bastante grande) de módulos junto con Python. Se puede pensar en ellos como una especie de "equipamiento adicional de Python".

Todos estos módulos, junto con las funciones integradas, forman la Librería Estándar de Python.

Si deseas ver la lista completa de todos los "volúmenes" recopilados en esa librería, se puede encontrar aquí: <https://docs.python.org/3/library/index.html>.

Cada módulo consta de entidades (como un libro consta de capítulos), estas entidades pueden ser funciones, variables, constantes, clases y objetos. Si se sabe cómo acceder a un módulo en particular, se puede utilizar cualquiera de las entidades que almacena.

Importando un modulo

Import

Para que podamos usar un módulo, primero hay que importarlo. La importación de un módulo se realiza mediante una instrucción llamada import.

Nota: import es también una palabra clave reservada (con todas sus implicaciones).

Supongamos que queremos usar entidades proporcionadas por uno de los módulos más usados en python, el módulo math, y que necesitamos la función sin() que sería el equivalente a la función matemática seno.

La forma más sencilla de importar un módulo en particular es usar la instrucción para importar de la siguiente manera:

Import math

¿Fácil, verdad? Esta cláusula contiene la palabra reservada import seguida del nombre del módulo que deseamos importar.

Si necesitamos importar más de un módulo se puede hacer repitiendo la cláusula import, o listando los módulos después de la palabra reservada import.



```
>>> import math
```

```
>>> import sys
```

```
>>>
```

```
# o de la siguiente forma
```

```
>>>
```

```
>>> import math, sys
```

from

O si necesitamos importar alguna entidad específica de un módulo, podemos realizarlo con la siguiente sentencia, utilizando la siguiente sintaxis:



```
>>> from math import cos, radians, pi
>>>
>>> print(math.pi)
```

- La palabra clave reservada from.
- El nombre del módulo a ser parcialmente importado.
- La palabra clave reservada import.
- El nombre o lista de nombres de la entidad o entidades que deseamos importar

dir()

Trabajando con módulos standard

Antes de comenzar a examinar algunos módulos en detalle, veamos la función dir(). La función devuelve una lista ordenada alfabéticamente, la cual contiene todos los nombres de las entidades disponibles en el módulo:



```
>>> import math
>>>
>>> print(dir(math))
>>> # Salida en Consola:
>>> #
>>> # ['__doc__', '__loader__', '__name__', '__package__', '__spec__',
>>> # 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'cbrt',
>>> # 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf',
>>> # 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'floor', 'fmod',
```

```
'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite',  
'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10',  
'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod',  
'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'sumprod', 'tan', 'tanh',  
'tau', 'trunc', 'ulp']
```

¿Notaste los nombres extraños que comienzan con `_` al inicio de la lista? Se hablará más sobre ellos cuando hablemos sobre los problemas relacionados con la escritura de módulos propios.

Modulo math

Comencemos con una vista previa de algunas de las funciones proporcionadas por el módulo `math`.

Se han elegido algunas arbitrariamente, pero esto no significa que las funciones no mencionadas aquí sean menos significativas:

- `ceil(x)` → Redondea el valor ingresado como argumento a su valor entero mayor más cercano.
- `floor(x)` → Redondea el valor ingresado como argumento a su valor entero menor más cercano.
- `pow(x)` → Hace el exponente.
- `sqrt(x)` → Resuelve la raíz cuadrada.

Veamos algunos ejemplos.



```
>>> import math  
>>>  
>>> numero = 4.6  
>>> print("Numero original: ", numero)  
>>> print("Aplicamos ceil() → ", math.ceil(numero))  
>>> print("Aplicamos floor() → ", math.floor(numero))  
>>> print("Aplicamos pow() → ", math.pow(numero, 2))  
>>> print("Aplicamos sqrt() → ", math.sqrt( math.floor(numero) ))
```

El módulo `math()` contiene muchísimos métodos para aplicar. Los podemos encontrar en la documentación de Python, en el siguiente link:

<https://docs.python.org/3/library/math.html>

Por ultimo, veamos 2 funciones: pese a que no forman parte del módulo `MATH`, ya que son funciones de la librería standard de Python, van de la mano con otras funciones similares, como `ceil()` y `floor()`

- `abs(x)` → Devuelve el valor absoluto del valor ingresado como argumento (convierte los valores negativos en positivos, los que ya son positivos no se ven afectados).
- `round(x,y)` → Devuelve un número flotante obtenido luego de redondear el primer valor ingresado como argumento. El segundo argumento le especifica a la función la cantidad de decimales que se desea conservar, este valor es opcional, y en caso de no ser especificado, su valor por defecto es 0.

Modulo random

Random provee algunos mecanismos que nos permiten operar con números pseudoaleatorios.

Nótese que hablamos pseudo; los números generados por Random pueden parecer generados al azar en el sentido de que es difícil predecir los valores subsecuentes, pero no olvidemos que son calculados por una serie de algoritmos.

Los algoritmos no son aleatorios, son determinísticos y predecibles. Solo procesos físicos que suceden completamente fuera de nuestro control (como la intensidad de la radiación cósmica) pueden ser usados como valores verdaderamente aleatorios.

La función general llamada `random()` (no debe confundirse con el nombre del módulo) produce un número flotante `x` entre el rango (0.0, 1.0) - en otras palabras: $0.0 \leq x < 1.0$.

Las series de números aleatorios que generan los ordenadores no son más que secuencias deterministas a partir de un valor inicial, al que se llama **semilla**. En la mayoría de las aplicaciones estas series se parecen suficientemente a números aleatorios. Por lo que no es un problema su utilización.

Es importante notar que al fijar la semilla se fija completamente los valores de toda la serie. Lo que hace este valor importante en múltiples casos.

A continuación, el programa de ejemplo producirá diez valores pseudoaleatorios, ya que sus valores están determinados por el valor semilla actual (bastante impredecible):



```
>>> import random
>>>
>>> print(" Vamos a imprimir diez números random entre 0 y 1")
>>>
>>> for i in range(10):
>>>     print(random.random() * 100)
```

Retomando en concepto de la semilla para generar el random si seteamos una semilla en el mismo valor e intentamos imprimir dos veces diferente con la función random, siempre nos van a dar los mismos valores por que está basado en un mismo valor de semilla.



```
>>> random.seed(3)
>>> print("Serie #1:", random.random(), random.random(), random.random())
>>> random.seed(3)
>>> print("Serie #2:", random.random(), random.random(), random.random())
>>>
```

```
>>> # Salida en Consola:
>>> #
>>> # Serie #1: 0.23796462709189137 0.5442292252959519 0.36995516654807925
>>> # Serie #2: 0.23796462709189137 0.5442292252959519 0.36995516654807925
```

Entonces, si deseas valores aleatorios enteros, una de las siguientes funciones sería mucho mejor:

- `randrange(fin)`
- `randrange(inico, fin)`
- `randrange(inicio, fin, incremento)`
- `randint(izquierda, derecha)`

Probamos alguna:



```
>>> for i in range(10):
>>>     print("Aleatorio del 1 al 10 → ", random.randint(1, 10))
>>>     print("Aleatorio del 20 al 30 → ", random.randrange(20, 30))
>>>     print('_____')
>>>
>>> # salida en Consola:
>>> #
>>> # Aleatorio del 1 al 10 → 10
>>> # Aleatorio del 20 al 30 → 20
>>> # _____
>>> # Aleatorio del 1 al 10 → 8
>>> # Aleatorio del 20 al 30 → 24
>>> # _____
```

choice() / sample()

Hay otros métodos aún mejora para poder trabajar.

Son los siguientes:

- choice(secuencia)
- sample(secuencia, elementos_a_elegir=1)

La primera variante elige un elemento "aleatorio" de la secuencia de entrada y lo devuelve.

El segundo crea una lista (una muestra) que consta del elemento elementos_a_elegir (que por defecto es 1) "sorteado" de la secuencia de entrada.

Veamos un ejemplo:



```
>>> dado = [1, 2, 3, 4, 5, 6]
>>>
>>> print("Dado: ", random.choice(dado))
>>> print(random.sample(dado, 3))
>>> # salida en Consola:
>>> #
>>> # Dado: 3
>>> # [6, 5, 2]
```

UNIDAD 6

FUNCIONES

Una función es un bloque de código diseñado para resolver un trabajo específico. Se agrupa bajo un nombre identificable para poder utilizarlo en cualquier parte del código. Llamarla cuando la necesitemos.

Las funciones permiten organizar y estructurar un programa dividiendo su funcionalidad en partes más pequeñas y manejables. Una función puede recibir datos de entrada (son sus parámetros) y, opcionalmente, devolver un resultado o valor.

El uso de funciones facilita la legibilidad, mantenimiento, y reutilización del código, ya que se puede invocar varias veces en diferentes lugares del programa sin tener que reescribirlo.

Escribir funciones permite dividir el código en bloques que resuelven tareas específicas, facilitando su lectura y comprensión. En lugar de leer miles de líneas, podemos enfocarnos en funciones que describen claramente su propósito. Un buen programador organiza el código en partes más manejables, lo que simplifica el desarrollo y permite probar cada sección de forma independiente. Este proceso se conoce como descomposición.

Entonces, si un fragmento de código se vuelve tan grande que se dificulta leerlo y entenderlo, consideremos la posibilidad de dividirlo en problemas más pequeños e implementar cada uno en forma de una función separada. La descomposición debería continuar hasta conseguir una serie de funciones más pequeñas, fáciles de entender y probar.

(Qué es una función)

<https://www.youtube.com/watch?v=pbWS76Mw-c&t=111>

(1m51s - 4m45s)

Nuestra primera función

Para crear una función es necesario definirla. Veamos como se define y como es la sintaxis:



```
>>> def mi_primer_funcion():
>>>     print("Hola desde mi primer función")
>>>     pass
>>>
>>> # Debemos llamarla para que se ejecute
>>> mi_primer_funcion()
```

- Comienza con la palabra reservada def (que significa definir).
- Sigue el nombre de la función (las reglas para el nombre son las mismas que variables)
- Luego, vienen los paréntesis. Por el momento vacíos.
- La línea debe terminar con los dos puntos.
- Luego, la próxima línea va a comenzar el cuerpo de la función. Todo el bloque indentado.
- Podemos escribir todas las líneas que necesitemos indentadas definiendo todo el bloque.
- Luego, ya por fuera de la indentación, que quiere decir que ya no escribimos más el código de la función, **debemos llamarla**. Ésto lo hace la línea 9.
- Importante! Si no la llamamos, el código anterior no va a mostrar nada en pantalla.

Simple, ¿verdad? Cada vez que invoquemos esta función imprimirá en pantalla el texto.

Ahora vamos a hacer algo más interesante. Vamos a crear un programa que pregunta tres veces al usuario por un valor y luego sume lo ingresado.

De esta manera vamos a ver como podemos hacer para que esa repetición del texto que imprimimos para que muestre en pantalla: "Ingresa un valor", lo podemos hacer una sola vez desde una función.

Parece algo sencillo, una función de este estilo es totalmente utilizable.

Ahora veamos un ejemplo más práctico, en el cual imprimimos en varias ocasiones un mensaje cada vez que es necesario que el usuario ingrese un valor. En lugar de tener una línea para cada mensaje vamos a reemplazarlo por una función que haga dicha impresión.



```
>>> def texto_ingrese_valor():
>>>     print("Ingrese un valor")
>>>     pass
>>>
>>> texto_ingrese_valor()
>>> a = int(input())
>>> texto_ingrese_valor()
>>> b = int(input())
>>> texto_ingrese_valor()
>>> c = int(input())
>>> print("La suma de los valores es: ", a + b + c)
>>>
>>> # Salida en Consola:
>>> #
>>> # Ingrese un valor
>>> # 1
>>> # Ingrese un valor
>>> # 2
>>> # Ingrese un valor
>>> # 3
>>> # La suma de los valores es: 6
```

Flujo de la función:

- Cuando se invoca una función, Python recuerda el lugar donde esto ocurre y salta hacia dentro de la función invocada.
- El cuerpo de la función es entonces ejecutado.
- Al llegar al final de la función, Python regresa al lugar inmediatamente después de donde ocurrió la invocación.
- Este proceso se repite todas las veces que nuestra función sea invocada

Existen dos consideraciones muy importantes, la primera de ella es:

No se debe invocar una función antes de que se haya definido.

Recordemos que Python lee el código de arriba hacia abajo, no va a adelantarse en el código para determinar si la función invocada está definida más adelante. El lugar correcto para definirla es antes de ser invocada, en caso de que no sea así, se interrumpe la ejecución, y un mensaje de error nos indicará que la función no fue definida.

La segunda consideración es más sencilla:

Una función y una variable no pueden compartir el mismo nombre.

Funciones con parámetros.

Una parte muy importante de crear una función es que pueda recibir datos de entrada. Aquí es donde entran los parametros que le pasamos a la misma. Éstos van directamente entre el parentesis cuando la llamamos. Entonces ahora además de llamarla, ya le vamos a pasar los datos que tiene que trabajar.

Un parámetro es una variable, pero existen dos factores que hacen a un parámetro diferente:

- Los parámetros sólo existen dentro de las funciones en donde han sido definidos, y el único lugar donde un parámetro puede ser definido es entre los paréntesis después del nombre de la función, donde se encuentra la palabra reservada def.
- La asignación de un valor a un parámetro de una función se hace en el momento en que la función es invocada, especificando el argumento correspondiente.

Veamos un ejemplo en donde le vamos a pasar el texto que queremos que imprima en pantalla.



```
>>> def imprimir_pantalla( texto_a_imprimir ):
>>>     print(texto_a_imprimir)
>>>     pass
>>>
```



```
>>> texto = "Hola Mundo, pasando parametros!"
>>> imprimir_pantalla(texto)
>>>
>>> # Salida en Consola:
>>> #
>>> # Hola Mundo, pasando parametros!
```

En este ejemplo, a diferencia del anterior estamos pasando el parametro a la función, y la misma imprime en pantalla lo que le pasemos.

Recordemos: especificar uno o más parámetros en la definición de la función es un requerimiento, y se debe cumplir durante la invocación de la misma. Se debe proveer el mismo número de argumentos como haya parámetros definidos.

El no hacerlo provocará un error.

Retorno de la función

Todas las funciones presentadas anteriormente tienen algún tipo de efecto: producen un texto y lo envían a la consola.

Por supuesto, las funciones, al igual que las funciones matemáticas, pueden tener resultados. Para lograr que las funciones devuelvan un valor (aunque no solo para este propósito) se usa la instrucción `return` (regresar o retornar en inglés).

Nota: es una palabra clave reservada de Python.

La instrucción `return` tiene dos variantes diferentes: la primera consiste en la palabra reservada en sí, sin nada que la siga.

Cuando se emplea dentro de una función, provoca la terminación inmediata de la ejecución de la función, y un retorno instantáneo (de ahí el nombre) al punto de invocación, básicamente se utiliza para terminar las actividades de la función antes de que el control llegue al final del cuerpo de la misma. Si una función no está destinada a producir un resultado, emplear la instrucción return no es obligatorio, se ejecutará implícitamente al final.

Veamos este ejemplo con una función simple, vamos a realizar una suma de dos parámetros dentro de la función y devolver el resultado del mismo.



```
>>> def suma(a, b):
>>>     return a + b
>>>
>>> numero_uno = input("Ingrese un valor: ")
>>> numero_uno = int(numero_uno)
>>> numero_dos = input("Ingrese otro valor: ")
>>> numero_dos = int(numero_dos)
>>> print("La suma de los valores es: ", suma(numero_uno, numero_dos))
>>>
>>> # Salida en Consola:
>>> #
>>> # Ingrese un valor: 1
>>> # Ingrese otro valor: 2
>>> # La suma de los valores es: 3
```

Acá podemos observar que la función que definimos provoca la salida con return devolviendo la suma de los valores que le pasamos como entrada.

Y directamente al ser llamada en la línea del print, nos va a imprimir la suma de resultado de la función.

De esta simple manera aprendimos a devolver datos de la función.

Argumentos en las Funciones

Los argumentos son los valores que pasamos a una función al llamarla. Dentro de la función, cada argumento se asigna a un parámetro.



```
>>> def sumar(a, b):  
>>>     return a + b  
>>>  
>>> resultado = sumar(5, 3) # Pasamos dos valores  
>>> print(resultado) # Imprimirá 8
```

Si llamamos a la función con demasiados o muy pocos argumentos, obtendremos un error:

```
>>> sumar(5) # Error: falta un argumento  
>>> sumar(5, 3, 2) # Error: hay un argumento de más
```

Para evitar esto, podemos definir valores por defecto:



```
>>> def sumar(a, b = 0):  
>>>     return a + b  
>>>  
>>> print(sumar(5)) # Como b no se especifica, toma el valor 0. Imprime  
5
```

 **Nota importante:** Los parámetros son los nombres que aparecen en la definición de una función. Por su parte, los argumentos son los valores que le pasamos (y que, por tanto, recibe) una función.

Parámetros con Número Variable de Argumentos

Si queremos que una función acepte cualquier cantidad de valores, usamos `*args`.



```
>>> def sumar_todos(*numeros):
>>>     total = sum(numeros) # sum() suma todos los valores
>>>     return total
>>>
>>> print(sumar_todos(1, 2, 3, 4, 5)) # Imprimirá 15
>>> print(sumar_todos(10, 20)) # Imprimirá 30
```

Aquí, `*args` indica que la función puede recibir cualquier cantidad de valores y los agrupa en una tupla.

Para recibir argumentos con nombre, usamos `**kwargs`:



```
>>> def mostrar_info(**datos):
>>>     for clave, valor in datos.items():
>>>         print(f"{clave}: {valor}")
>>>
>>> mostrar_info(nombre="Juan", edad=25, ciudad="Madrid")
>>>
>>> # Esto imprimirá:
```

```
>>> # nombre: Juan
>>> # edad: 25
>>> # ciudad: Madrid
```

Alcance de Variables en Python

¿Qué es el Scope?

El **scope** (alcance) de una variable en Python indica en qué partes del código se puede usar esa variable. Es decir, define dónde la variable es válida y puede accederse a su valor.

Si creamos una variable dentro de una función, solo existe en esa función y no se puede usar fuera de ella. Es decir, cuando termina la función esas variables no existen más, son eliminadas de la memoria.



```
>>> # Definimos una función con variables locales
>>> def dividir(a, b):
>>>     resultado = a / b # 'resultado' solo existe dentro de la función
>>>     return resultado
>>>
>>> division = dividir(10, 2)
>>>
>>> print(division) # Funciona bien
>>>
>>> print(resultado) # Esto dará un error porque 'resultado' no existe fuera
de la función
```

El error ocurre porque `resultado` se creó dentro de `dividir()` y desaparece cuando la función termina.

Variables Globales

Las variables globales se definen fuera de cualquier función y pueden usarse en todo el código.

Ejemplo:



```
>>> modelo = "Sedan" # Variable global
>>>
>>> def mostrar_modelo():
>>>     print(modelo) # Podemos acceder a 'modelo' dentro de la función
>>>
>>> mostrar_modelo() # Imprimirá "Sedan"
```

Si intentamos modificar una variable global dentro de una función sin indicarlo explícitamente, Python creará una nueva variable local en lugar de cambiar la global:



```
>>> modelo = "Sedan"
>>>
>>> def cambiar_modelo():
>>>     modelo = "SUV" # Se crea una nueva variable local
>>>     print(modelo)
>>>
>>> cambiar_modelo()
>>> print(modelo) # Seguirá imprimiendo "Sedan", la global no cambió
```

Si queremos modificar una variable global desde una función, usamos ``global``:



```
>>> modelo = "Sedan"
>>>
>>> def cambiar_modelo():
>>>     global modelo # Indicamos que queremos cambiar la variable global
>>>     modelo = "SUV"
>>>
>>> cambiar_modelo()
>>> print(modelo) # Ahora imprimirá "SUV"
```

⚠ Nota importante: Modificar variables globales dentro de funciones puede ser problemático y causar errores difíciles de encontrar. Debemos intentar **no usar variables globales dentro de nuestro script**. No es una buena práctica ya que puede traer confusiones a la hora de modificar y/o mejorar el código. Se recomienda siempre trabajar con las variables de manera local, y nunca tener que crear ninguna global. Aunque el lenguaje lo permita.

RESUMEN

- El `*scope*` define en qué partes del código una variable puede usarse.
- Las variables locales existen solo dentro de la función donde se crean.
- Las variables globales existen en todo el código, pero no es recomendable modificarlas dentro de funciones.
- Las funciones pueden recibir argumentos y podemos definir valores por defecto para evitar errores.
- ``*args`` y ``**kwargs`` permiten manejar un número variable de argumentos.

UNIDAD 7

LISTAS

¿Qué es una lista?

Hasta ahora, hemos trabajado con variables que almacenan un solo dato a la vez. Pero, ¿qué pasa si queremos guardar varios valores juntos? Aquí es donde entran en juego las listas.

Una **lista** es un tipo de dato en Python que nos permite almacenar una colección de elementos en un orden específico. Podemos pensar en una lista como una caja con compartimentos donde guardamos distintos objetos (números, palabras, otros elementos), y podemos acceder a cada uno de ellos según su posición.

```
>>> colores = ["rojo", "verde", "azul", "amarillo"]
>>> print(colores)
>>>
>>> # Salida en Consola:
>>> # ['rojo', 'verde', 'azul', 'amarillo']
```

Características de las listas

Las listas en Python tienen varias características importantes:

- **Ordenadas:** Los elementos de una lista tienen un orden definido y se accede a ellos por su posición (llamada "índice").
- **Modificables:** Podemos agregar, cambiar o eliminar elementos.
- **Pueden contener distintos tipos de datos:** Podemos mezclar números, cadenas de texto, booleanos, e incluso otras listas dentro de una lista.
- **Permiten elementos repetidos:** No hay restricción en la cantidad de veces que un mismo valor puede aparecer.

Ejemplo de una lista con diferentes tipos de datos:



```
>>> datos = ["Juan", 25, True, 1.75]
>>> print(datos)
>>>
>>> # Salida en Consola:
>>> # ['Juan', 25, True, 1.75]
```

La función `len()`

Para saber cuántos elementos tiene una lista, usamos la función `len()`:



```
>>> colores = ["rojo", "verde", "azul", "amarillo"]
>>> print(len(colores))
>>>
>>> # Salida en Consola:
>>> # 4
```

Accediendo a los elementos de una lista

Cada elemento de una lista tiene una posición o "índice" que empieza en `**0**`. Para acceder a un elemento, usamos corchetes `[]` con el índice del elemento deseado.



```
>>> colores = ["rojo", "verde", "azul", "amarillo"]
>>> print(colores[0]) # Primer elemento
>>> print(colores[2]) # Tercer elemento
>>>
>>> # Salida en Consola:
>>> # rojo
>>> # azul
```

Podemos también usar índices negativos para acceder desde el final:



```
>>> colores = ["rojo", "verde", "azul", "amarillo"]
>>>
>>> print(colores[-1])
>>> print(colores[-2])
>>>
>>> # Salida en Consola:
>>> # amarillo
>>> # azul
```

Este es solo el comienzo. En los siguientes apartados aprenderemos a modificar, agregar y eliminar elementos de una lista para aprovechar toda su potencia en Python.

Métodos principales de las listas en Python

`append(elemento)` – Agrega un elemento al final de la lista.



```
>>> numeros = [1, 2, 3]
>>> numeros.append(4)
>>> print(numeros)
>>> # Salida en Consola:
>>> # [1, 2, 3, 4]
```

`insert(posición, elemento)` – Inserta un elemento en una posición específica.



```
>>> numeros = [1, 2, 4]
>>> numeros.insert(2, 3)
>>> print(numeros)
>>> # Salida en Consola:
>>> # [1, 2, 3, 4]
```

`remove(elemento)` – Elimina la primera ocurrencia de un elemento.



```
>>> colores = ["rojo", "verde", "azul"]
```

```
>>> colores.remove("verde")
>>> print(colores)
>>>
>>> # Salida en Consola:
>>> # ['rojo', 'azul']
```

`pop([índice])` – Elimina y devuelve un elemento según su índice (por defecto, el último).



```
>>> numeros = [1, 2, 3, 4]
>>> eliminado = numeros.pop()
>>> print(numeros)
>>> print(eliminado)
>>>
>>> # Salida en Consola:
>>> # [1, 2, 3]
>>> # 4
```

`index(elemento)` – Devuelve la posición de la primera aparición de un elemento.



```
>>> colores = ["rojo", "verde", "azul"]
>>> print(colores.index("azul"))

>>> # Salida en Consola:
>>> # 2
```

`count(elemento)` – Cuenta cuántas veces aparece un elemento en la lista.



```
>>> numeros = [1, 2, 3, 1, 1, 4]
>>> print(numeros.count(1))

>>> # Salida en Consola:
>>> # 3
```

`sort([reverse=False])` – Ordena la lista en orden ascendente (o descendente si `reverse=True`).



```
>>> numeros = [3, 1, 4, 2]
>>> numeros.sort()
>>> print(numeros)
>>> numeros.sort(reverse=True)
>>> print(numeros)
>>>

>>> # Salida en Consola:
>>> # [1, 2, 3, 4]
>>> # [4, 3, 2, 1]
```

`reverse()` – Invierte el orden de los elementos de la lista.



```
>>> numeros = [1, 2, 3]
>>> numeros.reverse()
>>> print(numeros)
>>>
```

```
>>> # Salida en Consola:  
>>> # [3, 2, 1]
```

Función sorted()

Si necesitamos ordenar una lista sin modificar el orden original de sus elementos, podemos utilizar la función `sorted()`. A diferencia del método `.sort()`, que altera la lista original, `sorted()` devuelve una nueva lista ordenada sin afectar la estructura original.



```
>>> # Ordenamiento básico de una lista  
>>> numeros = [5, 2, 8, 1, 9]  
>>> ordenados = sorted(numeros)  
>>> print(ordenados) # [1, 2, 5, 8, 9]  
>>>  
>>> # Ordenamiento inverso  
>>> ordenados_inverso = sorted(numeros, reverse=True)  
>>> print(ordenados_inverso) # [9, 8, 5, 2, 1]  
>>>  
>>> # Ordenamiento de strings  
>>> palabras = ["python", "javascript", "ruby", "java"]  
>>> ordenadas = sorted(palabras)  
>>> print(ordenadas) # ['java', 'javascript', 'python', 'ruby']
```

FUNCIONES | METODOS

La diferencia principal entre funciones y métodos es su contexto de uso:

Funciones

- Son bloques de código **independientes** que realizan una tarea específica.
- Pueden ser llamadas desde cualquier lugar en el código, siempre que estén definidas o accesibles.
- En muchos lenguajes, como Python, hay funciones predefinidas (como `print()` o `len()`) y puedes crear tus propias funciones.



```
>>> # ejemplo de una función creada
>>> def sumar(a, b):
>>>     return a + b
>>>
>>> print(sumar(2, 3)) # Salida en consola: 5
>>>
>>> # ejemplo de una función nativa
>>>
>>> print(len("Python")) # Salida en consola: 6
>>>
>>> # Son dos funciones diferentes, una creada por nosotros y otra nativa de
Python
```

Métodos

- Son ****acciones**** que pertenecen a un objeto.
- Se usan para realizar tareas con los datos del objeto.
- Algunos métodos ya vienen incluidos en Python y permiten manipular diferentes tipos de datos.



```
>>> # ejemplo con strings
>>> texto = "hola mundo"
>>> resultado = texto.upper() # Convierte el texto a mayúsculas
>>> print(resultado) # "HOLA MUNDO"
>>>
>>> # ejemplo con listas
>>> numeros = [3, 1, 4, 1, 5]
>>> numeros.sort() # Ordena la lista en orden ascendente
>>> print(numeros) # [1, 1, 3, 4, 5]
```

En resumen, una **función** es un bloque de código independiente que realiza una tarea específica y puede usarse en distintos contextos. Y un **método** es una función especial que pertenece a un objeto y permite trabajar con sus datos. Diferentes tipos de datos tienen distintos métodos disponibles.

Recorrer una lista con un bucle **for**

Podemos usar un bucle **for** para recorrer una lista sin necesidad de manejar índices manualmente. Veamos un ejemplo donde sumamos todos los valores de una lista:



```
>>> numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>
>>> for numero in numeros:
>>>     print(numero)
>>>
>>> # Salida en Consola:
>>> # 1
>>> # 2
>>> # 3 .....
>>>
>>>
>>> multiplos = [7, 14, 21, 28, 35]
>>> total = 0
```

```
>>>
>>> for numero in multiplos:
>>>     total += numero
>>>
>>> print(total)
>>> # Salida en Consola:
>>> # 105
```

Intercambiar valores en una lista

Supongamos que queremos invertir el orden de los elementos de una lista. Antes de hacerlo, aprendamos cómo intercambiar el valor de dos variables, mediante la asignación multiple.



```
>>> a = 1
>>> b = 2
>>> a, b = b, a # Ahora a vale 2 y b vale 1
```

Podemos aplicar esto para intercambiar elementos en una lista:



```
>>> lista = [1, 2, 3, 4, 5]

>>> lista[0], lista[4] = lista[4], lista[0]
>>>
>>> # Intercambiamos el segundo y penúltimo
```

```
>>> lista[1], lista[3] = lista[3], lista[1]
>>> print(lista) # Salida: [5, 4, 3, 2, 1]
```

Si la lista es muy larga, podemos automatizarlo con un bucle:



```
>>> lista = [1, 2, 3, 4, 5]
>>> largo = len(lista)
>>>
>>> for i in range(largo // 2):
>>>     lista[i], lista[largo - i - 1] = lista[largo - i - 1], lista[i]
>>>
>>> print(lista)
>>>
>>> # Salida en Consola:
>>> # [5, 4, 3, 2, 1]
```

La parte principal de este script es que se intercambian los elementos opuestos usando asignación múltiple, que aprendimos más arriba.

Listas como argumentos de funciones

Las listas pueden ser pasadas como argumentos a funciones. Veamos una función que recibe una lista y devuelve la suma de sus elementos:



```
>>> def sumar_lista(lista):
>>>     total = 0
```

```
>>>     for elem in lista:
>>>         total += elem
>>>     return total
>>>
>>> print(sumar_lista([8, 7, 3, 4, 9]))
>>> # Salida en Consola:
>>> # 31
```

Devolver listas desde funciones

Las funciones también pueden devolver listas. A continuación, una función que genera una cuenta regresiva:



```
>>> def cuenta_regresiva(n):
>>>     lista = []
>>>     for i in range(n, -1, -1):
>>>         lista.append(i)
>>>     return lista
>>>
>>> print(cuenta_regresiva(8))
>>>
>>> # Salida en Consola:
>>> # [8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Encontrando extremos: `min()` y `max()`

Son como el termómetro de tus números. Miden quién es el más pequeño y el más grande en la lista.



```
>>> temperaturas = [23, 17, 25, 19, 20]
>>>
>>> print(min(temperaturas)) # 17 → El día más frío
>>> print(max(temperaturas)) # 25 → El día más caluroso
```

Funciona con:

- Números enteros y decimales
- Listas de letras (comparando orden alfabético)



```
>>> letras = ['z', 'a', 'm']
>>> print(min(letras))
>>> # Salida en Consola:
>>> # a
```

Total acumulado: `sum()`

Es tu calculadora personal para sumar **todos** los elementos numéricos de una lista.



```
>>> gastos_semana = [1500, 300, 750, 425]
>>> total = sum(gastos_semana)
>>>
>>> print(f"Gastaste ${total} esta semana")
>>> # Salida en Consola:
>>> # Gastaste $2975 esta semana
```

 **Cuidado:** Sólo funciona con números. Si intentas sumar textos Python va a arrojar un error.

Creando listas numéricas: `range()` + `list()`

El dúo dinámico para generar secuencias de números automáticamente.



```
>>> # Del 0 al 4 (5 no se incluye)
>>> print(list(range(5)))
>>> # [0, 1, 2, 3, 4]
>>>
>>> # Del 2 al 6 (7 no se incluye)
>>> print(list(range(2, 7)))
>>> # [2, 3, 4, 5, 6]
>>>
>>> # De 10 a 50, de 5 en 5
>>> print(list(range(10, 51, 5)))
>>> # [10, 15, 20, 25, 30, 35, 40, 45, 50]
```

UNIDAD 8

TUPLAS | DICCIONARIOS

Para comprender las tuplas, es esencial dominar dos conceptos clave: **tipos de secuencia** y **mutabilidad**.

Tipos de Secuencia

En Python, una **secuencia** es un tipo de dato que almacena múltiples elementos organizados en un orden específico. Su principal característica es que permite acceder a sus elementos de manera secuencial, ya sea mediante índices o iterando con estructuras como el bucle `for`. Ejemplos comunes de secuencias incluyen:

- Listas: Colecciones ordenadas y modificables (ya trabajadas previamente).
- Strings: Secuencias de caracteres (aunque son inmutables, como veremos más adelante).
- Tuplas: El tema central de esta explicación.

Mutabilidad: ¿Qué significa?

La **mutabilidad** determina si un objeto puede ser modificado después de su creación:

Datos mutables: Pueden alterarse directamente en la memoria. Ejemplo clásico: las **listas**.



```
>>> lista = [1, 2, 3]
>>> lista.append(4)
>>> # Modificación válida → Ahora lista = [1, 2, 3, 4]
```

Datos inmutables: No pueden modificarse una vez creados. Si necesitas cambios, debes crear un nuevo objeto. Ejemplo: ****strings**** y ****tuplas****.

```
```python
texto = "Hola"
texto[0] = "h" # Esto generaría un error, ya que los strings son inmutables
nuevo_texto = "h" + texto[1:] # Solución: crear un nuevo string → "hola"
```
```

Tuplas: Características Clave

Son secuencias **inmutables** y **ordenadas**, generalmente usadas para agrupar datos relacionados.

```
>>> tupla_ejemplo = (10, "Python", True) # Creación de una tupla
>>> otra_tupla = (4, 5, 6)
```

Las creamos utilizando parentesis, a diferencia de las listas que se generan con []

- Inmutabilidad: Una vez creadas, no puedes agregar, eliminar o cambiar sus elementos.



```
>>> tupla = (1, 2, 3)
>>> # tupla[0] = 100 # Error: las tuplas no permiten modificaciones
```

- ****Ventajas****:

- Mayor eficiencia en rendimiento que las listas (para operaciones de lectura).
- Ideales para representar datos fijos (ejemplo: coordenadas `(x, y, z)`).
- Pueden usarse como claves en diccionarios (algo imposible con listas, que son mutables).

Entonces, las tuplas son herramientas esenciales en Python para garantizar la integridad de datos que no deben modificarse. Su inmutabilidad las hace seguras y eficientes, especialmente cuando se trabaja con información constante o como claves en estructuras complejas.

Interacciones con Tuplas

Si queremos leer los elementos de una tupla, funciona de la misma manera con las listas, con los índices o con el for



```
>>> colores = ("rojo", "verde", "azul")
>>>
>>> # Iteración básica
>>> for color in colores:
>>>     print(color.upper())
>>>
>>> # Salida en Consola:
>>> # ROJO
>>> # VERDE
>>> # AZUL
```

Recordar! Las tuplas no pueden ser modificadas, en caso de que intentemos utilizar el método `append`, nos va a arrojar un error de ejecución.

¿Qué más pueden hacer las tuplas?

- La función `len()` acepta tuplas, y un valor igual a la cantidad de elementos que contiene.

- El operador + puede unir tuplas
- El operador * puede multiplicar las tuplas
- Los operadores `in` y `not` funcionan de la misma manera que en las listas.

Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares de clave y valor, permitiendo un acceso rápido a los datos mediante su clave única. A diferencia de las listas, los diccionarios no tienen un orden específico y son mutables, lo que significa que puedes modificar su contenido (añadir, eliminar o actualizar elementos) sin restricciones de tamaño.

Cada clave en el diccionario es única y puede estar asociada a cualquier tipo de dato, desde números y cadenas, hasta listas u otros diccionarios. Esto los convierte en una estructura muy flexible para organizar grandes cantidades de información.

En resumen, un diccionario es un conjunto de pares de clave-valor, donde la clave se usa para identificar rápidamente su valor asociado.

Características principales de los diccionarios:

Claves únicas: Cada clave (key) en un diccionario debe ser única. No se permiten claves duplicadas; si intentas agregar una clave que ya existe, el nuevo valor reemplazará al anterior.

Tipos de datos permitidos para las claves: Las claves pueden ser de varios tipos de datos, como enteros (int), flotantes (float) o cadenas de texto (string). Sin embargo, las claves deben ser inmutables, lo que significa que no se pueden usar listas ni otros diccionarios como claves, pero sí tipos inmutables como tuplas.

Estructura de pares clave-valor: A diferencia de las listas, donde los elementos están numerados por índices, un diccionario almacena sus datos en pares de clave-valor. La clave se utiliza para acceder rápidamente al valor asociado.

Desordenado, pero accesible muy rápido: Los diccionarios son estructuras de datos que permiten almacenar información mediante pares de clave-valor. Antes de la versión 3.7, no se garantizaba el orden de las claves dentro del diccionario, lo que significaba que podían aparecer en cualquier orden al imprimirlos o recorrerlos. Sin embargo, desde Python 3.7, los diccionarios mantienen el orden en que se insertan las claves, lo cual resulta útil en muchos casos prácticos.

Una de sus características más poderosas es la velocidad con la que puedes acceder a un valor utilizando su clave, normalmente en un tiempo constante, independientemente del tamaño del diccionario. Esto lo convierte en una herramienta muy eficiente para buscar datos.

Mutabilidad: Los diccionarios son mutables, es decir, pueden modificarse después de ser creados. Puedes agregar, modificar o eliminar pares clave-valor según sea necesario, sin necesidad de definir su tamaño desde el inicio.

Flexibilidad en los valores: Los valores de un diccionario pueden ser de cualquier tipo de dato, incluidos números, listas, otros diccionarios o cualquier objeto de Python. Esto les otorga una gran versatilidad en el manejo de datos complejos.

Acceso eficiente: Gracias a la estructura de hashing de las claves, los diccionarios permiten una búsqueda rápida de valores usando sus claves, lo que los hace ideales para situaciones en las que se requiere un acceso inmediato a los datos.

Creamos nuestro primer diccionario



```
>>> dict_1 = {
>>>     "Nombre": "Sara",
>>>     "Edad": 27,
>>>     "Documento": 1003882
>>> }
>>> print(dict_1)
>>> # {'Nombre': 'Sara', 'Edad': 27, 'Documento': 1003882}
>>>
>>> dict_2 = dict(Nombre="Sara", Edad=27, Documento=1003882)
>>> print(dict_2)
>>> # {'Nombre': 'Sara', 'Edad': 27, 'Documento': 1003882}
```

El diccionario emplea keys y valores las cuales ambas son strings. En el segundo, las keys son strings pero los valores son enteros. El orden inverso también es posible así como la combinación número a número.

La lista de todos los pares es encerrada con llaves, mientras que los pares son separados por comas, y las keys y valores por dos puntos. Los diccionarios vacíos son construidos por un par vacío de llaves.

Interacciones con diccionarios

Una vez que tenemos nuestros diccionarios armados, ¿cómo accedemos a ellos? Para obtener cualquiera de los valores que contiene, debemos proporcionar una key válida.

El obtener el valor de un diccionario es semejante a la indexación, gracias a los corchetes alrededor del valor de la key. Siempre teniendo en cuenta que si la key es un string, se debe especificar como tal. Y debemos asegurarnos que la key exista, o bien comprobar antes mediante el uso de IN o NOT IN

Vamos a ver ejemplos de accesos a los diccionarios.



```
>>> nro_telefono = {
>>>     'juan': 123456,
>>>     'pedro': 987654,
>>>     'maria': 456789
>>> }
>>>
>>> print("El numero de telefono de Juan es: ", nro_telefono['juan'])
>>> print("El numero de telefono de Pedro es: ", nro_telefono['pedro'])
>>>
>>> # Para asegurarnos que la clave existe en el diccionario
>>> if 'maria' in nro_telefono:
>>>     print("El numero de telefono de Maria es: ", nro_telefono['maria'])
>>> else:
>>>     print("Maria no tiene numero de telefono")
>>>
>>> # Si intentamos llamar a un diccionario por una clave que no existe, nos
va a dar un error
>>> print("El numero de telefono de Ana es: ", nro_telefono['ana'])
>>>
>>> # Salida en Consola:
>>> # El numero de telefono de Juan es: 123456
```

```

>>> # El numero de telefono de Pedro es: 987654
>>> # El numero de telefono de Maria es: 456789
>>> # Traceback (most recent call last):
>>>                                     #                                     File
"/var/www/html/istea/infra/programacion.lunes.2do.cuatr.2024/unidad-05/enclas
e/01-for.py", line 20, in <module>
>>> #     print("El numero de telefono de Ana es: ", nro_telefono['ana'])
>>> #                                     ~~~~~^~~~~^~~~~^~~~~^~~~~^~~~~^
>>> # KeyError: 'ana'

```

Método keys()

El método `keys()` de un diccionario en Python devuelve una vista que contiene todas las claves del diccionario.

Esto permite recorrer fácilmente el diccionario utilizando un bucle `for`, ya que convierte las claves en una estructura iterable. Aunque los diccionarios no son secuenciales por naturaleza, el método `keys()` permite adaptarlos para este tipo de operaciones.

Una vez que tienes la lista de claves, puedes acceder a los valores correspondientes a través de esas claves, haciendo el recorrido y manipulación del diccionario más simple.



```

>>> # Creamos un diccionario con información de una persona
>>> persona = {
>>>     "nombre": "Ana",
>>>     "edad": 25,
>>>     "ciudad": "Buenos Aires"
>>> }
>>>
>>> # Usamos el método .keys() para obtener las claves del diccionario
>>> claves = persona.keys()
>>>
>>> # Mostramos las claves
>>> print(claves) # Salida: dict_keys(['nombre', 'edad', 'ciudad'])

```

```
>>>
>>> # Podemos recorrer las claves con un bucle for
>>> for clave in persona.keys():
>>>     print(clave)
```

Metodo values()

Así como tenemos el método keys(); también Python nos ofrece el método values() para trabajar directamente con los valores del diccionario.



```
>>> persona = {
>>>     "nombre": "Ana",
>>>     "edad": 25,
>>>     "ciudad": "Buenos Aires"
>>> }
>>>
>>> print("Valores del diccionario:")
>>> for valor in persona.values():
>>>     print(valor)
```

Método items()

Y después tenemos un método un poco más complejo que directamente nos devuelve el par, key, value, en una tupla directamente.



```
>>> # Crear un diccionario de frutas y sus precios
>>> frutas = {
>>>     "manzana": 2.5,
>>>     "banana": 1.8,
>>>     "naranja": 3.0,
>>>     "uva": 4.5
>>> }
>>>
>>> # Iterar sobre cada par clave-valor usando items()
>>> for fruta, precio in frutas.items():
>>>     print(f"La {fruta} cuesta ${precio} por kilo.")
```

Interacciones con diccionarios

El asignar un nuevo valor a una key existente es sencillo, debido a que los diccionarios son completamente mutables, no existen obstáculos para modificarlos, se va a reemplazar el valor "chat" por "minou", lo cual no es muy adecuado, pero funcionará con nuestro ejemplo.

Para agregar una nueva key con su valor a un diccionario es tan simple como cambiar un valor, solo se tiene que asignar a una nueva key que no haya existido antes.

Por último, con la instrucción del(), podemos eliminar una key del diccionario, tengamos en cuenta que hacer esto también elimina el valor asociado a dicha key.

Vemos en el código directamente:



```
>>> # Crear un diccionario de animales y sus nombres en francés
```



```
>>> diccionario_animales = {
>>>     "perro": "chien",
>>>     "gato": "chat",
>>>     "pez": "poisson"
>>> }
>>>
>>> # Modificar el valor asociado a una clave existente
>>> print("Diccionario original:", diccionario_animales)
>>> diccionario_animales["gato"] = "minou" # Cambiar "chat" por "minou"
>>> print("Después de modificar 'gato':", diccionario_animales)
>>>
>>> # Agregar una nueva clave-valor al diccionario
>>> diccionario_animales["pájaro"] = "oiseau"
>>> print("Después de agregar 'pájaro':", diccionario_animales)
>>>
>>> # Eliminar una clave y su valor asociado
>>> del diccionario_animales["pez"]
>>> print("Después de eliminar 'pez':", diccionario_animales)
```

También podemos combinar los tipos de datos y utilizar diccionarios y Tuplas,
Ejemplo:



```
# Crear un diccionario donde las claves son nombres de países y los valores
son tuplas con coordenadas geográficas
>>> coordenadas_paises = {
>>>     "Argentina": (-34.6037, -58.3816), # Latitud, Longitud
>>>     "Brasil": (-15.7801, -47.9292),
>>>     "Chile": (-33.4489, -70.6693),
>>>     "Perú": (-12.0464, -77.0428)
>>> }
>>>
>>> # Acceder a las coordenadas de un país específico
>>> pais = "Brasil"
>>> latitud, longitud = coordenadas_paises[pais]
>>> print(f"Las coordenadas de {pais} son: Latitud {latitud}, Longitud
{longitud}")
>>>
>>> # Iterar sobre el diccionario para mostrar todos los países y sus
coordenadas
>>> print("\nCoordenadas de todos los países:")
>>> for pais, coordenadas in coordenadas_paises.items():
>>>     print(f"{pais}: Latitud {coordenadas[0]}, Longitud {coordenadas[1]}")
```

UNIDAD 9

TRATAMIENTO DE ERRORES

Los errores en la programación son inevitables, incluso para los mejores programadores.

Aunque aspiramos a la perfección, es importante entender que cometer errores es parte natural del proceso. No se trata de evitar fallos, sino de aprender a manejarlos.

Cada error solucionado mejora nuestro código, y con paciencia y perseverancia, logramos desarrollar software más robusto y confiable.

Como programadores y programadoras tenemos que amigarnos con los errores, por que nuestro día se va a basar básicamente en saber tratar y lidiar con los errores.

Diferentes tipos de errores

Errores de Sintaxis: Ocurren cuando el código no sigue las reglas del lenguaje. Son fáciles de detectar, ya que los editores suelen resaltarlos y el intérprete avisa antes de ejecutar el código.
Ejemplo: Olvidar cerrar comillas en una cadena.

Errores Semánticos o Lógicos: El código se ejecuta, pero con lógica incorrecta. Ejemplo: Usar el operador equivocado en una comparación. Son más difíciles de detectar por que no nos avisa el interpretador.

Errores en Tiempo de Ejecución: Suceden mientras el programa corre, como intentar dividir por cero o acceder a índices fuera de rango. Nos vamos a enterar con seguridad por la interrupción brusca.

En resumen podemos dividir los errores en:

De sintaxis (detectados antes de la ejecución)

En tiempo de ejecución (errores por excepciones, se detiene.)

Lógicos (comportamiento inesperado sin mensajes de error)

Entender y gestionar estos errores es esencial para crear código más robusto. Python ofrece herramientas como excepciones para ayudar a manejarlos eficazmente.

En Python, uno de los principios fundamentales al trabajar con errores es el estilo "Easier to Ask Forgiveness than Permission" (EAFP), traducida: Es más fácil pedir perdón que permiso. Esto significa que, en lugar de revisar muchas condiciones antes de realizar una acción, lo mejor es simplemente intentar hacerla y capturar cualquier error que ocurra.

Este enfoque hace que el código sea más claro, legible y eficiente, ya que evita tener que escribir múltiples comprobaciones innecesarias. Es una forma natural de programar en Python y te ayuda a concentrarte en el flujo principal del programa, manejando los casos especiales sólo cuando surgen.

Esto se logra usando las palabras clave `try` y `except`.

El bloque `try` contiene el código que puede fallar, mientras que el bloque `except` maneja cualquier error que surja. Así, si ocurre una excepción (como intentar dividir por cero), se detiene la ejecución normal y se ejecuta el código en `except` para resolver o notificar el problema.



Documentación oficial, tratamiento de errores.

<https://docs.python.org/es/3/tutorial/errors.html>

Podemos resumir su uso en las siguientes líneas para que se comprenda bien:

- La palabra clave reservada `try` marca el lugar donde intentas ejecutar un código que podría fallar.
- La palabra clave reservada `except` indica el lugar donde puedes manejar el error si ocurre, o notificarlo y detener la ejecución.
- Cualquier código dentro del bloque `try` se ejecuta de manera especial. Si se produce un error, la ejecución no se interrumpe. En su lugar, el control salta directamente a la primera línea del bloque `except`, y el resto del bloque `try` no se ejecuta.
- El código en el bloque `except` solo se ejecuta si ocurre una excepción en el bloque `try`. o se puede acceder a este bloque de ninguna otra forma.
- Una vez que el bloque `try` o `except` se ha ejecutado correctamente, el programa vuelve al flujo normal y continúa con las siguientes líneas de código.



```
>>> numero = input("Ingresa un número: ")
>>>
>>> try:
>>>     numero = float(numero)
>>>     print(f'el reciproco de {numero} es {1/numero}')
>>>
>>> except ValueError:
>>>     print("ERROR: El valor ingresado no es un número....")
>>>     exit()
>>>
>>> except ZeroDivisionError:
>>>     print("ERROR: No se puede dividir por cero ———")
>>>
>>> except as e:
>>>     print("ERROR: Algo salió mal....", e)
>>>
>>> print("Acá continua el programa.....")
```

Dependiendo de lo que ingrese el usuario, podemos manejar diferentes excepciones en Python.

Si el usuario ingresa un número mayor a cero, el bloque `try` se ejecuta sin problemas.

Si ingresa 0, el bloque `except` correspondiente se activa para manejar el error de división, imprimiendo un mensaje en lugar de generar una excepción.

Si ingresa letras en vez de números, se maneja el error de conversión con otro bloque `except` que muestra un mensaje adecuado.

Así, el programa responde de manera específica según el tipo de error que se produzca.

Vamos a ver otro ejemplo completando con `else` y `finally`



```

>>> def dividir(a, b):
>>>     try:
>>>         resultado = a / b
>>>     except ZeroDivisionError:
>>>         print("Error: No se puede dividir por cero.")
>>>     else:
>>>         print(f"Resultado: {resultado}")
>>>     finally:
>>>         print("Fin del intento de división.")
>>>
>>> # Ejemplo de uso
>>> dividir(10, 2) # Caso exitoso
>>> dividir(5, 0) # Caso con excepción

```

try: Intenta ejecutar la operación a / b.

except: Si ocurre un ZeroDivisionError (cuando b es 0), maneja la excepción y muestra un mensaje de error.

else: Se ejecuta solo si no hubo excepción, mostrando el resultado de la división.

finally: Se ejecuta siempre, haya ocurrido o no una excepción.

Excepciones muy utilizadas.

ZeroDivisionError

Se produce cuando intentamos dividir un número por cero o por algo que no puede distinguirse de cero, como en las operaciones `/`, `//` o `%`.

ValueError

Ocurre cuando una función recibe un tipo de dato correcto pero con un valor inapropiado. Por ejemplo, al usar `int()` o `float()` con un valor que no puede convertirse.

TypeError

Aparece cuando intentamos usar un tipo de dato en un contexto inadecuado. Por ejemplo, intentar usar un número decimal (float) como índice de una lista.

AttributeError

Se genera cuando tratamos de acceder o usar un método que no existe para un objeto. Un ejemplo es intentar usar `append()` en una tupla, que es inmutable.

SyntaxError

Ocurre cuando el código no sigue las reglas de la sintaxis de Python. Estos errores solo se detectan cuando el intérprete llega a esa línea de código durante la ejecución.

Es importante tener buenas prácticas en el manejo de las excepciones como las siguientes:

- Evitar bloques except: sin tipo de excepción. `except: print("Ocurrió un error")`
- Capturar excepciones específicas en lugar de genéricas.
Genérica -> `except Exception as e:`
Específica -> `except FileNotFoundError:`
- Usar `finally` para limpieza de recursos. El bloque `finally` se ejecuta siempre, independientemente de si ocurrió una excepción o no. Es ideal para cerrar archivos, conexiones a bases de datos o sockets.
- Registrar errores (logging) en lugar de solo imprimir. Es un módulo para generación de logs.
- No abusar de los bloques try-except (anti-patrón "Swallowing Exceptions"). Se refiere a la práctica de capturar una excepción sin hacer nada significativo con ella, como simplemente ignorarla o silenciarla con un `pass`, `print()` vacío o un mensaje genérico que no ayuda a diagnosticar el error.

UNIDAD 10

ARCHIVOS

Accediendo archivos desde código en Python

El manejo de archivos es esencial para cualquier programador, y en Python, esta tarea es potente y sencilla. Los archivos permiten almacenar datos de manera persistente, fundamental para muchas aplicaciones.

Importancia de los archivos.

Guardar y preservar datos

Los archivos permiten guardar resultados y datos importantes, asegurando que se puedan recuperar y usar en futuras sesiones del programa.

Interacción con el mundo exterior

Los programas de Python a menudo necesitan leer datos de archivos CSV, documentos de texto, o bases de datos. Saber manejar archivos permite integrar tus programas con diversas fuentes y destinos de datos.

Persistencia de datos

La persistencia de datos es la capacidad de mantener información después de que el programa ha terminado. Guardar datos en archivos asegura que la información importante sobreviva más allá de la ejecución del programa.

En este capítulo, aprenderás a abrir y cerrar archivos, leer y escribir datos, y las mejores prácticas para trabajar con archivos de manera eficiente y segura. Estas habilidades son cruciales para manejar archivos y datos efectivamente en Python, mejorando la funcionalidad y utilidad de tus aplicaciones.

Diferentes tratamientos según sistema operativo

Los sistemas operativos tratan los archivos de manera diferente. Por ejemplo, Windows usa una convención de nombramiento distinta a la de Unix/Linux.

Veamos algunas diferencias

| Windows | Unix/Linux |
|-----------------------|---------------------|
| C:\directorio\archivo | /directorio/archivo |

En sistemas Unix/Linux, no se usa una letra para identificar el disco; todos los directorios parten del directorio raíz /. En Windows, el directorio raíz es C:\.

Los nombres de archivos en Unix/Linux son sensibles a mayúsculas, permitiendo archivos como HOLA.txt y hola.txt en el mismo directorio. En Windows, no hay distinción entre mayúsculas y minúsculas para los nombres de archivos.

Windows usa \ como separador de directorios, mientras que Unix/Linux usa /. Aunque Windows reconoce ambos separadores, Unix/Linux solo reconoce /.

Python maneja automáticamente la conversión de barras a contrabarras según el sistema operativo, facilitando el manejo de archivos multiplataforma.

Qué son los handles (Manejadores):

Un **handle** es un identificador único que Python asigna a un archivo cuando se abre. Funciona como un acceso directo para interactuar con el sistema operativo sobre un archivo específico. Al abrir un archivo, Python crea un handle que permite realizar operaciones en él, estableciendo una conexión entre tu programa y el sistema de archivos.

Qué son los streams (Flujos):

Un **stream** es una secuencia de datos que representa el flujo continuo de información que entra o sale de un archivo. En Python, los streams se manejan con métodos como `read()` para leer datos y `write()` para escribir datos. Estos métodos permiten transferir información a través del handle del archivo.

Operaciones básicas a realizar con el stream:

Lectura: Recupera datos del archivo y los almacena en una variable.

Escritura: Transfiere datos de una variable al archivo.

Los streams pueden abrirse en tres modos básicos:

- **Modo Lectura ('r'):** Permite solo leer el archivo. Intentar escribir genera una excepción.
- **Modo Escritura ('w'):** Permite solo escribir en el archivo, sobrescribiendo su contenido. Intentar leer genera una excepción.
- **Modo Actualización ('r+' o 'w+'):** Permite tanto lectura como escritura.

Tipos de Streams: Texto y Binario

Los streams se dividen en dos tipos según su contenido:

- **Streams de Texto:** Estructurados en líneas y compuestos de caracteres tipográficos. Se leen y escriben carácter por carácter o línea por línea.
- **Streams Binarios:** Contienen una secuencia de bytes de cualquier valor (programas ejecutables, imágenes, audio, etc.). Se leen y escriben byte a byte o bloque a bloque.

En este libro, nos enfocaremos en operaciones simples sobre archivos de texto plano.

Abrir un stream:

```
stream = open(file, mode = 'r', encoding = None)
```

- Si la apertura es exitosa, `open()` devuelve un objeto stream; de lo contrario, se genera una excepción, como `FileNotFoundError` si el archivo no existe.
- `file`: especifica el nombre del archivo que se asociará al stream.
- `mode`: Modo de apertura del stream, especificado como una cadena de caracteres. Por ejemplo, 'r' indica un stream de lectura. Si no se especifica, el valor predeterminado es 'r'.
- `encoding`: Tipo de codificación del archivo, como UTF-8 cuando se trabaja con archivos de texto.

Modos de apertura de Streams

'r': Lectura (predeterminado).

'w': Escritura (sobrescribe el archivo si existe).

'a': Añadir (escribe al final del archivo si existe).

'b': Binario (añadido a otros modos, e.g., 'rb' para lectura binaria).

Se permiten operaciones de lectura y escritura en el stream.

Especificando al stream para manejar el archivo

Como hemos mencionado antes, según el contenido, los streams se dividen tanto en tipo binario y tipo texto. Al momento de abrir un nuevo stream, también podemos forzar el modo en el que queremos que se abra el stream.

Si hay una letra 'b' al final de la string del modo significa que el stream se debe abrir en el modo binario.

Si la string del modo termina con una letra 't' el stream es abierto en modo texto. En caso de que no se especifique, este será el modo por defecto.

Vamos a ver un cuadro que detalla los usos del modo en que abrimos el archivo:

| Modo | Descripción |
|-------------------|--|
| <code>'r'</code> | Modo lectura (read). Abre el archivo para lectura. |
| <code>'w'</code> | Modo escritura (write). Abre el archivo para escritura. Crea un nuevo archivo si no existe o trunca el archivo existente. |
| <code>'a'</code> | Modo agregar (append). Abre el archivo para escritura. Crea un nuevo archivo si no existe o coloca el cursor al final del archivo. |
| <code>'r+'</code> | Modo lectura y escritura. Abre el archivo para lectura y escritura. |
| <code>'rb'</code> | Modo binario de lectura. Abre un archivo binario para lectura. |
| <code>'wb'</code> | Modo binario de escritura. Abre un archivo binario para escritura. |
| <code>'ab'</code> | Modo binario de agregar. Abre un archivo binario para agregar contenido. |

Abriendo nuestro primer stream

Vamos a analizar el siguiente código:

```
>>> try:
>>>     archivo = 'recursos/temperaturas.txt'
>>>     stream = open(archivo, 'rt')
>>>     print("El archivo se abrió correctamente....")
>>>     stream.close()
>>> except Exception as e:
>>>     print("NO se pudo abrir el archivo / Error: ", e)
```

- Hemos abierto el bloque `try-except` ya que queremos manejar los errores de tiempo de ejecución.
- Se emplea la función `open()` para intentar abrir el archivo especificado (ten en cuenta la forma en que hemos especificado el nombre del archivo).

- El modo de apertura se define como lectura de texto (como texto es la configuración predeterminada, no es necesario la 't' en el parámetro de modo).
- En caso de éxito obtenemos un objeto de la función `open()` y lo asignamos a la variable del stream, agregamos un `print` para confirmar que el código se ejecutó.
- Si `open()` falla, manejamos la excepción imprimiendo la información completa del error (es bueno saber qué sucedió exactamente).

Cerrando nuestro primer stream

La última operación realizada en un stream debe ser cerrarlo, esa acción se realiza mediante un método invocado desde dentro del objeto del stream: `stream.close()`.

- El nombre de la función es fácil de entender `close()`, es decir cerrar.
- La función no espera argumentos; el stream no necesita estar abierto.
- La función no devuelve nada pero genera una excepción `IOError` en caso de un error.
- La mayoría de los desarrolladores creen que la función `close()` siempre tiene éxito y, por lo tanto, no hay necesidad de verificar si ha realizado su tarea correctamente, esta creencia está sólo parcialmente justificada. Si el stream se abrió para escribir y luego se realizó una serie de operaciones de escritura, puede ocurrir que los datos enviados al stream aún no se hayan transferido al dispositivo físico (debido a los mecanismos de caché o buffer), dado que el cierre del stream obliga a los buffers a descargarse, es posible que dichas descargas fallen y, por lo tanto, `close()` falle también.

Procesamiento de archivos de texto

Ahora veremos algunas técnicas básicas que podemos utilizar para leer el contenido del archivo y procesarlo.

El procesamiento será muy simple: vamos a copiar el contenido del archivo a la consola y contaremos todos los caracteres que el programa ha leído.

Pero, recordemos, nuestra comprensión de un archivo de texto es muy estricta, es un archivo de texto sin formato, puede contener solo texto, sin decoraciones adicionales, como formato,

diferentes fuentes, alineaciones, etc., es por eso que debemos evitar crear el archivo utilizando un procesador de texto avanzado como MS Word, LibreOffice Writer o algo así.

Utilizaremos los conceptos básicos que ofrece nuestro sistema operativo (Bloc de notas, vim, gedit, etc.).

Si nuestros archivos de texto contienen algunos caracteres nacionales no cubiertos por el charset ASCII estándar, es posible que necesitemos un paso adicional. La invocación de `open()` va requerir un argumento que denote una codificación específica del texto.

```
stream = open('file.txt', 'rt', encoding='utf-8')
```

La lectura del contenido de un archivo de texto se puede realizar utilizando diferentes métodos.

Veamos un ejemplo:

```
>>> # file: 02.abriendo_con_while.py
>>>
>>> from os import strerror
>>>
>>> try:
>>>     contador = 0
>>>     archivo = 'agendas.txt'
>>>     stream = open(archivo, 'rt', encoding='utf-8')
>>>     caracter = stream.read(1)
>>>
>>>     while caracter != '':
>>>         print(caracter, end='')
>>>         caracter = stream.read(1)
>>>         contador += 1
>>>
>>>     stream.close()
```

```
>>> print(f'\n\nSe leyeron {contador} caracteres del archivo
{archivo}')
>>>
>>> except Exception as e:
>>> print(f'Error: {strerror(e.errno)}')
```

Analicemos el código:

- Se usa el mecanismo `try / except` y se abre el archivo con el nombre (agendas.txt en este caso).
- Nota como la función `open()`, tiene especificado el tipo de codificación utf-8, esto se debe a que nuestro archivo contiene caracteres con tildes
- Intenta leer el primer carácter del archivo (`char = stream.read(1)`).
- Si tiene éxito (esto se demuestra por el resultado positivo de la condición `while`), se muestra el carácter. Nota: el argumento `end=`, ¡es importante! ¡No querrás saltar a una nueva línea después de cada carácter!.
- También se actualiza el contador (variable `contador`)
- Intenta leer el siguiente carácter y el proceso se repite.

Si quisiéramos manejar el contenido del archivo como un conjunto de líneas, no como un montón de caracteres, el método `readline()` es el que tenemos que usar.

El método intenta leer una línea completa de texto del archivo y la devuelve como una string en caso de éxito. De lo contrario, devuelve una string vacío.

Veamos ej empleo utilizando `readline()`

Asd

```
>>> # file: 02.aaaaaaaaaaaa.py
>>> import os
```

```

>>> from os import strerror
>>>
>>> try:
>>>     contador_caracter = contador_linea = 0
>>>     archivo = os.path.join(os.path.dirname(__file__), 'recursos',
>>> 'agenda.csv')
>>>     stream = open(archivo, 'rt', encoding='utf-8')
>>>
>>>     linea = stream.readline()
>>>
>>>     while linea != '':
>>>         contador_linea += 1
>>>         for caracter in linea:
>>>             print(caracter, end='')
>>>             contador_caracter += 1
>>>
>>>         linea = stream.readline()
>>>
>>>     stream.close()
>>>
>>>     print("\n\n")
>>>     print(f'Se leyeron {contador_caracter} caracteres.')
>>>     print(f'Se leyeron {contador_linea} líneas del archivo {archivo}')
>>>
>>> except IOError as e:
>>>     print(f'Error: {strerror(e.errno)}')

```

Acá además de ver el script realizado con el método `readline()`, podemos ver algo nuevo que hasta el momento no habíamos utilizado.

```
archivo = os.path.join(os.path.dirname(__file__), 'recursos', 'agenda.csv')
```

Construye una ruta al archivo ``agenda.csv``, ubicado en la carpeta ``recursos``, dentro del mismo directorio donde está el script.

- ``__file__`` → Ruta del script actual.

- ``os.path.dirname(__file__)`` → Obtiene el directorio donde está el script.

- `os.path.join(...)` → Une la ruta del directorio con `'recursos/agenda.csv'`, asegurando compatibilidad con cualquier sistema operativo.

Permite acceder al archivo sin importar desde dónde se ejecute el script, evitando rutas absolutas.

Vamos a ver otra manera simple de leer archivos (Utilizamos with):

```
>>> # file: 04.xxxxxxxxxxxx.py
>>> import os
>>> from os import strerror
>>>
>>>
>>> archivo = os.path.join(os.path.dirname(__file__), 'recursos',
'agenda.csv')
>>>
>>> with open(archivo, 'rt', encoding='utf-8') as stream:
>>>     contador_caracter = contador_linea = 0
>>>
>>>     for linea in stream:
>>>         print(linea, end='')
>>>
>>> # Y por otro lado vamos a ir guardando en una lista
>>> with open(archivo, 'rt', encoding='utf-8') as stream:
>>>     lista = stream.readlines()
>>>
>>> print("\n\n")
>>> print(lista)
```

En el script anterior vemos una palabra nueva **with** una manera más simple de leer archivos correctamente.

En la línea 6 y también en la 11 with open(archivo, 'r') as stream: Abre el archivo en modo de lectura ('r') y crea un contexto utilizando la declaración with, lo que garantiza que el archivo se cerrará correctamente después de su uso.

Luego, es lo mismo que ya vimos anteriormente, que con el objeto stream vamos a poder estar trabajando con el archivo.

Lo interesante de esta manera con “with” es que no debemos cerrar el archivo al terminar ya que lo cerrará automáticamente cuando salimos del bloque del with.

Ahora vamos a escribir archivos.

Escribir archivos es más simple, de hecho, hay un solo método que podemos usar para esta tarea: el nombre del método es write() y espera un único argumento, una string que será transferida al archivo (no olvidemos que hay que abrir el archivo en modo escritura, de lo contrario el uso de write() va a fallar).

Debemos tener en cuenta que no se agregan caracteres de fin de línea al final, con lo cual tendremos que hacerlo nosotros mismos si es que lo necesitamos. Escribamos un archivo en un breve ejemplo:

```
>>> # file: 04.xxxxxxxxxxx.py
>>> from os import strerror
>>>
>>> try:
>>>     directorio = '/var/www/html/python.desde.cero/'
>>>     archivo_relativo = 'unidad.10/recursos/ejemplo_escritura.txt'
>>>     archivo = directorio + archivo_relativo
>>>     stream = open(archivo, 'wt', encoding='utf-8')
>>>
>>>     for i in range(10):
>>>         string_linea = f'Esta es la línea {i+1}\n'
>>>         stream.write(string_linea)
>>>
>>>     stream.close()
```

```
>>> print("Archivo escrito correctamente")
>>>
>>> except IOError as e:
>>> print(f'Error: {strerror(e.errno)}')
```

Con esto finalizamos la unidad sobre tratamiento de archivos.

UNIDAD 11

(muy importante!)

ZEN | ENTORNOS VIRTUALES | MODULOS | PEP8 | DOCSTRINGS

zen de Python

El "Zen de Python" es un conjunto de principios que reflejan la filosofía de diseño y estilo de Python. Creado por Tim Peters, proporciona una guía sobre cómo escribir código Python legible y eficiente.

A continuación, te proporciono una breve explicación de cada punto del Zen de Python:

Bello es mejor que feo:

Python fomenta un código estéticamente agradable y fácil de leer. El código debería ser elegante y visualmente atractivo.

Explícito es mejor que implícito:

Es mejor ser claro y específico en lugar de depender de la magia o la ambigüedad. El código debe ser fácilmente comprensible para cualquier persona que lo lea.

Simple es mejor que complejo:

La simplicidad es clave en Python. Los enfoques sencillos suelen ser más fáciles de entender, depurar y mantener que los enfoques complicados.

Complejo es mejor que complicado:

Si una tarea es intrínsecamente compleja, está bien tener una solución compleja. Sin embargo, esto no debe confundirse con complicado, que implica confusión y dificultad para entender.

Plano es mejor que anidado:

Se prefiere tener estructuras planas en lugar de un anidamiento excesivo. Un código con menos niveles de anidamiento es más legible y fácil de seguir.

Espaciado es mejor que denso:

Utilizar espacios y líneas en blanco adecuadamente puede hacer que el código sea más legible. Un código bien espaciado facilita la lectura y comprensión.

La legibilidad es importante:

El código debe ser fácil de leer, tanto para el autor como para otros programadores. Un código legible es más fácil de entender, mantener y depurar.

Los casos especiales no son lo suficientemente especiales como para romper las reglas:

Aunque puede haber casos especiales, no deben violar las reglas fundamentales de diseño y estilo de Python. La coherencia en el código es esencial.

Sin embargo, la practicidad le gana a la pureza:

A veces, en la búsqueda de la perfección, se pueden hacer concesiones en favor de soluciones prácticas y utilizables en el mundo real.

Los errores nunca deberían pasar silenciosamente. A menos que se silencien explícitamente:

Es importante detectar y manejar los errores adecuadamente. Los errores no deben ignorarse y deben ser tratados para evitar problemas inesperados en el programa.

Frente a la ambigüedad, evitar la tentación de adivinar:

Cuando el significado de una parte del código es ambiguo, es mejor ser explícito y claro en lugar de hacer suposiciones inciertas.

Debería haber una, y preferiblemente solo una, manera obvia de hacerlo:

Python fomenta la simplicidad y la consistencia. Debería haber una forma clara y preferida de hacer las cosas para evitar confusiones.

A pesar de que eso no sea obvio al principio a menos que seas Holandés:

Una referencia humorística a Guido van Rossum, creador de Python. Implica que algunas decisiones de diseño pueden no ser obvias para todos, pero tienen sentido en el contexto de la comunidad y la historia de Python.

Ahora es mejor que nunca:

Es mejor tomar medidas ahora mismo en lugar de esperar una solución perfecta en el futuro. La acción inmediata es valiosa.

A pesar de que nunca es muchas veces mejor que ahora mismo:

Aunque la perfección puede ser inalcanzable, es importante tomar medidas y mejorar continuamente en lugar de esperar indefinidamente la situación ideal.

Si la implementación es difícil de explicar, es una mala idea. Si la implementación es fácil de explicar, puede que sea una buena idea:

La claridad y la comprensión son clave. Si una solución es difícil de explicar, probablemente sea complicada y problemática. Las soluciones simples y comprensibles son preferibles.

Los espacios de nombres son una gran idea, ¡tengamos más de esos! :

Los espacios de nombres (namespaces) son una forma de evitar conflictos de nombres al organizar el código. Python fomenta el uso de espacios de nombres para mantener el código limpio y organizado.

Dejo un link en donde detalla algunos puntos con código de ejemplo:

<https://elpythonista.com/zen-de-python>

ENTORNOS VIRTUALES

Un entorno virtual es un directorio aislado donde puedes instalar paquetes de Python específicos para un proyecto sin afectar al sistema global ni a otros proyectos.

Beneficios:

- Aislar dependencias: Cada proyecto puede tener sus propias versiones de librerías.
- Evitar conflictos: Evitas problemas por incompatibilidades entre proyectos o con el sistema.
- Portabilidad: Puedes compartir fácilmente las dependencias del proyecto con otros.
- Reproducibilidad: Garantizas que funcione igual en cualquier máquina.

Cómo crear un entorno virtual

Primero vamos a verificar que tenemos Python correctamente instalado

```
>>> python --version # Windows
>>> python3 --version # Linux/macOS
```

Crear el entorno virtual

Para Windows. Abrimos una terminal, con el comando cmd.

```
>>> python -m venv mi_entorno
```

Para Linux o macOS:

```
>>> python3 -m venv mi_entorno
```

Este comando crea una carpeta llamada `mi\_entorno` (puedes cambiar el nombre) que contiene:

- Una copia local de Python
- Carpeta `Scripts/` (Windows) o `bin/` (Linux/macOS) con herramientas como `pip`
- Carpeta `lib/` con los paquetes instalados

ACTIVAR el entorno virtual

Para Windows:

```
>>> mi_entorno\Scripts\activate.bat
```

Después de activarlo, verás algo así en la terminal:

```
>>> (mi_entorno) C:\Usuarios\tu_usuario>
```

Para Linux o macOS:

```
>>> source mi_entorno/bin/activate
```

Verás esto:

```
>>> (mi_entorno) usuario@maquina:~$
```

Instalar paquetes dentro del entorno

Una vez activado el entorno, todo lo que instales con `pip` se guardará sólo dentro de él:

```
>>> pip install requests numpy flask
```

Puedes guardar tus dependencias en un archivo `requirements.txt`:

```
>>> pip freeze > requirements.txt
```

Y luego instalarlas en otro entorno:

```
>>> pip install -r requirements.txt
```

Salir del entorno virtual. Desactivarlo

```
>>> deactivate
```

Eliminar un entorno virtual

Solo borra la carpeta del entorno:

```
>>> rm -rf mi_entorno # Linux/macOS
```

```
>>> rmdir /s /q mi_entorno # Windows
```

Estructura recomendada de un proyecto con entorno virtual

```
mi_proyecto/  
├── main.py      # Script principal  
├── requirements.txt # Lista de dependencias  
├── .gitignore   # Para ignorar 'mi_entorno' en Git  
└── mi_entorno/  # Carpeta del entorno virtual (no subida a repositorios)
```

Recomendación: Añade el nombre del entorno (`mi\_entorno/`) al `.gitignore`. No es recomendable guardar en los repositorios las librerías de terceros: por que ocupan lugar, y la realidad es que las debemos manejar desde el archivo de texto requirements.txt indicando la librería y la versión exacta que se debe instalar. Por lo tanto, siempre en los repositorios agregamos al .gitignore el directorio que contiene todo nuestro entorno virtual. Es una práctica habitual y recomendable hacerla.

RESUMEN

- 1- Crear entorno -> `python -m venv nombre_entorno`
- 2- Activar entorno (Linux/macOS) -> `source nombre_entorno/bin/activate`
- 3- Activar entorno (Windows) -> `nombre_entorno\Scripts\activate`
- 4- Instalar paquetes -> `pip install nombre_paquete`
- 5- Guardar dependencias -> `pip freeze > requirements.txt`
- 6- Instalar desde requirements.txt -> `pip install -r requirements.txt`

- 7- Salir del entorno -> deactivate
- 8- Borrar entorno (Linux/macOS) -> rm -rf nombre_entorno
- 9- Borrar entorno (Windows) -> rmdir /s /q nombre_entorno

PIP: Gestor de paquetes.

`pip` es el **sistema de gestión de paquetes predeterminado de Python**. Es una herramienta que permite descargar, instalar, actualizar y gestionar bibliotecas y paquetes de terceros desde el repositorio oficial de Python llamado PyPI (Python Package Index).

Vemos un ejemplo:

```
>>> pip install requests
```

Este comando descarga e instala el módulo `requests`, que sirve para hacer solicitudes HTTP.

¿Qué es un módulo en Python?

Un **módulo** es un archivo que contiene definiciones y sentencias de Python. Puede incluir funciones, clases o variables que tú o alguien más ya programó, y que tú puedes reutilizar en tus proyectos.

Los módulos pueden ser:

- Módulos nativos: Viene con Python por defecto (como `os`, `sys`, `json`, `datetime`, etc.).
- Módulos de terceros: Se instalan con `pip` (como `numpy`, `pandas`, `flask`, `requests`, etc.).

La filosofía de Python es: **Batteries included**, pero eso no significa que todo esté dentro del núcleo. Muchas funcionalidades avanzadas están disponibles a través de **paquetes externos** que puedes instalar cuando las necesites.

Ventajas de usar módulos:

1. Reutilización de código: No reinventes la rueda. Usa lo que otros ya han creado.
2. Ahorro de tiempo: Evitas escribir código desde cero.
3. Ampliación de capacidades: Con módulos puedes crear webs, analizar datos, automatizar tareas, hacer gráficos, etc.

4. Comunidad fuerte: Hay miles de módulos listos para usar en [\[https://pypi.org/\]](https://pypi.org/)(<https://pypi.org/>).

Principales comandos de `pip`

Instalar un paquete

```
>>> pip install nombre_del_paquete
```

Desinstalar un paquete

```
>>> pip uninstall nombre_del_paquete
```

Actualizar un paquete

```
>>> pip install --upgrade nombre_del_paquete
```

Ver lista de paquetes instalados

```
>>> pip list
```

Buscar un paquete

```
>>> pip search "nombre"
```

Ver información de un paquete

```
>>> pip show nombre_del_paquete
```

Guardar dependencias

```
>>> pip freeze > requirements.txt
```

Instalar desde requirements.txt

```
>>> pip install -r requirements.txt
```

Buenas prácticas al usar `pip`

- Usa siempre entornos virtuales: Para evitar conflictos entre proyectos.
- Evita usar `sudo` innecesariamente: A menos que quieras instalar algo globalmente (no recomendado).
- Usa `requirements.txt`: Para mantener el control de las dependencias.
- No ignores las advertencias de seguridad: Algunos paquetes pueden tener vulnerabilidades.

PEP8

PEP 8 (Python Enhancement Proposal 8) es la guía oficial de estilo para el código Python, creada por Guido van Rossum, Barry Warsaw y Nick Coghlan. Publicada originalmente en 2001 y actualizada periódicamente, establece las convenciones de codificación que definen cómo debe escribirse código Python legible y consistente.

Filosofía y propósito

PEP 8 se fundamenta en el principio de que el código se lee mucho más frecuentemente de lo que se escribe. Por tanto, prioriza la legibilidad sobre la escritura rápida. La guía reconoce que la consistencia es importante, pero también establece que la legibilidad cuenta más que la adherencia ciega a las reglas cuando estas entran en conflicto.

Principios fundamentales

La guía establece varios principios rectores para la escritura de código Python. El más importante es la legibilidad, seguido por la consistencia dentro de un proyecto, luego la consistencia dentro de un módulo, y finalmente la consistencia global. PEP 8 también enfatiza que las reglas pueden romperse cuando aplicarlas haría el código menos legible o cuando se mantiene compatibilidad con código existente.

Convenciones de nomenclatura

PEP 8 define convenciones específicas para nombrar diferentes elementos del código. Las variables y funciones deben usar ``snake_case`` (palabras separadas por guiones bajos), las clases deben usar ``PascalCase`` (primera letra de cada palabra en mayúscula), las constantes deben escribirse en ``MAYÚSCULAS_CON_GUIONES_BAJOS``, y los módulos deben tener nombres cortos en minúsculas, preferiblemente sin guiones bajos.

Formato y estructura

Las reglas de formato cubren aspectos como la indentación, que debe ser de 4 espacios por nivel (nunca tabs), las líneas no deben exceder 79 caracteres para código y 72 para comentarios, y debe haber líneas en blanco para separar funciones de nivel superior y definiciones de clases. Los operadores binarios deben ir precedidos de espacios, y las comas deben ir seguidas de un espacio.

Importaciones

PEP 8 establece que las importaciones deben colocarse al inicio del archivo, después de comentarios de módulo y docstrings, pero antes de constantes y globales del módulo. Las importaciones deben organizarse en grupos separados: primero las importaciones de la biblioteca estándar, luego las de terceros, y finalmente las importaciones locales de la aplicación. Cada grupo debe separarse por una línea en blanco.

Comentarios y documentación

Los comentarios deben ser oraciones completas y mantenerse actualizados con los cambios del código. Los comentarios en línea deben usarse con moderación y separarse del código por al menos dos espacios. Los docstrings deben escribirse para todos los módulos, funciones, clases y métodos públicos, siguiendo las convenciones de PEP 257.

Expresiones y declaraciones

PEP 8 recomienda evitar espacios extraños alrededor de paréntesis, corchetes y llaves, no usar espacios alrededor del signo igual cuando se usa para indicar argumentos de palabra clave, y ser consistente con el uso de comillas simples o dobles para strings, aunque prefiere las dobles comillas para docstrings.

Herramientas de verificación

Existen múltiples herramientas que automatizan la verificación de cumplimiento de PEP 8, incluyendo ``pycodestyle`` (anteriormente ``pep8``), ``flake8``, ``pylint``, y ``black`` para formateo automático. Estas herramientas pueden integrarse en editores y entornos de desarrollo para proporcionar retroalimentación en tiempo real.

Impacto en la comunidad

PEP 8 ha tenido un impacto profundo en la comunidad Python, estableciendo un estándar que facilita la colaboración entre desarrolladores. Cuando el código sigue estas convenciones, los programadores pueden leer y entender más fácilmente el código escrito por otros, reduciendo la curva de aprendizaje y mejorando la productividad en proyectos colaborativos.

La adopción de PEP 8 representa un compromiso con la calidad del código y la profesionalidad en el desarrollo Python, siendo considerada una práctica esencial en el ecosistema del lenguaje.



<https://peps.python.org/pep-0008/>

Docstrings

Un **docstring** (cadena de documentación) es una cadena literal que aparece como la primera declaración en la definición de un módulo, función, clase o método en Python. Los docstrings sirven como documentación integrada del código, proporcionando descripciones claras y accesibles de la funcionalidad, parámetros, valores de retorno y comportamiento esperado.

Características técnicas

Los docstrings se definen utilizando cadenas de texto delimitadas por triple comillas (`'''` o `"""`), aunque la convención estándar prefiere las comillas dobles. A diferencia de los comentarios regulares que son ignorados por el intérprete, los docstrings se almacenan como el atributo `__doc__` del objeto correspondiente, haciéndolos accesibles programáticamente durante la ejecución.

Propósito y beneficios

Los docstrings cumplen múltiples funciones esenciales en el desarrollo de software Python. Proporcionan documentación autodescriptiva que permanece cerca del código, reduciendo la posibilidad de que la documentación se desactualice. Facilitan la comprensión del código para otros desarrolladores y para el propio autor en el futuro. Además, muchas herramientas de desarrollo utilizan docstrings para generar documentación automática, proporcionar ayuda contextual en IDEs, y realizar verificaciones de tipo y comportamiento.

Estándares y convenciones

PEP 257 establece las convenciones oficiales para docstrings en Python. Los docstrings deben escribirse para todos los módulos, funciones, clases y métodos públicos. Deben comenzar con una línea que resuma brevemente el propósito, seguida opcionalmente por una línea en blanco y una descripción más detallada. Para funciones y métodos complejos, deben incluir información sobre parámetros, valores de retorno, excepciones que pueden lanzarse, y efectos secundarios.

Tipos de docstrings

Existen diferentes tipos de docstrings según el contexto. Los **docstrings de módulo** describen el propósito general del módulo y su funcionalidad principal. Los **docstrings de clase** explican el propósito de la clase, sus atributos principales y su uso típico. Los **docstrings de función y método** detallan qué hace la función, qué parámetros acepta, qué retorna, y qué excepciones puede lanzar.

Formatos de documentación

La comunidad Python ha desarrollado varios formatos estandarizados para estructurar docstrings. El formato `**reStructuredText**` es ampliamente utilizado y compatible con herramientas como Sphinx. El formato `**Google Style**` ofrece una sintaxis clara y legible. El formato `**NumPy/SciPy**` es popular en el ámbito científico y proporciona estructura detallada para documentación técnica.

Acceso programático

Los docstrings pueden accederse durante la ejecución mediante el atributo ``__doc__`` o utilizando la función integrada ``help()``. Esta característica permite crear sistemas de ayuda dinámicos, herramientas de introspección, y generadores automáticos de documentación que extraen información directamente del código fuente.

Importancia en el ecosistema Python

Los docstrings representan una práctica fundamental en el desarrollo Python profesional. Facilitan el mantenimiento del código a largo plazo, mejoran la experiencia de desarrollo al proporcionar ayuda contextual, y permiten la generación automática de documentación técnica. Su uso consistente y bien estructurado distingue el código profesional del amateur, estableciendo una base sólida para proyectos escalables y colaborativos.

La inversión en docstrings de calidad se traduce en reducción de tiempo de debugging, facilita la incorporación de nuevos desarrolladores al proyecto, y mejora la calidad general del software al forzar al desarrollador a pensar claramente sobre el propósito y comportamiento de cada componente del código.

En ejemplo:

```
>>> def dividir(dividendo, divisor):
>>>     """
>>>     Realiza la división entre dos números.
>>>     Esta función divide el dividendo entre el divisor y retorna
>>>     el resultado redondeado con una precisión de dos decimales.
>>>
>>>     Args:
>>>         dividendo (float): El número que será dividido.
>>>         divisor (float): El número por el cual se divide.
>>>     Returns:
>>>         float: El resultado de la división redondeado.
>>>     Raises:
>>>         TypeError: Si los argumentos no son números.
>>>         ZeroDivisionError: Si el divisor es cero.
>>>     Ejemplo:
>>>         >>> dividir(10, 3)
>>>         3.33
>>>     """
>>>     if not isinstance(dividendo, (int, float)):
>>>         raise TypeError("El dividendo debe ser un número")
>>>
>>>     if not isinstance(divisor, (int, float)):
>>>         raise TypeError("El divisor debe ser un número")
>>>
>>>     if divisor == 0:
>>>         raise ZeroDivisionError("No se puede dividir entre cero")
>>>
>>>     resultado = dividendo / divisor
>>>
>>>     return round(resultado, 2)
```

README.MD | Que és ?

El nombre "README" viene de "Read Me" (léeme), indicando que debe ser lo primero que lea alguien interesado en tu proyecto.

Un buen `README.md` no solo ayuda a los usuarios a entender tu proyecto, sino también a:

- Explicar el propósito del proyecto: ¿qué problema resuelve?
- Guiar al usuario paso a paso: Para instalar y ejecutar el proyecto.
- Mostrar cómo funciona: Podés poner alguna imagen de alguna pantalla, tipo demo.
- Atraer colaboradores** al proyecto si es open source.
- Dar crédito a autores o herramientas usadas: Nombras los autores del proyecto, nombrás todas las herramientas que se utilizaron.
- Definir cómo contribuir: Si es open source, podés explicar el proceso ideal que se espera para que otros/as contribuyan y mejoren el proyecto.
- Mencionar las dependencias necesarias: Detallar las dependencias de terceros, y también documentar como poder instalarlas.

¿Por qué es importante incluir un `README.md` en proyectos Python? Y también en proyectos realizados en cualquier otro lenguaje..

Es el primer contacto con el proyecto

Es lo primero que ve cualquier persona que entra al repositorio. Ayuda a decidir si vale la pena explorarlo más.

Es una Guía técnica rápida

Permite a otros desarrolladores o compañeros instalar y ejecutar el proyecto sin tener que buscar instrucciones por otro lado.

Profesionalismo y claridad

Un buen `README.md` demuestra compromiso, organización y profesionalismo. Es especialmente útil en trabajos prácticos, parciales o proyectos grupales.

Facilita la colaboración

Si el proyecto es público, un `README.md` claro ayuda a otros a entender el código y contribuir más fácilmente.

Uso de buenas prácticas

Ayuda a documentar el uso de entornos virtuales, archivos `.gitignore`, `requirements.txt`, etc., que son buenas prácticas en Python.



Documentación sintaxis markdown

<https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Entonces, como resumen, un `README.md` bien escrito es **una herramienta fundamental** para cualquier proyecto en GitHub, especialmente en Python, donde la instalación de dependencias y la correcta ejecución pueden depender de pasos precisos. No solo facilita el uso del proyecto, sino que también mejora la comunicación y la transparencia del desarrollo.

SECCION DE MODULOS

UNIDAD 12 (general)

DATETIME | TIME

datetime

El módulo ``datetime`` proporciona clases para manipular fechas y horas de una manera simple y compleja. Las clases más utilizadas en este módulo son ``datetime``, ``date``, ``time`` y ``timedelta``.

Clase ``datetime``

La clase ``datetime`` combina una fecha y una hora. Algunos de sus métodos más importantes son:

- ``datetime.now()``: Retorna la fecha y hora actual.
- ``datetime.strptime()``: Convierte una cadena en un objeto ``datetime`` utilizando un formato específico.
- ``datetime.strftime()``: Convierte un objeto ``datetime`` en una cadena según un formato especificado.

Ejemplo:



```
>>> from datetime import datetime
>>>
>>> # fecha y hora actual
>>> ahora = datetime.now()
>>> print("Fecha y hora actual:", ahora)
>>>
>>>
>>> # Convertir una cadena a un objeto datetime
>>> fecha_string = '2021-12-31 23:59:59'
>>> fecha_dt = datetime.strptime(fecha_string, '%Y-%m-%d %H:%M:%S')
>>> print("Fecha y hora convertida:", fecha_dt)
>>>
>>> # Convertir un objeto datetime a una cadena
>>> fecha_dt = datetime.now()
>>> fecha_string = fecha_dt.strftime('%Y-%m-%d %H:%M:%S')
>>> print("Fecha y hora convertida:", fecha_string)
>>>
>>> # Salida:
>>> # Fecha y hora actual: 2025-02-06 11:19:47.599656
>>> # Fecha y hora convertida: 2021-12-31 23:59:59
>>> # Fecha y hora convertida: 2025-02-06 11:19:47
```

Clase `date`

La clase `date` representa una fecha (año, mes, día) sin la parte de la hora.

- `date.today()`: Retorna la fecha actual.
- `date.fromisoformat()`: Convierte una cadena en formato ISO en un objeto `date`.

Ejemplo



```
>>> from datetime import date
>>>
>>> # Obtener la fecha actual
>>> hoy = date.today()
>>> print(f'Fecha actual: {hoy}')
>>>
>>> # Convertir una cadena a un objeto date
>>> fecha_string = '2021-12-31'
>>> fecha_dt = date.fromisoformat(fecha_string)
>>>
>>> print(f'Fecha convertida: {fecha_dt}')
>>>
>>> # Salida:
>>> # Fecha actual: 2025-02-06
>>> # Fecha convertida: 2021-12-31
```

Clase `time`

La clase `time` representa una hora (hora, minuto, segundo, microsegundo) sin la parte de la fecha.

- `time(hour, minute, second)`: Crea un objeto `time`.



```
>>> from datetime import time
>>>
>>> # Crear un objeto time
>>> hora = time(14, 30, 0)
```

```
>>> print(f'Hora: {hora}')
```

```
>>>
>>> # Salida:
>>> # Hora: 14:30:00
```

Clase `timedelta`

La clase `timedelta` representa una duración, la diferencia entre dos fechas u horas.

- `timedelta(days, seconds, microseconds)`: Crea un objeto `timedelta`.
- Operaciones aritméticas con `datetime` y `timedelta`.



```
>>> from datetime import timedelta, datetime
>>>
>>> # Crear un objeto timedelta
>>> duracion = timedelta(days=5, hours=3, minutes=30)
>>> print(f'Duración: {duracion}')
```

```
>>>
>>> # Sumar un objeto timedelta a un objeto datetime
>>> ahora = datetime.now()
>>> nueva_fecha = ahora + duracion
>>> print(f'Nueva fecha: {nueva_fecha}')
```

```
>>>
>>> # Salida:
>>> # Duración: 5 days, 3:30:00
>>> # Nueva fecha: 2025-02-11 15:02:16.191878
```

Módulo `time`

El módulo `time` proporciona varias funciones para trabajar con el tiempo y la medición del mismo en el sistema operativo.

`time.time()`

Retorna el tiempo en segundos desde el "epoch".

El epoch es el momento de referencia a partir del cual se cuentan los segundos en los sistemas Unix y similares.

Fecha de inicio: 1 de enero de 1970 a las 00:00:00 UTC

`time.sleep()`

Suspende (pausa) la ejecución del programa durante el número de segundos especificado.

`time.strftime()`

Convierte una tupla o estructura de tiempo en una cadena según un formato especificado.

`time.localtime()`

Convierte un tiempo en segundos desde el epoch a una estructura de tiempo en la hora local.



```
>>> import time
>>>
>>> # Obtener tiempo actual en segundos desde el epoch
>>> tiempo_actual = time.time()
>>> print(f'Tiempo actual en segundos: {tiempo_actual}')
>>>
>>>
>>> # Probando sleep
>>> print("Esperando 3 segundos ... ")
>>> time.sleep(3)
```

```

>>> print(";Listo!")
>>>
>>> # Obtener la estructura de tiempo actual
>>> hora_actual = time.localtime()
>>>
>>> # Formatear la estructura de tiempo a una cadena
>>>
>>> hora_formateada = time.strftime('%Y-%m-%d %H:%M:%S', hora_actual)
>>>
>>> print(f'Hora formateada: {hora_formateada}')

```

time.perf_counter()

Mide el tiempo de ejecución de código con alta precisión, incluyendo el tiempo que el programa pasa durmiendo. Sirve por ejemplo para hacer marcas de tiempo dentro de tu script para medir tiempos de procesos.

```

>>> # 06-time-perf_counter.py
>>> import time
>>>
>>> # Iniciar contador
>>> inicio = time.perf_counter()
>>>
>>> # Simulo un proceso que dura 5 segundos
>>> print("Iniciando proceso...")
>>> time.sleep(5)
>>> print("Proceso finalizado.")
>>>
>>> fin = time.perf_counter()
>>>
>>> # Detener contador
>>>
>>> # Calcular duración

```

```
>>> duracion = fin - inicio
>>>
>>> print(f"El proceso tomó {duracion} segundos.")
```

En resumen, los módulos ``datetime`` y ``time`` de Python son herramientas poderosas y versátiles para trabajar con fechas y horas. ``datetime`` es más adecuado para manipular y realizar cálculos complejos con fechas y horas, mientras que ``time`` es útil para funciones más básicas relacionadas con el tiempo del sistema. Con estos módulos, puedes realizar una amplia variedad de operaciones de manera eficiente y precisa.

UNIDAD 13 - (Infraestructura)

PLATFORM | OS | SYS | SUBPROCESS

Modulo platform, os, sys, subprocess

¿Cómo saber dónde estás?

A veces puede ser necesario encontrar información no relacionada con Python. Por ejemplo, es posible que necesites conocer la ubicación de tu programa dentro del entorno de la computadora.

Imagina el entorno de tu programa como una pirámide que consta de varias capas o plataformas.



Las capas son:

- **El código**: Es la parte superior de la pirámide. Código en ejecución.

- **Python**, su entorno de ejecución. Directamente debajo del código. Utiliza servicios del sistema operativo para algunas funcionalidades.
- **Sistema Operativo**: Proporciona servicios que Python utiliza para tareas como el procesamiento de archivos y la comunicación con dispositivos físicos.
- **Hardware**: Incluye procesadores, interfaces de red, dispositivos de interfaz humana (ratones, teclados, etc.). El sistema operativo interactúa con el hardware para mantener un funcionamiento constante.

Por ejemplo en una Ejecución de Tareas, en el proceso de Crear un Archivo:

- El código invoca una función de Python para crear un archivo.
- Python traduce la solicitud para que el SO la entienda.
- El SO valida y ejecuta la solicitud, involucrando múltiples pasos y el hardware correspondiente.

Hay veces que queremos o necesitamos saber más sobre el sistema operativo, por ejemplo, el nombre y algunas características que describen el hardware que aloja el sistema operativo.

Hay un módulo que proporciona algunos medios para permitir saber dónde se encuentra y qué componentes funcionan. El módulo se llama platform.

Modulo platform

Proporciona acceso a datos del hardware, sistema operativo e información de la versión del intérprete de Python.

Ofrece una interfaz fácil de usar para obtener información detallada sobre el sistema y el hardware.

Veamos un ejemplo:



```
>>> import platform
>>>
>>> sistema_operativo = platform.system()
>>> print("Sistema operativo:", sistema_operativo)
>>>
>>> version_sistema = platform.version()
>>> print("Versión del sistema:", version_sistema)
>>>
>>> arquitectura = platform.architecture()
>>> print("Arquitectura:", arquitectura)
>>>
>>> interprete = platform.python_implementation()
>>> print("Interprete:", interprete)
>>>
>>> version_python = platform.python_version()
>>> print("Versión de Python:", version_python)
>>>
>>> version_en_tupla = platform.python_version_tuple()
>>> print("Versión de Python en tupla:", version_en_tupla)
```

La función `platform.system()` devuelve el nombre del sistema operativo subyacente.

El método `version()` devuelve una cadena que representa la versión del sistema operativo.

Luego, `platform.architecture()` devuelve una tupla que indica la arquitectura del hardware (por ejemplo, 32 bits o 64 bits). Y el otro valor es el tipo de enlace usado por el ejecutable del intérprete de Python. Enlace en Programación es el proceso de combinar varias partes de un programa (módulos o archivos) en un ejecutable completo.

Ejemplo en Linux:

El formato de conexión devuelto es ELF (Executable and Linkable Format).

¿Qué es ELF?

ELF (Executable and Linkable Format):

Es un estándar de formato de archivo utilizado en sistemas operativos Unix y Unix-like, como Linux y macOS.

Diseñado para contener programas ejecutables y la información necesaria para el enlazador dinámico.

Es el formato que el sistema operativo y el hardware pueden ejecutar directamente.

Detalles del Intérprete de Python

`platform.python_implementation():`

Devuelve una cadena que representa la implementación de Python en uso, por ejemplo:

CPython: La implementación estándar escrita en C.

Jython: Python para la máquina virtual Java.

IronPython: Python para el entorno .NET.

El resultado se almacena en la variable `interprete` y se imprime.

¿Qué es CPython?:

Es la implementación de referencia del lenguaje Python.

Creada por Guido van Rossum y su equipo.

Adhiere a los principios del Zen de Python, proporcionando una implementación clara y legible.

La "C" en CPython se refiere a que gran parte del intérprete está escrita en C.

`platform.python_version():`

Devuelve una cadena con la versión actual de Python.

`platform.python_version_tuple():`

Devuelve la versión de Python en forma de tupla.

Ambas versiones se imprimen en la consola.

Modulo OS

Permite interactuar con el sistema operativo usando Python. Proporciona funciones disponibles en sistemas Unix y/o Windows.

Funciones Principales:

- Crear Directorios: Función mkdir, similar al comando mkdir en Unix y Windows.
 - Operaciones en Archivos y Directorios: Realiza diversas operaciones en el sistema de archivos.
 - Información del Sistema Operativo: Obtiene detalles sobre el SO actual.
 - Manejo de Procesos: Permite la gestión de procesos (tema avanzado).
 - Streams de Entrada/Salida: Trabaja con descriptores de archivos (tema avanzado).
-
- Obtener Información del Sistema Operativo / Función uname(): Devuelve un objeto con atributos sobre el sistema operativo:

systemname: Nombre del sistema operativo.

nodename: Nombre de la máquina en la red.

release: Actualización del sistema operativo.

version: Versión del sistema operativo.

machine: Identificador de hardware, como x86_64.

Llevado a la práctica se vería así

import os

Vamos a analizar el siguiente script:



```
import os
>>>
>>> uname = os.uname()
>>>
>>> sysname = uname.sysname
>>> nodename = uname.nodename
>>> release = uname.release
>>> version = uname.version
>>> machine = uname.machine
```

```
>>>
>>> print("Sistema operativo:", sysname)
>>> print("Nombre del nodo:", nodename)
>>> print("Versión:", release)
>>> print("Número de versión:", version)
>>> print("Arquitectura:", machine)
>>>
>>> # Salida en Consola, en mi máquina con Linux:
>>> # Sistema operativo: Linux
>>> # Nombre del nodo: IdeaPad-1
>>> # Versión: 6.8.0-52-generic
>>> # Número de versión: #53-Ubuntu SMP PREEMPT_DYNAMIC Sat Jan 11 00:06:25
    UTC 2025
>>> # Arquitectura: x86_64
```

Desafortunadamente, la función `uname` solo funciona en algunos sistemas Unix. Si usamos Windows, podemos usar la función `uname` en el módulo `platform`, que devuelve un resultado similar.

El módulo `os` permite distinguir rápidamente el sistema operativo mediante el atributo `name`, que soporta uno de los siguientes nombres:

- `posix`: obtendremos este nombre si usamos Unix.
- `nt`: obtendremos este nombre si usamos Windows.
- `java`: obtendremos este nombre si nuestro código está escrito en Jython.



```
>>> import os
>>>
>>> print("Sistema operativo:", os.name)
```

Creando directorios en Python

El módulo `os` proporciona una función llamada `mkdir`, la cual, como el comando `mkdir` en Unix y Windows, nos permite crear un directorio. La función `mkdir` requiere una ruta que puede ser relativa o absoluta.

La función `mkdir` puede tomar opcionalmente el argumento `mode`, que especifica los permisos del directorio, sin embargo, en algunos sistemas, el argumento `mode` se ignora.

También contamos con una función llamada `listdir`. La función `listdir` devuelve una lista que contiene los nombres de los archivos y directorios que se encuentran en la ruta pasada como argumento, si no se le pasa ningún argumento, se utilizará el directorio de trabajo actual.

Veamos ambas funciones en un ejemplo:



```
>>> import os
>>>
>>> print(os.listdir("/var/www/html/proyecto_calculadora"))
>>>
>>> os.mkdir("/var/www/html/proyecto_calculadora/imagenes")
>>>
>>> print(os.listdir("/var/www/html/proyecto_calculadora"))
```

Nota: Tengamos en cuenta que ejecutar el programa dos veces generará un `FileExistsError`, ya que no es posible crear un directorio que ya existe

¿Qué sucede si necesitamos crear otro directorio dentro del directorio que acabamos de crear?

Por supuesto, podemos ir al directorio creado y crear otro directorio dentro de él, pero afortunadamente el módulo `os` proporciona una función llamada `makedirs`, que facilita esta tarea. La función `makedirs` permite la creación recursiva de directorios, lo que significa que se crearán todos los directorios de la ruta.

Para movernos entre directorios podemos usar una función llamada `chdir`, que cambia el directorio de trabajo actual a la ruta especificada. Como argumento toma cualquier ruta relativa o absoluta.

Para nuestro siguiente ejemplo lo usaremos para mostrar la creación de los nuevos directorios. Por otra parte, si en lugar de necesitar movernos entre directorios necesitamos saber nuestro directorio actual de trabajo, podemos usar la función `getcwd`, la cual no llega argumentos, y nos devolverá como resultado el directorio de trabajo actual.

Por último, ¿qué sucede cuando queremos borrar un directorio? Python nos provee de otras 2 funciones para esto: `rmdir` y `removedirs`. La primera función `rmdir`, tomará la ruta como argumento y eliminará el directorio indicado, la segunda función `removedirs`, funciona de manera similar, tomará la ruta como argumento y eliminará el directorio y todos sus subdirectorios que estén vacíos.

Nota: para las dos funciones, en caso de que alguno de los directorios no exista o no esté vacío, generará una excepción.

Veamos estas nuevas funciones en acción:



```
>>> import os
>>>
>>> print("Path actual:", os.getcwd())
>>> print("Antes de crear: ",os.listdir("/var/www/html/python.desde.cero/unidad.12/"))
>>>
>>> # Creamos los nuevos directorios
>>> os.mkdir("/var/www/html/python.desde.cero/unidad.12/imagenes")
```



```
>>> os.mkdir("/var/www/html/python.desde.cero/unidad.12/imagenes/arquitectura")
>>> os.mkdir("/var/www/html/python.desde.cero/unidad.12/imagenes/programacion")
>>> os.mkdir("/var/www/html/python.desde.cero/unidad.12/recursos")
>>>
>>> print("Listamos los directorios creados: ")
>>> print(os.listdir("/var/www/html/python.desde.cero/unidad.12/"))
>>>
>>> # Cambiamos de directorio
>>> os.chdir("/var/www/html/python.desde.cero/unidad.12/imagenes")
>>> print("Subdirectorios de imagenes: ", os.listdir())
>>>
>>> # Borramos los directorios
>>> os.chdir("/var/www/html/python.desde.cero/unidad.12/")
>>> os.removedirs("imagenes/arquitectura")
>>> os.removedirs("imagenes/programacion")
>>> os.rmdir("recursos")
>>>
>>> print("Parch actual:", os.getcwd())
>>> print("Listo directorio: ", os.listdir())
```

Modulo sys

El módulo sys en Python es extremadamente útil y proporciona acceso a algunas funcionalidades del intérprete.

Aquí tienes una descripción de los métodos más importantes del módulo sys que podrías incluir en tu libro:

sys.argv

Esta lista en Python contiene los argumentos de la línea de comandos. sys.argv[0] es el nombre del script en sí y los elementos subsiguientes contienen los argumentos pasados al script.

sys.path

Esta lista de cadenas contiene los directorios donde Python busca módulos cuando se utiliza la importación. Modificar `sys.path` puede ser útil si deseas agregar rutas personalizadas para tus módulos.

sys.modules

Es un diccionario que mapea nombres de módulos a los módulos ya importados. Puedes usar `sys.modules` para verificar si un módulo específico ha sido importado.



```
>>> import sys
>>>
>>>
>>> if 'math' in sys.modules:
>>>     print('math módulo está importado')
>>> else:
>>>     print('math modulo no está importado')
>>>
```

sys.platform

Devuelve una cadena que identifica la plataforma en la que se está ejecutando Python. Esto puede ser útil para escribir código que sea específico de la plataforma.

sys.version

Devuelve una cadena que indica la versión de Python en uso. Esto puede ser útil para asegurarse de que el código se está ejecutando en la versión de Python esperada.

sys.exit()

Permite salir del programa. Puedes proporcionar un código de estado como argumento opcional para indicar si el programa terminó correctamente (por ejemplo, `sys.exit(0)`) o si ocurrió un error (`sys.exit(1)`).

Vamos a ver un poco más en detalle un script para poder traer los argumentos cuando es llamado:



```
>>> import sys
>>>
>>> print("Número de parámetros: ", len(sys.argv))
>>> print("Lista de argumentos: ", sys.argv)
```

Entonces con el script anterior podemos ir agarrando los argumentos y utilizarlos de la siguiente manera:



```
>>> import sys
>>>
>>> archivo = sys.argv[0]
>>> arg_1 = sys.argv[1]
>>> arg_2 = sys.argv[2]
>>>
>>>
>>> print("Nombre del archivo: ", archivo)
>>> print("Argumento 1: ", arg_1)
```

```
>>> print("Argumento 2: ", arg_2)
```

Modulo subprocess

El módulo subprocess en Python proporciona una forma de trabajar con procesos del sistema operativo, interactuar con ellos, y redirigir la entrada/salida estándar.

Es una herramienta poderosa que te permite ejecutar comandos del sistema operativo como si los estuvieras ejecutando desde la línea de comandos de tu sistema.

Vamos a ver el método más importante dentro del módulo subprocess:

subprocess.run()

subprocess.run(args, *, stdin=None, input=None, stdout=None, stderr=None, capture_output=False, text=False, shell=False, cwd=None, timeout=None, check=False, encoding=None, errors=None):

Ejecuta el comando especificado en args como un subprocesso.

Devuelve un objeto CompletedProcess que contiene información sobre el proceso ejecutado.

cwd: Puedes redirigir la entrada/salida y especificar un directorio de trabajo.

capture_output: cuando se establece en Verdadero, capturará la salida estándar y el error estándar.

text: cuando se establece en Verdadero, devolverá stdout y stderr como cadena; de lo contrario, como bytes.

check: un valor booleano que indica si se debe verificar el código de retorno del subprocesso, si la verificación es verdadera y el código de retorno no es cero, entonces se genera el subprocesso "CalledProcessError".

timeout: un valor en segundos que especifica cuánto tiempo esperar para que se complete el subprocesso antes de que se agote el tiempo de espera.

shell: valor booleano que indica si se debe ejecutar el comando en un shell. Esto significa que el comando se pasa como una cadena y se pueden utilizar funciones específicas del shell, como la expansión de comodines y la sustitución de variables.

Vamos a ver un ejemplo utilizando el shell directamente.



```
>>> import subprocess
>>>
>>> # Comando que queremos ejecutar en la shell
>>> comando = "ls -l"
>>>
>>> # Utilizando subprocess.run() con shell=True
>>> resultado = subprocess.run(comando, shell=True, stdout=subprocess.PIPE,
stderr=subprocess.PIPE)
>>>
>>> # Imprimiendo el resultado
>>> print("Código de salida:", resultado.returncode)
>>> print("Salida estándar:")
>>> print(resultado.stdout.decode())
>>> print("Error estándar:")
>>> print(resultado.stderr.decode())
```

En este ejemplo, el comando "ls -l" se ejecutará en la shell del sistema operativo. La salida estándar del comando se captura y se imprime en la consola de Python. Además, se imprime el código de salida del comando, que indica si el comando se ejecutó correctamente (0) o si ocurrió algún error (cualquier otro valor).

Es importante tener cuidado al usar `shell=True` para evitar vulnerabilidades de seguridad, como la inyección de comandos. Si el comando que se ejecuta incluye entradas del usuario, es preferible evitar `shell=True` y usar argumentos y listas en su lugar para pasar los comandos y los argumentos al proceso.

Cuando se utiliza `subprocess.run()` para ejecutar un comando, la salida estándar y el error estándar del proceso ejecutado se capturan como objetos de tipo `bytes`. Para que esta salida sea legible como texto en lugar de datos binarios, se necesita convertir los objetos de tipo `bytes` a cadenas de caracteres (`strings`). Aquí es donde entra en juego la función **`decode()`**.

La función `decode()` se utiliza para convertir los datos binarios (`bytes`) en cadenas de caracteres (`str`) usando un cierto esquema de codificación, como UTF-8. Por defecto, `decode()` usa UTF-8 para realizar la conversión.

UNIDAD 14 - (Ciencia de Datos)

NUMPY | PANDAS

Introducción a NumPy

Imagina que estás organizando una tabla gigante con datos: puede ser una lista de temperaturas diarias, resultados de encuestas o la información de tus películas favoritas. Hacer cálculos rápidos y precisos con muchos datos usando las herramientas básicas de Python puede volverse complicado. Aquí es donde entra NumPy.

NumPy es una biblioteca de Python que nos permite trabajar de forma eficiente con grandes cantidades de datos numéricos. Su nombre viene de "Numerical Python" y es una de las herramientas más importantes para Ciencia de Datos.

¿Por qué usar NumPy?

- **Velocidad:** NumPy realiza operaciones matemáticas mucho más rápido que las listas tradicionales de Python.
- **Facilidad:** Permite escribir menos código para hacer tareas complejas.
- **Herramientas poderosas:** Ofrece funciones para matemáticas avanzadas, estadística, álgebra lineal y más.

Instalación de NumPy

Antes de empezar, necesitas instalar NumPy en tu computadora. Abre tu terminal o consola y escribe:

```
$ pip install numpy
```

Si ya está instalado, puedes comenzar a usarlo directamente.

Empezando con NumPy

Con el siguiente código vamos a estar generando un array de una sola dimensión

```
>>> import numpy as np
>>>
```

```
>>> # Crear un array de una sola dimensión
>>> mi_array = np.array([1, 2, 3, 4, 5])
>>> print(mi_array)
```

Arrays de más de una dimensión

```
>>> import numpy as np
>>>
>>> # Array de dos dimensiones (como una tabla)
>>> mi_matriz = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(mi_matriz)
>>>
>>>
>>> # Salida en consola
>>> # [[1 2 3]
>>> #  [4 5 6]]
```

Operaciones con Arrays

Una de las mejores cosas de NumPy es que puedes hacer operaciones matemáticas de forma directa:

```
>>> import numpy as np
>>>
>>> # Suma de arrays
>>> array1 = np.array([1, 2, 3])
>>> array2 = np.array([4, 5, 6])
>>>
>>> resultado = array1 + array2
>>> print(resultado)
>>>
>>> # Salida:
>>> # [5 7 9]
```


También puedes hacer operaciones como resta (`-`), multiplicación (`*`), división (`/`), etc.

Funciones útiles de NumPy

NumPy ofrece muchas funciones para crear arrays de manera rápida:

- `np.zeros()`: crea un array lleno de ceros.
- `np.ones()`: crea un array lleno de unos.
- `np.arange()`: crea un array con un rango de números.
- `np.linspace()`: crea un array con números igualmente espaciados.

Ejemplo:

```
>>> import numpy as np
>>>
>>>
>>> # Crear un array de ceros
>>> ceros = np.zeros(5)
>>> print(ceros)

>>> # Salida:
>>> # [0. 0. 0. 0. 0.]
>>>
>>> # Crear un array de números del 0 al 9
rango = np.arange(10)
>>> print(rango)
>>>
>>> # Salida:
>>> # [0 1 2 3 4 5 6 7 8 9]
```

Trabajando con Estadísticas

NumPy también facilita el cálculo de estadísticas básicas:

```
>>> import numpy as np
>>>
```

```
>>>
>>> datos = np.array([10, 20, 30, 40, 50])
>>>
>>> print("Media:", np.mean(datos))
>>> print("Mediana:", np.median(datos))
>>> print("Desviación estándar:", np.std(datos))
>>>
>>>
>>> # Salida:
>>> # Media: 30.0
>>> # Mediana: 30.0
>>> # Desviación estándar: 14.142135623730951
```

UNIDAD 15 - (Ciencia de Datos)

MATPLOTLIB | SEABORN

Introducción: ¿Por qué visualizar datos?

Imaginate que tienes una hoja llena de números. ¿Podés ver los patrones, comportamientos a simple vista? Probablemente no.

Los seres humanos procesamos mejor la información a través de imágenes. Con Python y estas dos bibliotecas, vas a poder transformar datos en historias visuales claras y atractivas.

Matplotlib

La herramienta fundamental para crear todo tipo de gráficos.

Instalación

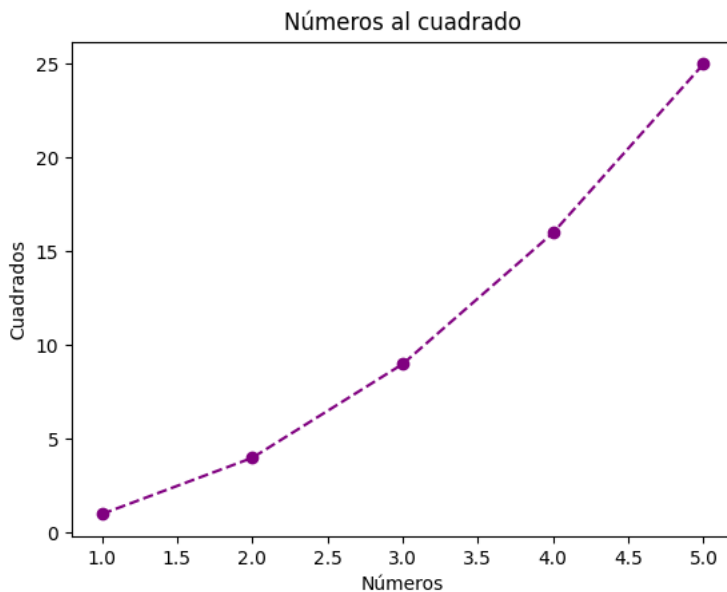
```
>>> $ pip install matplotlib
```

Recordá siempre utilizar entornos virtuales como estuvimos viendo en la Unidad 11.

Vamos a crear una simple línea que muestre cómo aumentan los números al cuadrado.

```
>>> import matplotlib.pyplot as plt
>>>
>>> # Datos
>>> x = [1, 2, 3, 4, 5]
>>> y = [1, 4, 9, 16, 25]
>>>
>>> # Crear gráfico
>>> plt.plot(x, y, color='purple', linestyle='--', marker='o')
>>> plt.title("Números al cuadrado")
>>> plt.xlabel("Números")
>>> plt.ylabel("Cuadrados")
```

```
>>>
>>> plt.show() # ¡Muestra tu obra de arte!
>>>
>>>
>>> # Guardar el gráfico en un archivo
>>> plt.savefig("grafico.png")
>>>
```



El método `plot()` es el encargado de crear el gráfico. A través de sus parámetros, configuramos su apariencia. Primero, pasamos las variables correspondientes a los ejes `**X**` e `**Y**`. Luego, utilizamos lo que se conoce como `**argumentos de palabras clave**` para personalizar el gráfico.

En este caso, usamos tres argumentos clave:

- `**color**`: Define el color de la línea.
- `**linestyle**`: Establece el estilo de la línea (continua, discontinua, etc.).
- `**marker**`: Determina la forma del marcador en cada punto. En este caso, `marker='o'` indica que queremos círculos.

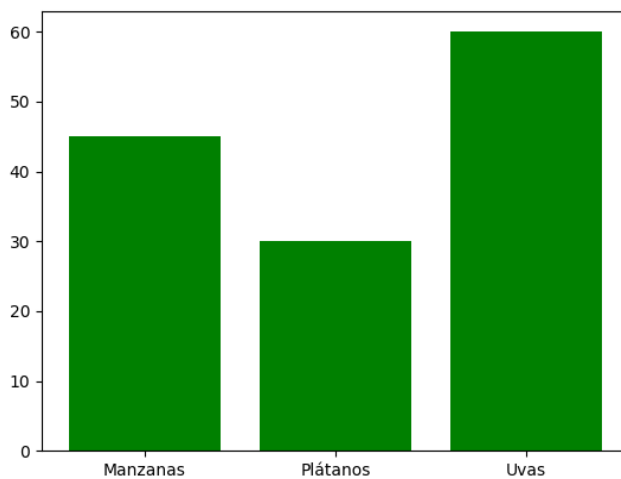
Además, utilizamos otros métodos de `pyplot` para mejorar el gráfico:

- `title()` → Establece el título del gráfico.
- `xlabel()` → Define la etiqueta del eje `**X**` (eje de abscisas).
- `ylabel()` → Define la etiqueta del eje `**Y**` (eje de ordenadas).

- `show()` → Muestra el gráfico en pantalla, si el sistema lo permite.
- `savefig()` → Guarda el gráfico como un archivo `*.png`, útil en caso de que no se pueda mostrar en pantalla.

Gráfico de barras: (Ideal para categorías)

```
>>> import matplotlib.pyplot as plt
>>>
>>> frutas = ['Manzanas', 'Plátanos', 'Uvas']
>>> ventas = [45, 30, 60]
>>> plt.bar(frutas, ventas, color='green')
>>> plt.show()
>>>
>>>
>>> plt.savefig("grafico_barras.png")
```



****2. Seaborn: Belleza estadística****

Seaborn es una biblioteca de visualización de datos de alto nivel construida sobre Matplotlib y estrechamente integrada con Pandas. Está diseñada para crear gráficos estadísticos atractivos e informativos con sintaxis minimalista, facilitando el análisis exploratorio de datos (EDA) y la comunicación de resultados.

Se usa para poder generar gráficos estadísticos avanzados.

Tiene muchas funcionalidades, como ser algunas, realizar histogramas, gráficos de dispersión, boxplots, violín plots, heatmaps, pair plots y más.

Tiene soporte nativo para variables categóricas, integración con Pandas, trabaja directamente con DataFrames, automapeo de columnas a ejes y categorías, estética automática

Tiene temas predefinidos (darkgrid, whitegrid, dark, white, ticks).

Paletas de colores profesionales (divergentes, categóricas, secuenciales).

Visualización de matrices de correlación (heatmap + clustering).

Gráficos de distribución conjunta (jointplot).

Regresiones estadísticas integradas (lmplot).

Vamos a ver unos simples ejemplos, por que el objetivo de esta unidad es presentar el módulo, para poder estudiarlo mas a fondo necesitaríamos un libro entero por que es un arma muy poderosa.

Dejamos un link a un tutorial muy interesante: <https://seaborn.pydata.org/tutorial.html>

Vamos a instalar este modulo.

```
>>> $ pip install seaborn
```

Nos puede devolver algo similar a la siguiente imagen, dependiendo del sistema operativo que tengamos. La imagen es sobre un sistema GNU/Linux, basado en distro Debian.

```
(.venv) html/python.desde.cero $ pip install seaborn
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in ./venv/lib/python3.12/site-packages (from seaborn) (2.2.2)
Collecting pandas>=1.2 (from seaborn)
  Using cached pandas-2.2.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (89 kB)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in ./venv/lib/python3.12/site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in ./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Collecting pytz>=2020.1 (from pandas>=1.2->seaborn)
  Downloading pytz-2025.1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas>=1.2->seaborn)
  Downloading tzdata-2025.1-py2.py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
294.9/294.9 kB 4.4 MB/s eta 0:00:00

Using cached pandas-2.2.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.7 MB)
Downloading pytz-2025.1-py2.py3-none-any.whl (507 kB)
507.9/507.9 kB 4.5 MB/s eta 0:00:00
Carpeta personal
Downloading tzdata-2025.1-py2.py3-none-any.whl (346 kB)
346.8/346.8 kB 2.4 MB/s eta 0:00:00

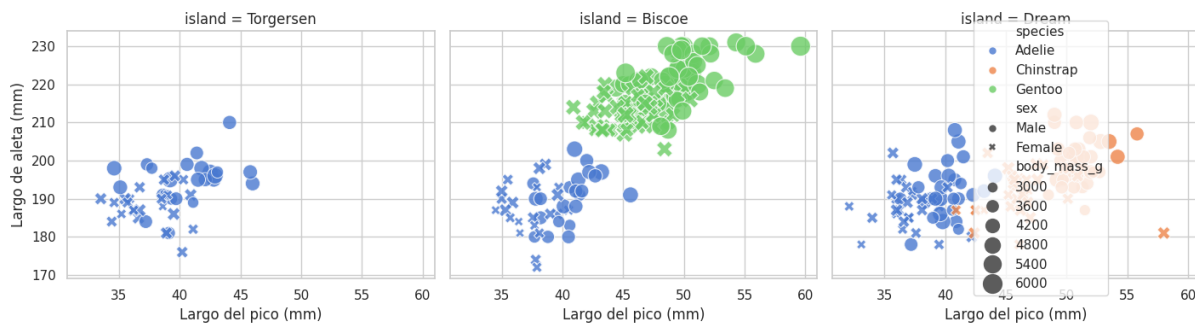
Installing collected packages: pytz, tzdata, pandas, seaborn
Successfully installed pandas-2.2.3 pytz-2025.1 seaborn-0.13.2 tzdata-2025.1
(.venv) html/python.desde.cero $
```

Vamos a dejar un buen ejemplo de la potencia de este módulo. Recuerden que al tener tanta cantidad de funcionalidades, ya para verlo más profundamente, van a necesitar ir a la documentación que dejamos más arriba.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Configuración y datos
>>> sns.set_theme(style="whitegrid", palette="muted")
>>> pinguin = sns.load_dataset("penguins").dropna()
>>>
>>> # Visualización multivariante
>>> g = sns.relplot(
>>>     data=penguin, x="bill_length_mm", y="flipper_length_mm",
>>>     hue="species", size="body_mass_g", style="sex",
>>>     col="island", sizes=(50, 300), alpha=0.8,
>>>     height=4, aspect=1.2, facet_kws={"despine": False}
>>> )
>>>
>>> # Mejoras estéticas
>>> g.set_axis_labels("Largo del pico (mm)", "Largo de aleta (mm)")
```

```
>>> g.fig.suptitle("Relaciones en Pingüinos: Especies, Sexo y Masa >>> Corporal",
y=1.05)
>>> sns.move_legend(g, "upper left", bbox_to_anchor=(0.72, 0.95), frameon=True)
>>>
>>> plt.show()
>>> plt.savefig("relplot_pinguinos.png")
>>>
```



Último ejemplo y con esto ya finalizamos la unidad.

```
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
>>> import pandas as pd
>>> import numpy as np
>>>
>>> # Crear dataset simulando resultados de experimentos científicos
>>> np.random.seed(42)
>>> df = pd.DataFrame({
>>>     'Experimento': np.repeat(['A', 'B', 'C', 'D'], 25),
>>>     'Temperatura': np.random.normal(20, 5, 100),
>>>     'Eficiencia': np.random.uniform(60, 95, 100),
>>>     'Tiempo': np.random.gamma(5, 1, 100)
>>> })
>>>
>>> # Visualización avanzada con un solo gráfico
>>> g = sns.relplot(data=df, x='Temperatura', y='Eficiencia', hue='Experimento',
```

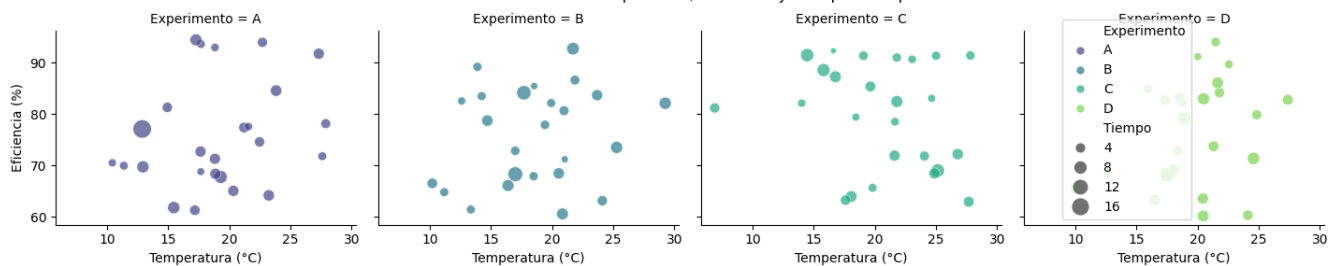


```

size='Tiempo', col='Experimento', kind='scatter', palette='viridis',
height=3, aspect=1.2, sizes=(20, 200), alpha=0.7)

>>>
>>> # Mejoras estéticas
>>> g.set_axis_labels("Temperatura (°C)", "Eficiencia (%)")
>>> g.fig.suptitle("Relación entre Temperatura, Eficiencia y Tiempo en Experimentos",
y=1.05)
>>> sns.move_legend(g, "upper left", bbox_to_anchor=(0.72, 0.95), frameon=True)
>>>
>>> plt.show()
>>> plt.savefig("relplot_experimentos.png")

```



Notas Finales

Este proyecto, cultivado con dedicación a lo largo de años y nutrido día a día con actualizaciones que reflejan el pulso cambiante de la tecnología, nace de dos pasiones fundamentales: el amor por la enseñanza y el compromiso con el crecimiento colectivo en programación.

Si estas páginas han logrado ampliar tu comprensión sobre Python ese lenguaje versátil que sigue transformando nuestra era digital- habrán cumplido su propósito esencial. Cada corrección, cada ejemplo y cada ejercicio fueron diseñados como puentes entre el conocimiento teórico y su aplicación vibrante.

Acepto con gratitud sugerencias y diálogos constructivos en **juanpablososa@gmail.com**. Este libro no es un punto final, sino un testimonio de un viaje en constante evolución... ¡y tú eres parte fundamental de esta travesía!

Que el entusiasmo por aprender nunca deje de impulsarte. Sigamos codificando futuro, línea a línea, con esa curiosidad que nos hace grandes.

Con profundo respeto y gran abrazo,
Profe Juan Pablo Sosa